

物流信息管理汇总作业



学 号:	<u>2220232872</u>
姓 名:	<u>栾智健</u>
指导教师:	<u>孙卓</u>
报告时间:	<u>2026 年 1 月 10 日</u>

目录

1 作业 1	1
1.1 阶段 1	1
1.2 阶段 2	1
1.3 阶段 3	2
2 作业 2	3
2.1 基础版	3
2.2 创新版	4
3 作业 3	5
3.1 创新版	5
4 作业 4	7
4.1 创新版	7
5 作业 5	10
5.1 创新版	10
6 作业 6	11
6.1 问题 1（创新版）	12
6.2 问题 2	13
6.3 问题 3	15
6.4 总结与反思	16
7 作业 7	16
7.1 数学模型	17
7.2 求解可视化	19
8 作业 8	20
8.1 问题 1	20
8.2 问题 2	23
9 作业 9	24
9.1 深度优先遍历	24
9.2 广度优先遍历	28
10 作业 10	31
10.1 问题 1	31
10.2 问题 2	34
10.3 结论	37
11 作业 11	37
11.1 问题 1	37
11.2 问题 2	39
11.3 问题 3	43
12 作业 12	46
12.1 算法理解	47
12.2 代码逻辑	48
12.3 创新版本	50
13 作业 13	53
13.1 代码整体架构与设计思路	53
13.2 三种基础算法的实现逻辑	53
13.3 算法创新点	53
13.4 代码链接	54
13.5 结果展示图	54
14 作业 14	54
14.1 基础版：3x2 网格的随机游走与最短路	54
14.2 创新版：4x4 网格的全路径遍历与可视化	55
15 作业 15	57
15.1 基础版	57
15.2 创新版	58
16 作业 16	60
16.2 创新版一	62
16.3 创新版二	64
16.4 结论与思考	65
总体感想与思考	66

1 作业 1

问题：以原创诗歌生成二维码

1.1 阶段 1

在学期初，我直接使用了草料二维码，因为操作简单。但是我在草料里面对我的这个二维码的内容做了美化美观，如图：



我觉得这样远远不够，没有体现出我的探索能力。

1.2 阶段 2

后来，在老师带领一下学习了 Lua，也学习了如何在 MicroCity 中实现简单的代码。我就想把诗歌放在 MicroCity 中打印出来，然后把这个网站转换成二维码。

在 11 月份的时候，微信还可以直接扫描出 MicroCity 这个网站。后来，给封掉了，我申诉了几次

都没成功，就放弃了申诉这条道路了…。但是其实，把 MicroCity 导入到二维码中最大的问题就是，MicroCity 生成出来专属的链接太长了，基本都导不进去（我没找到能导入进去的方法）。



1.3 阶段 3

最后我想的就是把生成过程录制成视频导入到二维码中，这个二维码中的内容，既有视频，又有图片。微信直接扫描可以打开里面内容，不需要在什么复制文本。





物流信息管理作业1-栾智健

```
MicroCity Web
5  "指上代码生锦绣，云端算策定星河。"
6  "且看银网通天下，瞬息风雷送万货。"
7  }
8  -- 2. 控制节奏，模拟真实阅读速度
9  local function sleep(n)
10     local t0 = os.clock()
11     while os.clock() - t0 <= n do end
12 end
13 -- 3. 主程序入口
14 function main()
15
16     print("《物流信息行》")
17     print("作者：栾智健 2220232872")
18     print("-----")
19     sleep(1.0) -- 稍作停顿，准备开始
20     -- 遍历数组直接打印内容
21     for _, line in ipairs(poem_lines) do
22         -- 直接输出每句本身，没有任何修饰
23         print(line)
24         -- 每一句念完停顿1.5秒，节奏感
25         sleep(1.5)
26     end
27     print("-----")
28 end
29 -- 4. 执行
30 main()
31
```

[2026/1/8 09:24:31]-Embedded project is loaded!
 [2026/1/8 09:24:30]-All changes has been saved to 作业1 (诗歌) .mw!
 [2026/1/8 09:24:35]>《物流信息行》
 [2026/1/8 09:24:35]>作者：栾智健 2220232872
 [2026/1/8 09:24:35]>-----
 [2026/1/8 09:24:36]>辽海临风望太虚，万艘竞渡泛沧波。
 [2026/1/8 09:24:37]>千箱堆叠如山峙，谁遣神兵化坎坷？
 [2026/1/8 09:24:37]>指上代码生锦绣，云端算策定星河。
 [2026/1/8 09:24:37]>且看银网通天下，瞬息风雷送万货。
 [2026/1/8 09:24:37]>-----

```
cached code is loaded.
> The 作业1 (诗歌) .mw has been opened!
> 《物流信息行》
> 作者：栾智健 2220232872
> -----
> 辽海临风望太虚，万艘竞渡泛沧波。
> 千箱堆叠如山峙，谁遣神兵化坎坷？
> 指上代码生锦绣，云端算策定星河。
> 且看银网通天下，瞬息风雷送万货。
> -----
```

2 作业 2

问题：编写寻找 1-100 中所有质数的 lua 代码

2.1 基础版

2.1.1 代码思路

在基础版中，我采用了最试除法的逻辑，遍历并筛选数字。

- (1). 双层循环结构：外层循环遍历 2 到 100 的自然数，作为待检测对象。
- (2). 假设与验证：对每个数 n ，首先默认其为质数（`prime=true`）。随后进入内层循环，从 2 开始尝试除每一个数。
- (3). 设定边界：这里我设定了试除的上限为 $0.5n$ 。虽然老师课上所讲利用开根号的方式更优，但我想的是优先保证直观的逻辑，保证我以后还能看懂这段代码。
- (4). 整除判断：利用取模运算 `%` 判断是否整除。一旦发现任何因子，立即将布尔值设为 `false` 并跳出循环。
- (5). 结果输出：循环结束后，为 `true`，则输出该数字。

2.1.2 代码链接

[MicroCity Web](#)

2.1.3 结果展示图

```
[2025/12/18 08:30:20]>2
[2025/12/18 08:30:20]>3
[2025/12/18 08:30:20]>5
[2025/12/18 08:30:20]>7
[2025/12/18 08:30:20]>11
[2025/12/18 08:30:20]>13
[2025/12/18 08:30:20]>17
[2025/12/18 08:30:20]>19
[2025/12/18 08:30:20]>23
[2025/12/18 08:30:20]>29
[2025/12/18 08:30:20]>31
[2025/12/18 08:30:20]>37
[2025/12/18 08:30:20]>41
[2025/12/18 08:30:20]>43
[2025/12/18 08:30:20]>47
[2025/12/18 08:30:20]>53
[2025/12/18 08:30:20]>59
[2025/12/18 08:30:20]>61
[2025/12/18 08:30:20]>67
[2025/12/18 08:30:20]>71
[2025/12/18 08:30:20]>73
[2025/12/18 08:30:20]>79
[2025/12/18 08:30:20]>83
[2025/12/18 08:30:20]>89
[2025/12/18 08:30:20]>97
```

2.2 创新版

根据老师提供可选创新，我对孪生质数进行搜索，并且在这过程的当中，利用一个更高效的方法。

2.2.1 代码逻辑

首先，定义 `isPrime(n)` 函数。

在函数内部，首先排除小于 2、能被 2 整除、能被 3 整除的基础情况。

在后面的循环中，利用“所有大于 3 的质数都分布在 $6k-1$ 和 $6k+1$ 位置上”这一规律，循环变量 i 从 5 开始，每次步进 6。每次仅需检查 i 和 $i+2$ 两个位置是否能整除 n ，从而跳过了大量明显为合数的检查（如 2, 3 的倍数）。

主循环利用“孪生质数对必须包含奇数”的特性，从 3 开始按步长 2 递增 13。同时检测 p 和 $p+2$ 是否均为质数，若满足则输出这对数。

2.2.2 创新点

创新点一：老师给定可选创新：寻找孪生质数。

创新点二：在孪生质数的代码编程过程中，寻找数学规律：质数总是以 $6k \pm 1$ 的形式出现。这意味着我只需要检测 $n \% i$ 和 $n \% (i+2)$ ，并在迭代中执行 $i = i + 6$ 。这种方法直接跳过了所有 2 和 3 的倍数。

创新点三：降低了时间复杂度。将循环终止条件设定为 $i * i \leq n$ 。这使得算法的时间复杂度从 $O(n)$ 降低到了 $O(\sqrt{n})$ 。

创新点四：我观察到，除了(3,5)之外，所有孪生质数对必然是(奇数, 奇数+2)。因此，我将外层循环的步长设为 2（只遍历奇数），直接减去了一半的无效判断。

2.2.3 代码链接

[MicroCity Web](#)

2.2.4 结果展示图

```
[2025/12/18 08:40:04]>3 5
[2025/12/18 08:40:04]>5 7
[2025/12/18 08:40:04]>11 13
[2025/12/18 08:40:04]>17 19
[2025/12/18 08:40:04]>29 31
[2025/12/18 08:40:04]>41 43
[2025/12/18 08:40:04]>59 61
[2025/12/18 08:40:04]>71 73
```

3 作业 3

问题：绘制航线图并且可以在 MicroCity 中显示。

3.1 创新版

先在 geojson 上绘制航线，绘制的航线没有溢出地图限制，因为我发现 geojson 有一条很明显的边界，只要不画过去就不会出地图。随后，我发现直接把 geojson 的航线复制过去，整个地图单调，没有信息价值，所以我做了些创新，让航线、地图都带有颜色。

3.1.1 代码逻辑

在代码中，并不是对每一个板块，一点点赋予颜色，或者是在 geojson 上一点一点绘制颜色，而是编写几个函数，自动赋予颜色。

- (1). 编写数据判断函数：我编写了 `getCountryColor`、`getLineWeight` 等函数，对人口和 GDP 都进行了限制（如人口>1 亿或 GDP>100 万）。程序会自动读取 geojson 中的属性数据，通过多层 `if-elseif` 逻辑判断，将国家划分为“主要、重要、中等、小、微型”五个等级，并动态返回对应的颜色代码（红、青、蓝、绿、黄）。
- (2). 保证标签的定位：在一开始的尝试当中，我发现标签位置偏移的问题，我分析是直接取第一个顶点导致标签在地图边界上。所以我编写了 `calculateCenter` 函数。该函数采用了几何质心算法，通过遍历地块所有顶点的 (x,y,z) 坐标并求取平均值，计算出多边形的物理中心，确保标签能精准落在板块中央。
- (3). 对航线赋予颜色：针对航线网络，我编写了独立的遍历循环。利用 `math.random(50, 255)` 在 RGB 颜色空间内动态生成随机颜色值，并将其赋予每一组航线。这使得原本混杂在一起的航运网络瞬间变得条理清晰，每条航线的独立性得到了视觉上的确认。避免了混乱、无法辨别哪一条航线是哪一条航线的问题。

3.1.2 创新点

创新点一：我发现了 geojson 里面是可以存储数据的。所以我调用了其中的数据作为地图颜色绘制的基础，依据各地区的人口规模与 GDP，实现了从“鲜红（核心枢纽）”到“金黄（边缘节点）”的五级色谱绘制。

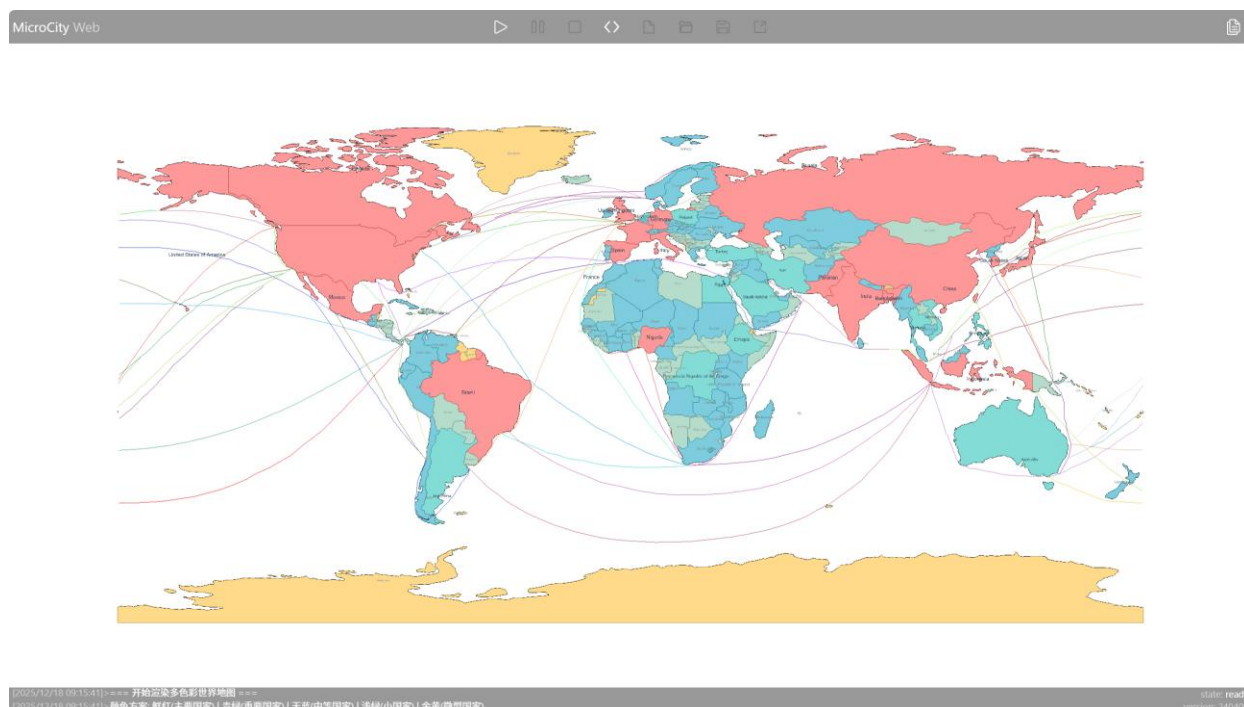
创新点二：使用函数辅助标签定位。一开始的尝试中，国家名称显示在边境线上。随后我利用了一个多边形质心计算函数。

创新点三：一开始我直接导入的航线，我想对着地图沿着一条航线一直走下去看看他到底到哪，但是跟着跟着就找不到了，因为都是黑色的。随后我设计了一个随机着色的功能，为每一组航线生成独立的 RGB 签名。

3.1.3 代码链接

[MicroCity Web](#)

3.1.4 结果展示图



4 作业 4

问题：以不同的颜色对各个国家着色并且改变标注的文字。

4.1 创新版

本次作业我没有采用传统的、平面的、静态的地图，而是学习了一下 MicroCity 上的案例，复现出了能够同时展示 GDP（经济维度）和人口（社会维度）的三维动态可视化系统。

4.1.1 代码逻辑

为了实现从原始 GeoJSON 数据到三维动态场景的转化，我学习后构建了三个核心部分：

(1). 数据整理和语句转换

Geojson 中的原始数据是英文。我首先建立了一个哈希映射表，把国家的名称转换为中文。

同时，在遍历 `getchildren()` 时，我编写了严格的数据清洗逻辑：利用 `tonumber` 和逻辑判断 `if name ~= "" and gdp > 0`，自动过滤掉数据缺失或无效的图层，防止空指针报错。

(2). 多维视觉映射算法

这是这段代码的核心。通过对 MicroCity 上的案例的学习，我摒弃了一开始线性的可视化的思想（就是每一个板块上面有一个柱体），采用了更美观的，实用的非线性算法：

- a) 几何计算：针对标签位置偏差问题，与问题 3 一致，我编写了 `calculateCenter` 函数，通过遍历多边形所有顶点并计算算术平均值，求得任意形状国家的几何质心，确保标签精准定位在板块正上方。
- b) 高度拟合：在 GDP 可视化上，为了避免贫富差距过大导致柱体比例失调（如美国过高

而小国看不见)，我采用了平方根平滑处理 $\text{math.sqrt}(\text{gdp} / \text{maxGDP})$ ，将巨大的数值差异减小压缩在合理的可视的区间内。

- c) 色彩插值：在人口着色上，我实现了了色彩空间的插值算法。通过 `colorByPopulation` 函数，根据人口密度计算归一化系数 t ，动态调整颜色从蓝色（210）渐变至橙红色（20），并且一同调整饱和度与亮度，实现数据的直观表现。

(3). 动态场景与渲染

为了解决实现的柱体遮挡地图的问题，我在 `scene.render()` 循环中引入了正弦波时间函数 $\text{math.sin}(\text{time})$ 。通过实时计算 `alpha` 通道值，让柱体不断的变化透明度（0.1~0.8）。这不仅增加了视觉的动感，更实现了透视的效果，让用户透过柱体依然能看清底部的地图轮廓，以便对地图的阅读。

4.1.2 创新点

创新点一：三维映射

通过对 `MicroCity` 的学习，我想打破传统地图单一维度的限制，采用“高度-色彩”可视化。利用柱体的高度（Z 轴）物理表征 GDP 总量，同时利用柱体的颜色热力表示人口规模。让读图者可以快速读图，同时对比出一个国家的经济实力与人口压力（例如：某国柱体很高但颜色很蓝，说明其人均经济极高；某国柱体低但颜色红，说明发展压力大）。

创新点二：减少信息输出打印

为了避免全球两百多个国家的数据造成画面杂乱，影响地图的可读性。我实现了一个函数，自动计算并提取前三名 GDP 和前八名人口，仅对这些关键节点赋予标签。

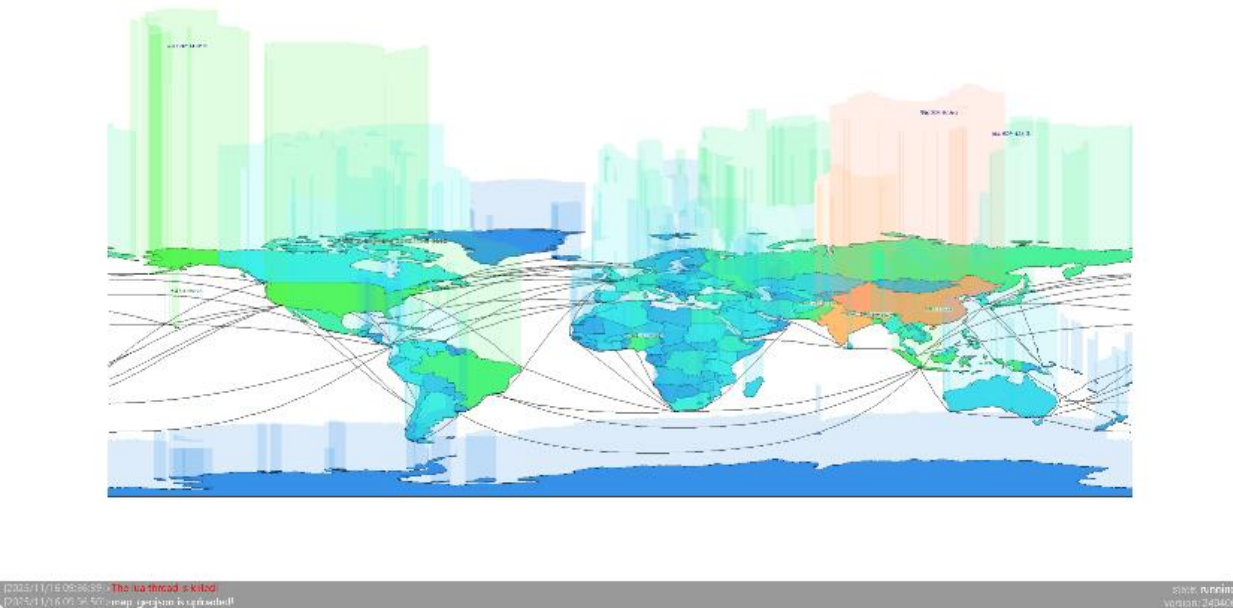
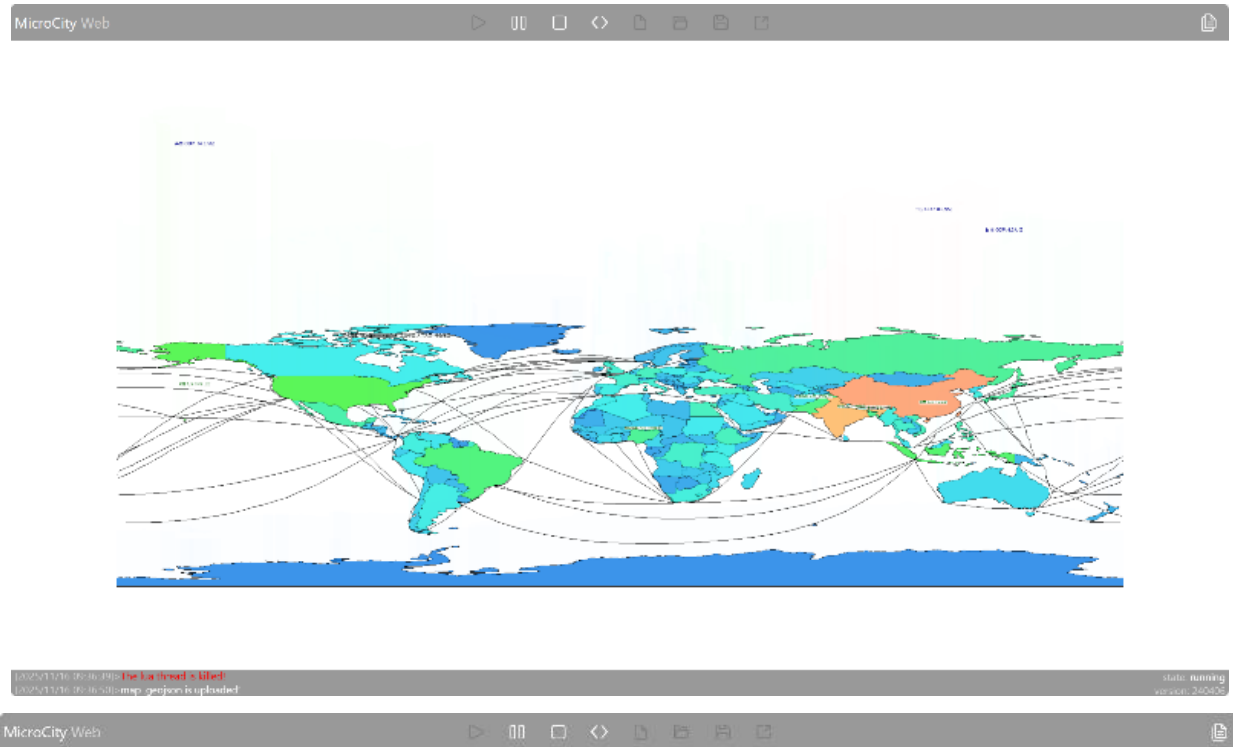
创新点三：柱体不断的动态变化

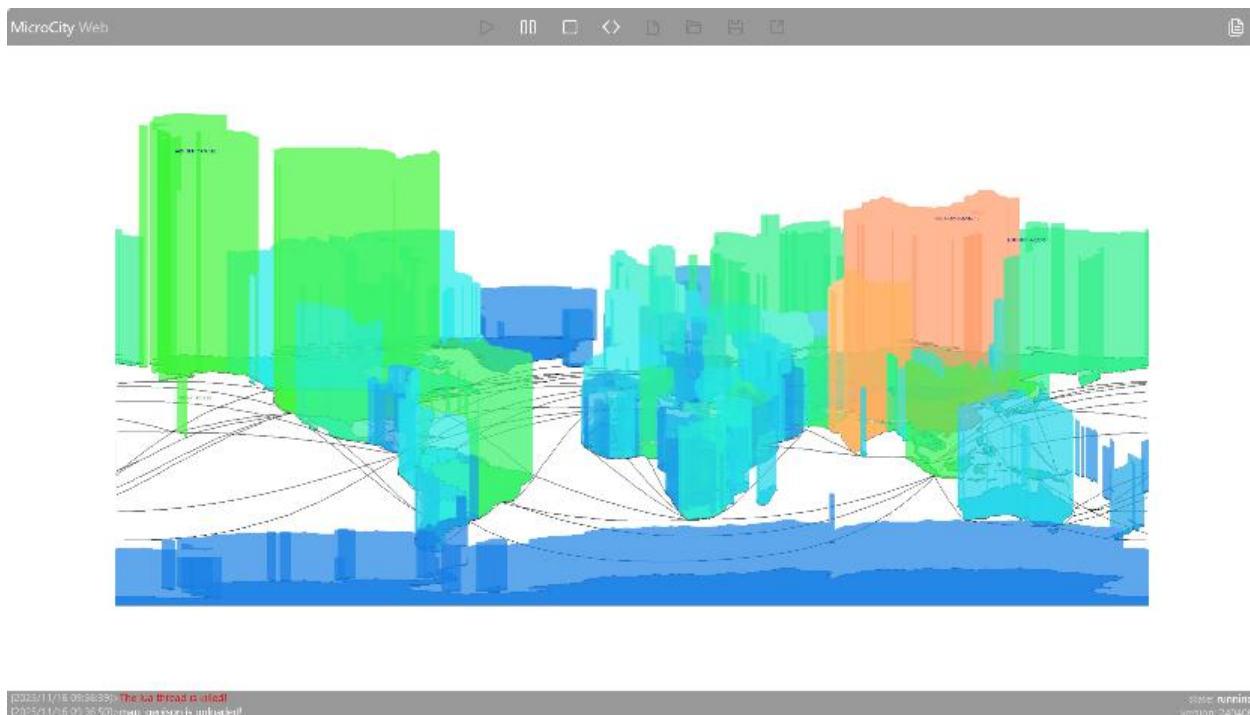
我引入了时刻变化的动画。通过将柱体材质设置为动态透明度，柱体呈现出周期性的“呼吸”效果。通过透明度的周期性降低，可以透过高大的柱体看到被遮挡的后方地图细节。

4.1.3 代码链接

[MicroCity Web](#)

4.1.4 结果展示图





5 作业 5

问题：找出并且输出与中国接壤进口大于出口的国家，并且输出进出口值

5.1 创新版

在本次作业中，我没有简单的调用 `geojson` 中的数据去判断与中国接壤进口大于出口的国家并且输出。而是利用 `lua` 代码编写了一套可以分析国家之间关系，计算贸易差额并且最终进行可视化的地图。

5.1.1 代码逻辑

先整理数据，再判断数据之间逻辑，最后进行可视化的处理流程：

- (1). 同前几次作业一致，先对各个国家的名字进行规范。代码通过逻辑或 `or` 运算符检查多个可能的字段名（如 `tonumber(data['IMPORT']) or tonumber(data['进口'])`）。
- (2). 我没有只把找到的数据 `print` 出来，而是在循环中嵌入了计算逻辑。系统实时执行 `tradeBalance = import - export` 运算。通过逻辑判断 `if isNeighbor and import > export`，进行筛选：只有同时满足“相邻”且“存在贸易逆差”两个维度的国家，才会被进入到输出并且可视化之中。

5.1.2 创新点

创新点一：对于相邻关系的判断，我通过将非结构化的几何数据与结构化的贸易数据、地缘关系数据相结合，不是盲目的做可视化，而是“理解”了谁是中国，谁是邻国，谁在经济上存在逆差。再进行输出判断。

创新点二：可视化的创新

为了在三维场景中突出重点，我设计了不同的颜色突出各个国家。

- (1). 将中国渲染为半透明黄色（Yellow, Opacity 0.6）；
- (2). 将符合“进口大于出口”条件的邻国渲染为半透明红色（Red, Opacity 0.7）。这种色彩鲜明的对比，可以快速判断出所求国家区域。

创新点三：代码改进过程的改进

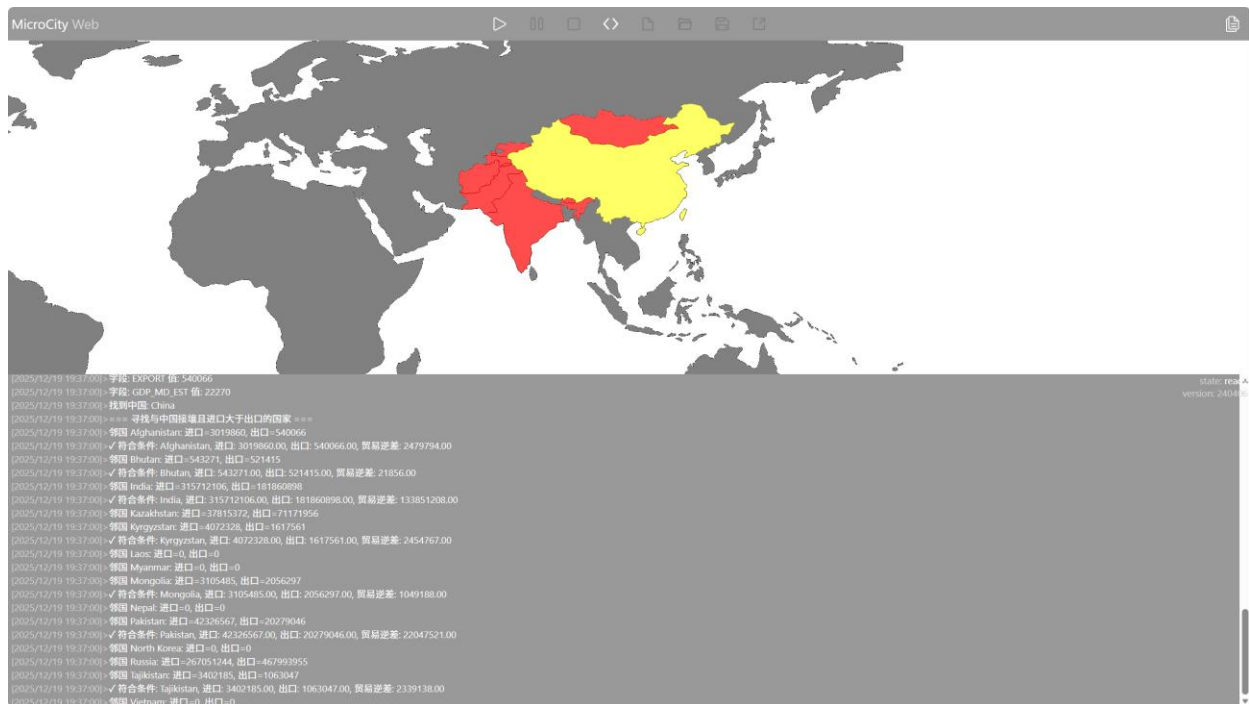
在编写代码时，我遇到了可视化不显示的问题，通过老师课上所讲，对代码进行断点处理检查，但是我意识到如果代码太长了很浪费精力。所以我做了如下工作：

- (1). 预检机制：代码运行之初会打印样本数据结构 `print("字段:", k, "值:", v)`，实现数据源自检。
- (2). 异常捕获：如果 `foundCountries == 0`，系统会自动触发 `fallback` 机制，打印出所有有贸易数据的国家列表以及可能的原因分析（如字段缺失或定义不准）。

5.1.3 代码链接

[MicroCity Web](#)

5.1.4 结果展示图



6 作业 6

问题：① 计算航线在地球球面上的长度② 修改重心法，利用迭代重心法求解简单问题③ 使用 MicroCity 求解如下选址问题，注意球面的环绕特性。

国家	需求量 (万吨/年)
中国	60
美国	45
巴西	20
德国	30
澳大利亚	15

6.1 问题 1（创新版）

在问题 1 中，我主要完成了关于地球球面距离的计算，目标是计算北京与其他国际城市之间的球面距离，并且进行可视化。同时为了让地图更加美观，我改变了航线的颜色。

6.1.1 代码逻辑

在前几次的作业的基础上，加上了根据球面的距离对航线的计算以及对航线数据的输出。

(1). 对于地图着色与航线变色功能与之前一致，这里简单叙述

考虑到单一指标的局限性，我编写了一套复合判断逻辑 `getCountryColor` 和 `getLineWeight`。代码读取人口和 GDP 两个关键字段。通过 `if pop > 1 亿 or gdp > 1 万亿` 这样的多级 `if-elseif` 筛选漏斗，将国家划分为五个等级。根据等级分配不同颜色，实现了地图信息的自动化分层展示。

(2). 计算真实球面距离

考虑到球面的特性，我首先将经纬度转换为三维空间坐标 (x, y, z) 。利用公式 $x = R \cos(\text{lat}) \cos(\text{lon})$ 等，将坐标转换到三维场景当中，以便绘制城市节点。同时利用反余弦函数 `math.acos` 结合三角函数公式实现球面距离计算公式。

(3). 可视化输出

分别统计了总航线数量，总球面距离，平均航段程度。遍历城市列表，计算距离。但是最后只输出几大著名航线的计算，以及统计其中的最长航线和最短航线。

6.1.2 创新点

我在完成基础地图绘制中的创新点，在本次作业便不做解释，本次作业的创新点如下：

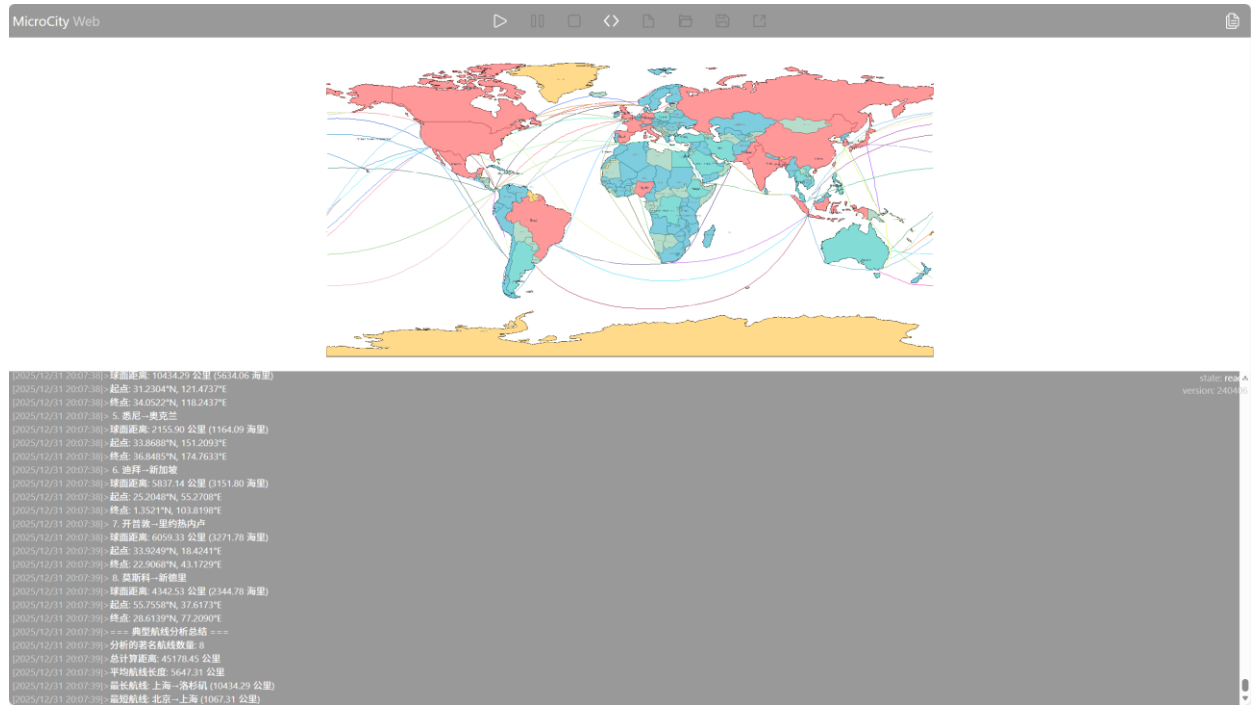
创新点一：计算城市间距离，考虑到了球面球面特性。

创新点二：航线颜色的转换。

6.1.3 代码链接

[MicroCity Web](#)

6.1.4 结果展示图



6.2 问题 2

本问题中，我编写代码实现利用迭代重心法求解欧式距离最小化问题，寻找使得总运输成本最小的设施坐标。

6.2.1 代码逻辑

首先对场景进行搭建，对每一个客户点根据题目要求赋予权重。

随后，随机选择一个初始解 (x, y) ，构建一个 `while` 循环，再每一次迭代中，计算当前设施位置到所有客户点的欧氏距离 d_i 。根据迭代公式 $x_{new} = \frac{\sum(w_i x_i / d_i)}{\sum(w_i / d_i)}$ 更新设施坐标。

最后，引入 `diff` 变量记录坐标变化的距离，当变量小于阈值时自动停止迭代。

6.2.2 创新点

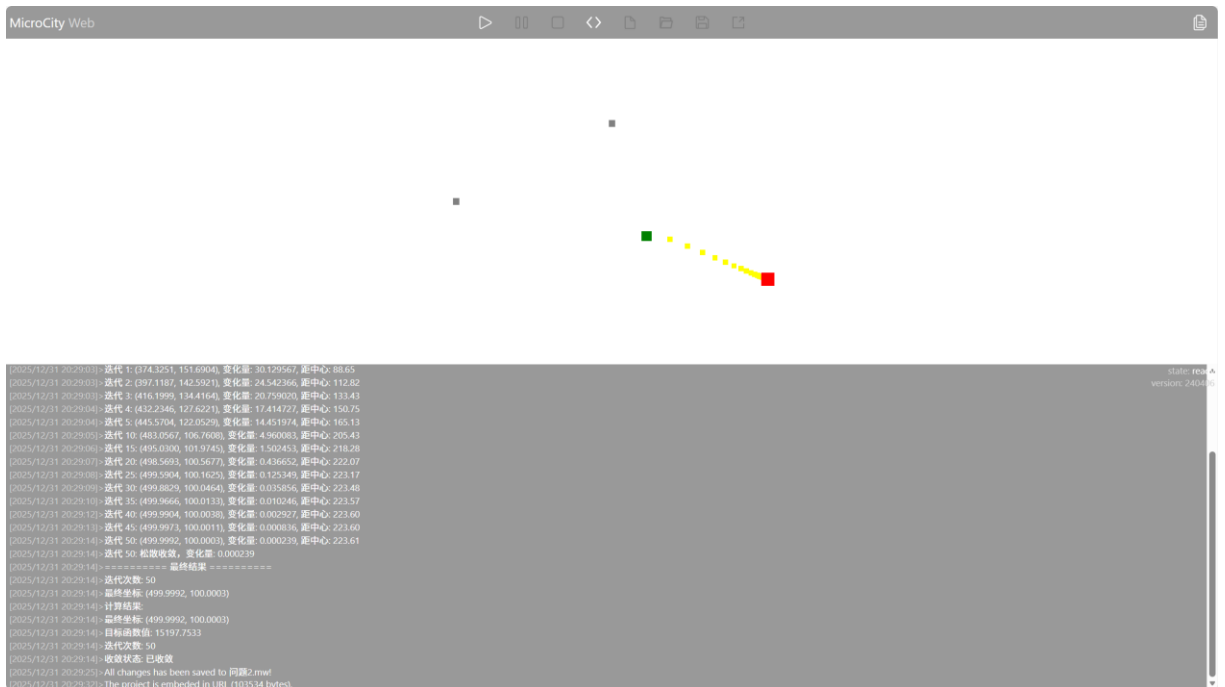
创新点一：实现迭代重心法的可视化，清晰的展示了迭代的过程，这为我后续进一步探索迭代重心法做了铺垫。

创新点二：为防止迭代公式的分母为 0，或者迭代无线循环，我加入了一个极小的阈值保证了结果的高精度。

6.2.3 代码链接

[MicroCity Web](#)

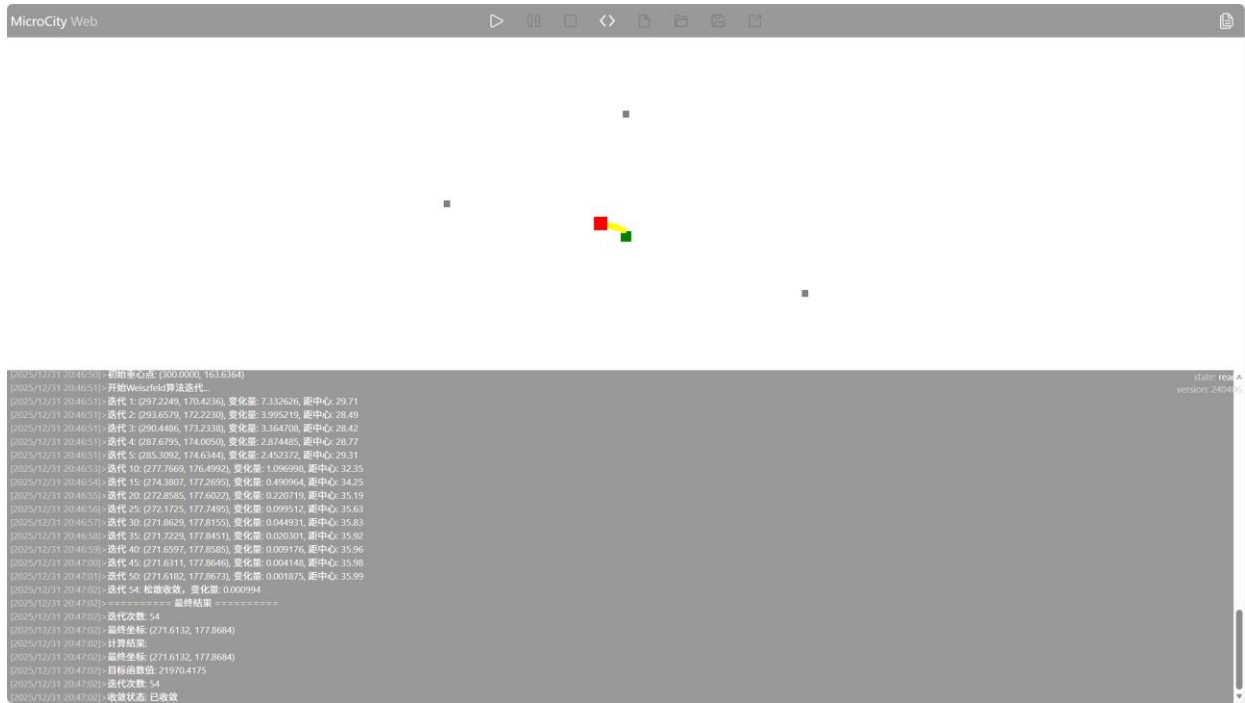
6.2.4 结果展示图



6.2.5 发现与感悟

我在运行结果后发现，最后的选址一直偏向于某个点，经过仔细的理解和询问老师后得知：这是由该客户点的需求权重大所导致的。重心法最小化的是距离的平方和。平方会放大长距离的惩罚，所以重心法给出的结果是一个“折中”的位置，位于所有点的中间，不会完全偏向某一个点。而迭代重心法修正重心法的“错误”，试图最小化欧氏距离和（而不是平方）。算法发现：“只要我往右下角走，总成本就会下降”。

我通过调整参数，验证了这个原理。同时我也在思考如果两个客户的权重一样会发生什么，如图所示：



很明显，在两个权重相同的点的中间位置附近。

6.3 问题 3

本次作业为求解环绕特性选址的问题，我决定利用 MicroCity 中自带的地球，进行选址问题的求解。

本次作业为求解环绕特性选址的问题，我决定不再使用平面地图进行近似，而是调用 MicroCity 自带的地球模型进行仿真。在真实的球面上，利用经纬度数据构建城市节点，并通过改进的迭代算法求解球面上的最优设施选址坐标。

6.3.1 代码逻辑

首先加载地球模型，定义问题中的关键国家的真实经纬度数据，将坐标转换，利用小圆球定义城市。

因为是在三维空间中计算，我定义了一个函数，利用反余弦公式计算两点间的球面距离，确保优化的目标是地表航行距离而不是穿越地心的直线距离。

随后进行迭代算法，随机生成一个初始点，然后开始迭代。在每一步迭代中，我首先类比平面重心法，计算三维空间中所有需求点对当前点的“引力”合成向量，得到一个新的临时坐标。由于直接合成的新坐标可能会落入地球内部，我在代码中加入了归一化的步骤：计算原点到新坐标的距离 len ，然后将新坐标除以 len 并乘以地球半径 100。

最后同样是设置一个阈值，以免无线循环。

6.3.2 创新点

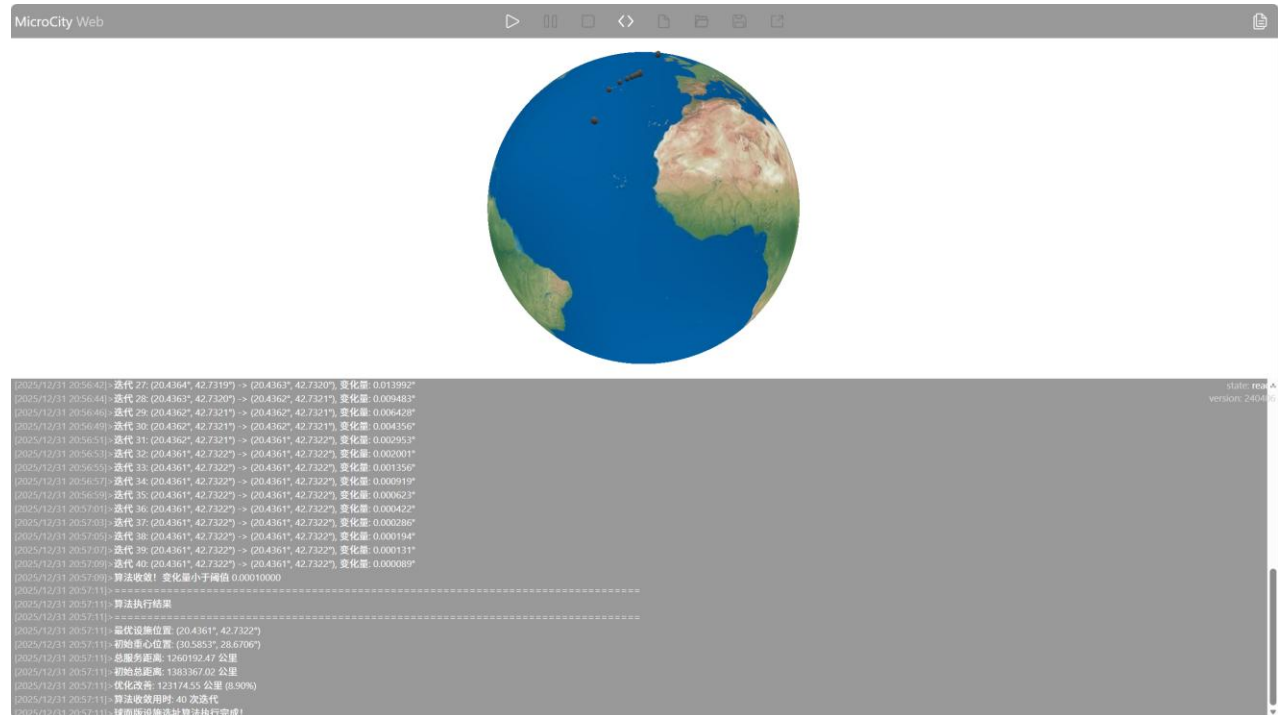
创新点一：不同于直接利用迭代重心法，把重心法求出的位置转换为球面坐标，也就是“迭代+投影”，有效地解决了球面上的无约束优化问题。

创新点二：可视化的选取直接利用球体，凸显出我对于球面特性的考虑所在。将“地球”、“城市分布”、“最优解”和“迭代过程”直观地展示了出来。让迭代过程更加明显。

6.3.3 代码链接

[MicroCity Web](#)

6.3.4 结果展示图



6.4 总结与反思

通过本次作业的完成，我有以下收获感悟：

- (1). 把数学公式转换成可以运行的代码，我认为这是一个从数学建模到数学仿真的过程。
- (2). 在具体问题分析当中，我们必须要考虑球面的环绕特性，基于实际的场景进行分析。
- (3). 我在解决问题时，通常的思维是想着怎么直接去解决问题，但是在问题三中，我偶然发现了 MicroCity 中自带的地球模型。这启示了我在以后的作业里，要对整体的问题以及实用工具做一个彻底的学习把控再进行代码的编写完成去解决问题，会有事半功倍的效果。

7 作业 7

问题：对下图问题建模求解可视化。

练习问题

目标函数：考虑建设成本（使用期100年）和运输成本之和最小

- 约束：
- 1、至少建设3个站点
 - 2、每个客户点至少有一个站点为其服务
 - 3、每个客户有最小需求，每个站点有最大供应

注意：如果有多个站点为一个客户点服务，那么客户的需求该如何分配？

国家	最小需求量 (万吨/年)	候选港口	最大供应量 (万吨/年)	建设成本 (亿美元)
中国	60	黑德兰港	150	20
美国	45	宁波舟山港	120	18
巴西	20	鹿特丹港	90	15
德国	30	桑托斯港	55	11
澳大利亚	15	休斯顿港	50	9

本次作业的目标是进行离散选址决策。我首先建立 MILP，随后利用贪心策略编写 lua 代码。

7.1 数学模型

在建模阶段，我想起老师课上所提出：如果有多个站点为一个客户点服务，那么客户的需求该如何分配的问题。

我的解决方案是通过定义决策变量 x_{ij} 为连续变量，而非二元变量。这样，线性规划求解器会自动根据成本 c_{ij} 和容量约束 M_j ，决定将客户 i 的需求 D_i 是由一个港口独占，还是分拆给多个港口，从而自动解决了需求分配问题。

7.1.1 索引

$i \in I$ (客户集合: {中国, 美国, 巴西, 德国, 澳大利亚})

$j \in J$ (候选站点集合: {黑德兰港, 宁波舟山港, 鹿特丹港, 桑托斯港, 休斯顿港})

7.1.2 参数表

建模设计参数如表所示：

参数	描述	单位
D_i	客户 i 的年最小需求量	万吨/年
M_j	候选站点 j 的年最大供应量	万吨/年

F_j	建设站点 j 的成本	亿美元
c_{ij}	从 j 向 i 运输 1 单位的成本	亿美元/万吨
T	规划周期	100 (年)
B	可用的总建设预算上限	亿美元
k	每个客户所需的最小覆盖设施数	个

7.1.3 决策变量

决策变量如表所示：

变量	描述	类型
y_j	如果在候选点 j 建设站点，则为 1，否则为 0	二元 (0/1)
x_{ij}	每年 从站点 j 运输到客户 i 的货物量	连续 (非负)
z_{ij}	如果站点 j 覆盖客户 i ，则为 1，否则为 0	二元 (0/1)

7.1.4 目标函数

最小化 100 年规划周期内的建设成本与总运输成本：

$$\text{Min } Z = \sum_{j \in J} F_j \cdot y_j + T \cdot \left(\sum_{i \in I} \sum_{j \in J} c_{ij} \cdot x_{ij} \right)$$

7.1.5 约束条件

基础约束 1（最小设施数）：至少建设 3 个站点。

$$\sum_{j \in J} y_j \geq 3$$

基础约束 2（需求满足）：客户需求必须被满足。

$$\sum_{j \in J} x_{ij} \geq D_i (\forall i \in I)$$

基础约束 3（最大供应能力）：运输量受限于建设状态和最大供应量。

$$\sum_{i \in I} x_{ij} \leq M_j \cdot y_j (\forall j \in J)$$

新增约束 1（建设预算约束）：考虑总投资上限 B 。

$$\sum_{j \in J} F_j \cdot y_j \leq B$$

新增约束 2（ k -覆盖约束）：提高供应链韧性，要求每个客户至少被 k 个有效设施覆盖范围指派。

$$\sum_{j \in J} z_{ij} \geq k (\forall i \in I)$$

新增约束 3（逻辑连接）：只有建设了站点才能通过覆盖检验。

$$z_{ij} \leq y_j (\forall i \in I, j \in J)$$

变量约束：

$$y_j \in \{0,1\} (\forall j \in J)$$

$$z_{ij} \in \{0,1\} (\forall i \in I, j \in J)$$

$$x_{ij} \geq 0 (\forall i \in I, j \in J)$$

7.2 求解可视化

7.2.1 代码逻辑

为将上述模型求解可视化，我编写了 lua 代码进行实现。

首先，利用前面所学知识完成对国家坐标的提取，数据的定义等基本工作。

随后，利用欧几里得距离计算成本，结合定义的运输系数（0.001 亿美元/万吨/单位距离）生成成本矩阵。

考虑到 MILP 的复杂性，我选取贪心算法，嵌套一个循环，每次迭代遍历所有没被选择的港口，然后计算引入该港口后对整个系统的成本贡献，选择收益最大的港口加入解集。

最后利用前几次作业的基础完成简单的可视化。

7.2.2 创新点

创新点一：利用前几次的可视化成果完成本次作业的可视化，将所要求解的地点全部在地图上进行可视化。

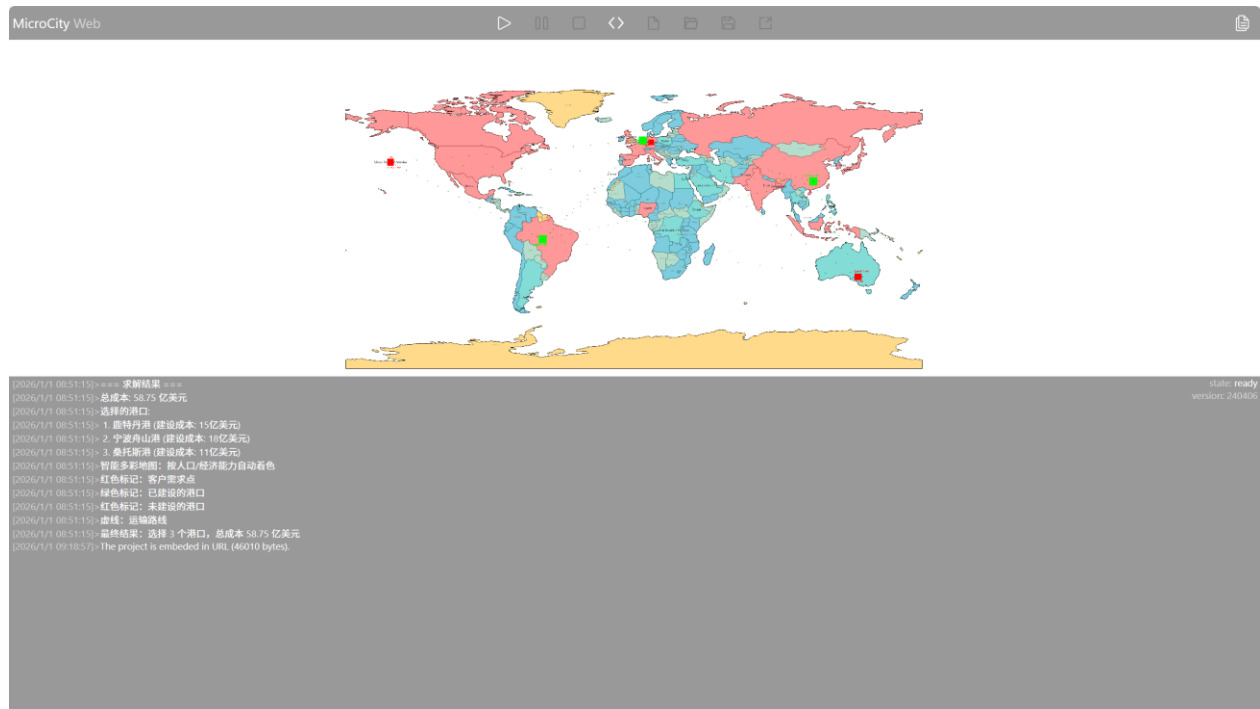
创新点二：添加三条约束，通过精妙的设计数学模型，解决“如果有点为一个客户点服务，那么客户的需求该如何分配的问题”。

创新点三：对客户点与选址点利用不同颜色绘制。

7.2.3 代码链接

[MicroCity Web](#)

7.2.4 结果展示图



8 作业 8

问题：① 试评 Weiszfeld 算法的算法复杂度；② 对航线进行拓扑化。

8.1 问题 1

根据课上所讲内容，Weiszfeld 算法本质上是一种迭代下降算法。

我先对算法复杂度进行**数学分析**，随后再利用**编程代码验证**。

8.1.1 数学分析

我将从时间复杂度和空间复杂度两个方面进行评估。

一、时间复杂度

算法的总时间复杂度由单次迭代的复杂度和收敛所需的迭代次数共同决定。

(一) 单词迭代的时间复杂度： $O(nd)$

算法的核心迭代公式为：

$$x^{(k+1)} = \frac{\sum_{i=1}^n \frac{w_i x_i}{|x^{(k)} - x_i|}}{\sum_{i=1}^n \frac{w_i}{|x^{(k)} - x_i|}}$$

为了计算 $x^{(k+1)}$ ，在每一次迭代中，算法必须执行以下操作：

1. 遍历所有 n 个点：算法需要对 $i = 1$ 到 n 的所有点进行循环。
2. 计算内部项：在循环内部，对于每一个点 x_i ：

- a) 计算距离：计算当前迭代点 $x^{(k)}$ 与 x_i 之间的欧几里得距离 $|x^{(k)} - x_i|$ 。这涉及 d 维向量的减法、 d 次平方、 $d-1$ 次加法和1次开方。此操作的时间复杂度为 $O(d)$ 。
 - b) 计算分子项： $\frac{w_i x_i}{|x^{(k)} - x_i|}$ 。这是一个标量 $w_i/|\dots|$ 与一个 d 维向量 x_i 的乘法。时间复杂度为 $O(d)$ 。
 - c) 计算分母项： $\frac{w_i}{|x^{(k)} - x_i|}$ 。这是一个标量计算。时间复杂度为 $O(1)$ 。
3. 求和：
 - a) 分子求和：将 n 个 d 维的分子向量累加起来。总复杂度为 $O(nd)$ 。
 - b) 分母求和：将 n 个标量的分母项累加起来。总复杂度为 $O(n)$ 。
 4. 最终除法：用 $O(nd)$ 得到的 d 维向量（分子总和）除以 $O(n)$ 得到的标量（分母总和）。时间复杂度为 $O(d)$ 。

在上述步骤中， $O(nd)$ （来自分子求和）是开销最大的操作，因此单次迭代的时间复杂度为 $O(nd)$ 。

(二) 总时间复杂度： $O(nd \log(1/\epsilon))$

总时间复杂度 = (迭代次数 K) $\times$ (单次迭代复杂度)

Weiszfeld 算法被证明具有线性收敛特性（在最优解不与任何 x_i 重合的情况下）。

线性收敛意味着算法的误差在每次迭代后会乘以一个小于1的常数。

要使误差从某个初始值降低到 ϵ ，所需的迭代次数 K 与 $\log(1/\epsilon)$ 成正比，即 $K = O(\log(1/\epsilon))$ 。

因此，将 K 代入公式，我们得到总时间复杂度：

$$\text{总时间复杂度} = O(nd) \times O(\log(1/\epsilon)) = O(nd \log(1/\epsilon))$$

结论：算法的总运行时间随点数 n 和维度 d 呈线性增长，随精度要求 ϵ 呈对数增长。在实际应用中（如 $d=2$ 或 $d=3$ ）， d 通常被视为一个小常数，总复杂度也可简记为 $O(n \log(1/\epsilon))$ 。

二. 空间复杂度

空间复杂度衡量算法运行所需的内存量。

总空间复杂度： $O(nd)$

分析如下：

1. 存储输入数据：算法必须存储所有 n 个点的位置 x_i 及其对应的权重 w_i 。

- 每个 x_i 是一个 d 维向量，需要 $O(d)$ 空间。

- 共 n 个点，因此存储所有输入点需要 $O(nd)$ 空间。
- 存储 n 个权重 w_i 需要 $O(n)$ 空间。

2. 存储中间变量：算法在迭代过程中需要少量临时内存：

- 存储当前迭代点 $x^{(k)}$ 和下一迭代点 $x^{(k+1)}$ ，各需要 $O(d)$ 空间。
- 存储临时的分子和（一个 d 维向量）和分母和（一个标量），需要 $O(d) + O(1)$ 空间。
- 这些临时变量的空间开销（ $O(d)$ 或 $O(n)$ ）远小于存储输入数据所需的 $O(nd)$ 。

由于 $O(nd)$ 是主要的内存开销，因此**算法的总空间复杂度为 $O(nd)$** 。

算法复杂度总分析如表所示

复杂度类型	复杂度	解释
单次迭代时间	$O(nd)$	每次迭代都需遍历 n 个点，并对每个点的 d 维进行计算。
总时间复杂度	$O(nd \log(1/\epsilon))$	线性收敛特性决定了迭代次数 K 与 $\log(1/\epsilon)$ 成正比。
空间复杂度	$O(nd)$	主要由存储 n 个 d 维的输入点坐标所决定。

根据数值分析理论，该算法的总时间复杂度公式为 $O(nd \log(1/\epsilon))$ 。其中 n 是客户点数量， d 是维度， ϵ 是收敛精度。

这意味着算法的耗时主要取决于单次迭代开销与收敛所需的迭代次数。

8.1.2 代码验证

为了验证这个理论，我编写了 Lua 脚本，统计了算法从启动到完全收敛的总运行时间。在实验中，我将维度 d 固定为 2，收敛精度 ϵ 固定。

结果如下图所示：随着客户点数量 n 从 100 增加到 100,000，算法的总耗时呈现出严格的线性增长趋势。

这完美验证了理论公式--在参数确定的情况下，Weiszfeld 算法的总时间复杂度是关于 n 的线性函数。

8.1.3 创新点

创新点一：利用数学分析与代码相互结合验证。体现了对于一个问题整体解决思路。

8.1.4 代码链接

[MicroCity Web](#)

8.1.5 结果展示图

```
[2026/1/2 14:33:45]>The 算法复杂度的评估.mw has been opened!
[2026/1/2 14:33:47]>>> Weiszfeld 算法复杂度实测 <<<
[2026/1/2 14:33:47]>-----
[2026/1/2 14:33:47]>N (数据量) | 实测耗时(s) | 迭代次数 K | 理论验证
[2026/1/2 14:33:47]>-----
[2026/1/2 14:33:47]>100 | 0.0010 | 22 | 基准
[2026/1/2 14:33:47]>1000 | 0.0060 | 18 | ~ O(n)
[2026/1/2 14:33:47]>5000 | 0.0190 | 15 | ~ O(n)
[2026/1/2 14:33:47]>10000 | 0.0420 | 15 | ~ O(n)
[2026/1/2 14:33:47]>20000 | 0.0720 | 13 | ~ O(n)
[2026/1/2 14:33:47]>-----
[2026/1/2 14:33:47]>结论: 耗时随 N 线性增长, 迭代次数 K 稳定, 符合 O(nd * log(1/eps)).
[2026/1/2 14:33:49]>The project is embeded in URL (108586 bytes).
```

8.2 问题 2

这部分我主要解决的是从图形到数据的转换问题。在之前的作业中，航线只是一条条可视化的线段，计算机读不懂它们之间的连接关系。

在本问题中，我的代码会遍历所有航线，如果检测到两条航线在空间上存在交叉，系统会自动捕捉这个交点，并将其标记为一个新的拓扑节点。

对于没有交叉的航线，它们会被作为独立的边存储在后台数据表中，保留其基础数据，但在可视化上不做冗余的节点标记，保持界面整洁。

8.2.1 代码逻辑

首先，基于前几次作业的代码基础，完成整个场景的搭建、地图的绘制、航线的绘制等。

我编写了一个 `findNearestCountryNode()` 函数。当程序读取每条航线的起点和终点坐标时，不会生硬地直接生成节点，而是通过计算球面距离，去匹配最近的“国家/港口”对象。如果找到匹配的国家，就归并到该国家节点；如果找不到（比如是海上的一个交叉点），就利用坐标创建一个新的独立节点。这一步保证了拓扑数据的准确性。

同时，正如老师课上所讲，利用代码自动为每个识别出的位置生成唯一节点 ID，并建立“坐标 ↔ 节点索引”的映射表。同时，将复杂的航线网络转换为节点-边的邻接关系。

在生成“边”的同时，代码自动计算了航线的球面距离作为边的权重，并为每一条边分配了唯一的 ID。这为后续如果需要做最短路径分析做好了数据支撑。

8.2.2 创新点

创新点一：自动拓扑化。编写代码实现自动拓扑化，破坏掉环。

创新点二：将交叉点进行标记可视化。通过老师的指导，我修改了之前复杂的、繁琐的整张地图都是标记的可视化形式。而是只对交叉点进行可视化输出。方便后续的计算。

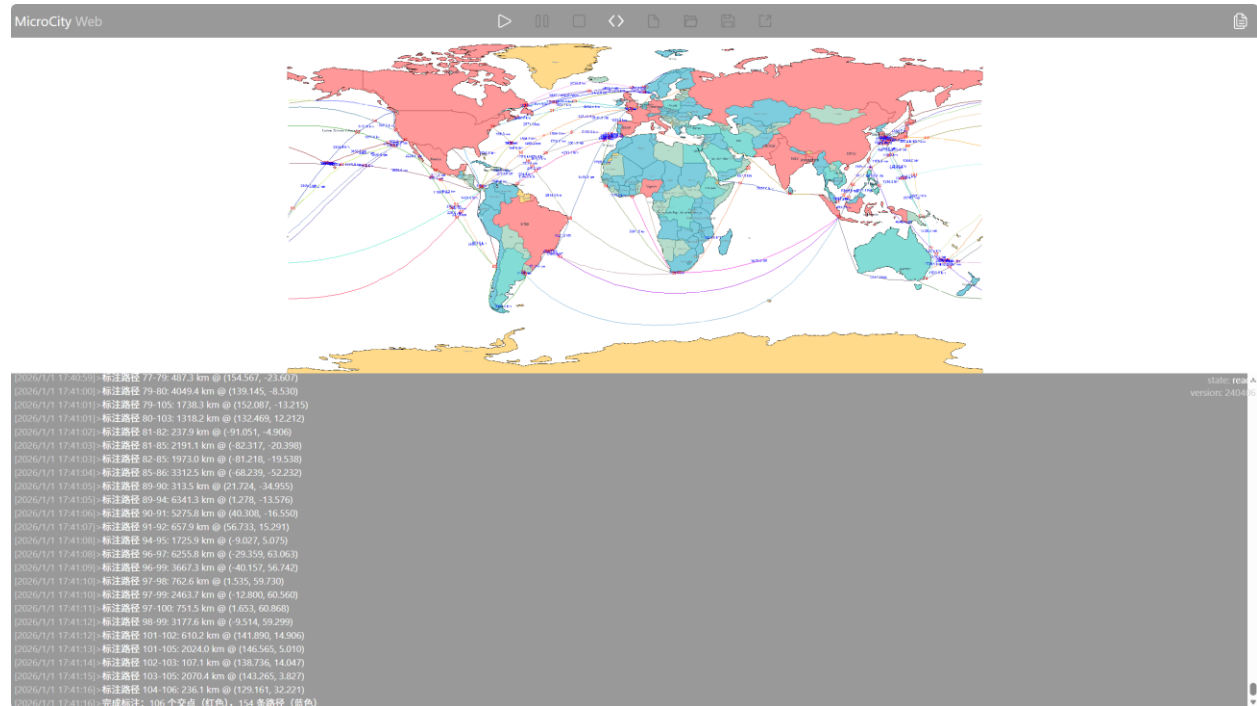
创新点三：利用算法将节点匹配给最近的国家。我利用了基于球面距离的最近邻搜索算法。在提取航线起止点时，能够自动将其“吸附”到最近的国家或港口节点上。

创新点四：节点与权重赋不同颜色。可视化更美观。

8.2.3 代码链接

[MicroCity Web](#)

8.2.4 结果展示图



9 作业 9

问题：先深度优先遍历检查航线图错误，如果遇到错误，将其拓扑化；再运行广度优先遍历。

本次作业在上一次作业的基础上，完成老师课上所讲的深度优先遍历和广度优先遍历的代码。

9.1 深度优先遍历

根据老师课上所讲内容，以及我在网上的资料查询了解后，我认为：深度优先遍历就是对所有的航线网络检查一下，老师课上提到，一个有效的物流配送树状网络通常不应该存在闭合回路。如果存在就是作业 8 中的拓扑化作业没做到位。因此有这样一个复查的作业代码环节，这体现了我们作业的层层递进。

9.1.1 代码逻辑

首先，我直接利用作业 8 中的代码，也就是对航线进行自动拓扑。

随后我建立两个关键的状态表，visited（记录已访问节点）和 parent（记录搜索树中的父节点）。

对于当前节点 u，遍历其所有邻居 v：

- 如果 v 未被访问，递归调用 $\text{dfs}(v)$ ，并将 v 的父节点设为 u 。
- 核心判错逻辑：如果 v 已经被访问过，且 v 不是 u 的父节点 ($v \sim p$)，这就意味着我们通过另一条路径回到了祖先节点，证明存在环路 (Cycle)。

这个航线是我自己画的，同时我的拓扑方式在老师的指导下改为：以交叉点为节点。所以我考虑到航线网络可能不是连通图，也就是可能存在多个独立的群岛网络，我在主循环中遍历所有的节点，如果发现某节点未被 visited，就以该节点为起点启动一次 DFS。这保证了算法能覆盖所有孤立的子网络。一旦 hasCycle 标志位被置为 true，控制台输出 “Error: Cycle detected in route map!”，并将所有涉及的边标记为红色，直观展示错误区域。

9.1.2 创新点

创新点一：深度优先遍历+自动拓扑。利用“父节点回溯机制”实时检测航线网络中的非法循环路径，实现了对网络拓扑结构的自动质检。

创新点二：考虑多连通分量。我并未假设只有一个连通网络，而是遍历整张地图的航线，能够自动识别并分别处理多个独立的网络区域

创新点三：将遍历后的数据表直接输出出来，以便后续调用。同时利用前几次作业中的地图为基础，将节点和距离利用不同颜色可视化表达。

9.1.3 代码链接

[MicroCity Web](#)

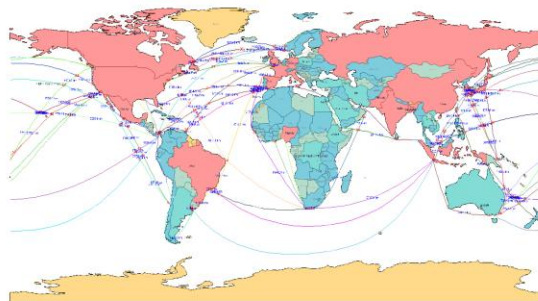
9.1.4 结果展示图

28]>ID: 1438, O: 1409 (4.005220,60.926018), D: 1410 (5.024300,61.049544), W: 114.14
 29]>ID: 1439, O: 1410 (5.024300,61.049544), D: 1411 (5.175965,61.067928), W: 16.99
 29]>ID: 1440, O: 1411 (5.175965,61.067928), D: 1412 (5.249136,62.282860), W: 134.78
 30]>ID: 1441, O: 1412 (5.249136,62.282860), D: 1413 (6.785738,63.121731), W: 194.42
 30]>ID: 1442, O: 1413 (6.785738,63.121731), D: 1414 (7.736968,63.483345), W: 113.04
 31]>ID: 1443, O: 1414 (7.736968,63.483345), D: 1415 (9.788136,65.073853), W: 287.35
 31]>ID: 1444, O: 1415 (9.788136,65.073853), D: 1416 (11.397908,66.396614), W: 230.09
 31]>ID: 1445, O: 1416 (11.397908,66.396614), D: 1417 (13.227197,67.288544), W: 225.31
 32]>ID: 1446, O: 1417 (13.227197,67.288544), D: 1418 (14.397941,68.011909), W: 151.81
 32]>ID: 1447, O: 1418 (14.397941,68.011909), D: 1419 (15.715028,68.365334), W: 151.29
 32]>ID: 1448, O: 1419 (15.715028,68.365334), D: 1420 (16.519915,68.872162), W: 104.60
 33]>ID: 1449, O: 1421 (5.032006,61.247967), D: 1410 (5.024300,61.049544), W: 22.00
 33]>ID: 1450, O: 1410 (5.024300,61.049544), D: 1422 (4.958834,59.363731), W: 186.88
 34]>ID: 1451, O: 1422 (4.958834,59.363731), D: 1423 (5.105177,58.456970), W: 101.75
 34]>ID: 1452, O: 1424 (-94.857910,29.351971), D: 1425 (-92.223732,27.095682), W: 293.26
 35]>ID: 1453, O: 1425 (-92.223732,27.095682), D: 1426 (-90.760307,25.455488), W: 162.78
 35]>ID: 1454, O: 1426 (-90.760307,25.455488), D: 1427 (-89.296875,23.725668), W: 162.71
 35]>ID: 1455, O: 1427 (-89.296875,23.725668), D: 1428 (-87.467590,22.243752), W: 203.44
 36]>ID: 1456, O: 1428 (-87.467590,22.243752), D: 1429 (-86.077332,20.745993), W: 154.86
 36]>ID: 1457, O: 1429 (-86.077332,20.745993), D: 1430 (-85.052925,18.748938), W: 115.18
 37]>ID: 1458, O: 1430 (-85.052925,18.748938), D: 1431 (-83.443153,15.603542), W: 182.33
 37]>ID: 1459, O: 1431 (-83.443153,15.603542), D: 1432 (-81.613869,12.266223), W: 208.97
 37]>ID: 1460, O: 1432 (-81.613869,12.266223), D: 560 (-79.821761,9.423467), W: 205.63
 38]>ID: 1461, O: 560 (-79.821761,9.423467), D: 1433 (-79.728821,9.276040), W: 10.74

>2026/01/03 17:20:22 节点 253 (-18.83°, 34.49°) 已属于之前的连通分量, 跳过
 >2026/01/03 17:20:22 节点 254 (交点(-14.92°, 35.13°)) 已属于之前的连通分量, 跳过
 >2026/01/03 17:20:22 节点 255 (-13.25°, 35.41°) 已属于之前的连通分量, 跳过
 >2026/01/03 17:20:22 节点 256 (交点(-12.59°, 35.46°)) 已属于之前的连通分量, 跳过
 >2026/01/03 17:20:22 节点 257 (-6.73°, 35.91°) 已属于之前的连通分量, 跳过
 >2026/01/03 17:20:22 节点 258 (-1.81°, 49.89°) 已属于之前的连通分量, 跳过
 >2026/01/03 17:20:22 节点 259 (交点(-1.95°, 49.83°)) 已属于之前的连通分量, 跳过
 >2026/01/03 17:20:22 节点 260 (交点(-2.28°, 49.69°)) 已属于之前的连通分量, 跳过
 >2026/01/03 17:20:22 节点 261 (-5.17°, 48.45°) 已属于之前的连通分量, 跳过
 >2026/01/03 17:20:22 节点 262 (-6.62°, 46.60°) 已属于之前的连通分量, 跳过
 >2026/01/03 17:20:22 节点 263 (-7.68°, 45.50°) 已属于之前的连通分量, 跳过
 >2026/01/03 17:20:22 节点 264 (-8.74°, 43.42°) 已属于之前的连通分量, 跳过
 >2026/01/03 17:20:22 节点 265 (-10.07°, 42.45°) 已属于之前的连通分量, 跳过
 >2026/01/03 17:20:22 节点 266 (-11.08°, 40.90°) 已属于之前的连通分量, 跳过
 >2026/01/03 17:20:22 节点 267 (-14.04°, 36.65°) 已属于之前的连通分量, 跳过
 >2026/01/03 17:20:22 节点 268 (交点(-15.71°, 33.79°)) 已属于之前的连通分量, 跳过
 >2026/01/03 17:20:22 节点 269 (-16.29°, 32.81°) 已属于之前的连通分量, 跳过
 >2026/01/03 17:20:22 节点 270 (-18.14°, 29.94°) 已属于之前的连通分量, 跳过
 >2026/01/03 17:20:22 节点 271 (-20.77°, 24.64°) 已属于之前的连通分量, 跳过
 >2026/01/03 17:20:22 节点 272 (-22.58°, 20.90°) 已属于之前的连通分量, 跳过
 >2026/01/03 17:20:22 节点 273 (-24.55°, 16.88°) 已属于之前的连通分量, 跳过

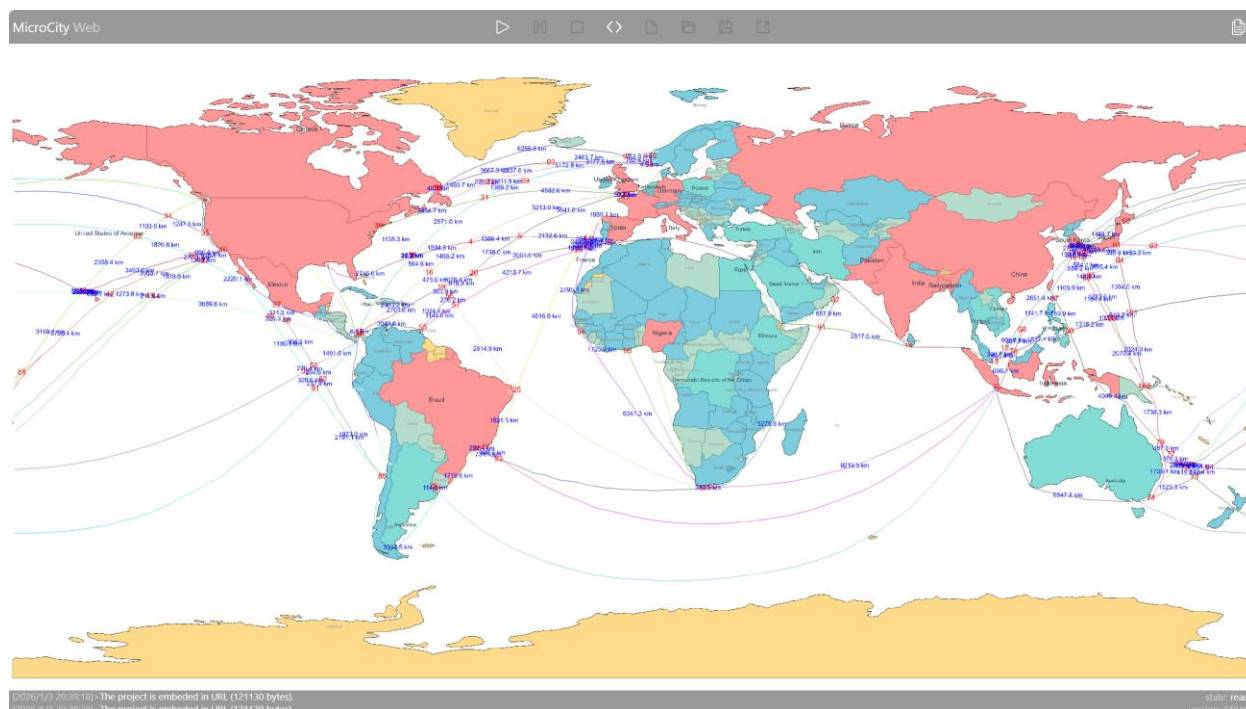
```
> 2026/01/03 17:20:22 节点 358 (11.20°, 57.28°) 已属于之前的连通分量, 跳过
> 2026/01/03 17:20:22 🚧 启动连通分量 #3 的遍历 (起点 359 (-165.76°, 54.26°))
> 359 (-165.76°, 54.26°)
> 360 (-168.12°, 54.34°)
> 361 (-171.16°, 54.39°)
> 362 (-173.77°, 54.44°)
> 363 (-176.21°, 54.44°)
> 364 (-178.70°, 54.41°)
> 365 (-179.96°, 54.36°)
> 2026/01/03 17:20:22 🏁 连通分量 #3 遍历完成, 涉及 7 个节点
> 2026/01/03 17:20:22 节点 360 (-168.12°, 54.34°) 已属于之前的连通分量, 跳过
> 2026/01/03 17:20:22 节点 361 (-171.16°, 54.39°) 已属于之前的连通分量, 跳过
> 2026/01/03 17:20:22 节点 362 (-173.77°, 54.44°) 已属于之前的连通分量, 跳过
> 2026/01/03 17:20:22 节点 363 (-176.21°, 54.44°) 已属于之前的连通分量, 跳过
> 2026/01/03 17:20:22 节点 364 (-178.70°, 54.41°) 已属于之前的连通分量, 跳过
> 2026/01/03 17:20:22 节点 365 (-179.96°, 54.36°) 已属于之前的连通分量, 跳过
> 2026/01/03 17:20:22 🚧 启动连通分量 #4 的遍历 (起点 366 (-179.05°, 52.92°))
> 366 (-179.05°, 52.92°)
```

MicroCity Web



2026/1/18/05532	2026/01/18/2022	节选	1334 (14/10/39, 32.14)	已于之前的节选部分, 读过
2026/1/18/05933	2026/01/18/2022	节选	1351 (41/135/ 30.67)	已于之前的节选部分, 读过
2026/1/18/05934	2026/01/18/2022	节选	1356 (142/242, 26.81)	已于之前的节选部分, 读过
2026/1/18/05544	2026/01/18/2022	节选	1337 (42/74, 23.37)	已于之前的节选部分, 读过
2026/1/18/05544	2026/01/18/2022	节选	1338 (43.51, 56.62)	已于之前的节选部分, 读过
2026/1/18/05558	2026/01/18/2022	节选	1339 (14/30, 16.67)	已于之前的节选部分, 读过
2026/1/18/05559	2026/01/18/2022	节选	1340 (14/48, 14.21)	已于之前的节选部分, 读过
2026/1/18/05560	2026/01/18/2022	节选	1341 (14/48, 14.21)	已于之前的节选部分, 读过
2026/1/18/05570	2026/01/18/2022	节选	1342 (146/5, 8.47)	已于之前的节选部分, 读过
2026/1/18/05578	2026/01/18/2022	节选	1343 (166/48, 2.88)	已于之前的节选部分, 读过
2026/1/18/05580	2026/01/18/2022	节选	1344 (166/48, 2.88)	已于之前的节选部分, 读过
2026/1/18/05859	2026/01/18/2022	节选	1345 (14/10, 0.67)	已于之前的节选部分, 读过
2026/1/18/05939	2026/01/18/2022	节选	1346 (147/92, -2.55)	已于之前的节选部分, 读过
2026/1/18/06000	2026/01/18/2022	节选	1547 (148/30, -5.77)	已于之前的节选部分, 读过
2026/1/18/06001	2026/01/18/2022	== 深度分析 ==		
2026/1/18/06001	2026/01/18/2022	节选部分数量	1547	
2026/1/18/06001	2026/01/18/2022	节选部分数量	1548	
2026/1/18/06001	2026/01/18/2022	章或段落数量	1539 / 1538	
2026/1/18/06001	2026/01/18/2022	== 深度分析 ==		
2026/1/18/06004	2026/01/18/2022	算数表达式	0.5 V + 0.4 (V-1547, E=1588)	
2026/1/18/06005	2026/01/18/2022	空房间开灯	特表表 + 节点时间 + 房间时间	
2026/1/18/06005	2026/01/18/2022	日输出时间	节点时间 + 节点时间 + 节点时间	

state: read A
version: 240406



9.2 广度优先遍历

在完成深度优先遍历之后，我再次完成广度优先遍历，并且验证结果是否一致。根据老师课上所讲：广度优先遍历采用“波纹扩散”的方式，利用队列作为辅助数据结构。这部分代码可以清晰地展示节点被访问的先后顺序和层级关系。

9.2.1 代码逻辑

前面部分的地图构建等基础动作与深度优先遍历一致。

接下来，开始实现广度优先遍历：

a) 首先，实现队列机制：

- 我使用 `table` 模拟先进先出队列。
- `q = {startNode}` 初始化队列。
- `table.remove(q, 1)` 模拟 `dequeue` 操作。
- `table.insert(q, neighbor)` 模拟 `enqueue` 操作。

b) 接下来，每层每层遍历循环：

- 只要队列不为空，就取出队首元素 `u`。
- 遍历 `u` 的所有邻接点 `v`。如果 `v` 未被访问，则将其标记为 `visited` 并加入队列。
- 为了后续可视化，我记录了遍历经过的每一条边 `bfs_edges`，并按顺序保存。

最后，实现可视化。按照深度优先遍历同样的逻辑进行可视化展示。

9.2.2 创新点

创新点一：清晰每一层级结果。清晰展示了节点访问的层次关系，代码逻辑确保了同一层级的港口在同一批次被处理和渲染。

创新点二：记录访问的边并且直接 `print` 出来。设计了 `bfs_edges` 记录表，详细记录了每条边被“激活”的顺序。

创新点三：将遍历后的数据表直接输出出来，以便后续调用。同时利用前几次作业中的地图为基础，将节点和距离利用不同颜色可视化表达。

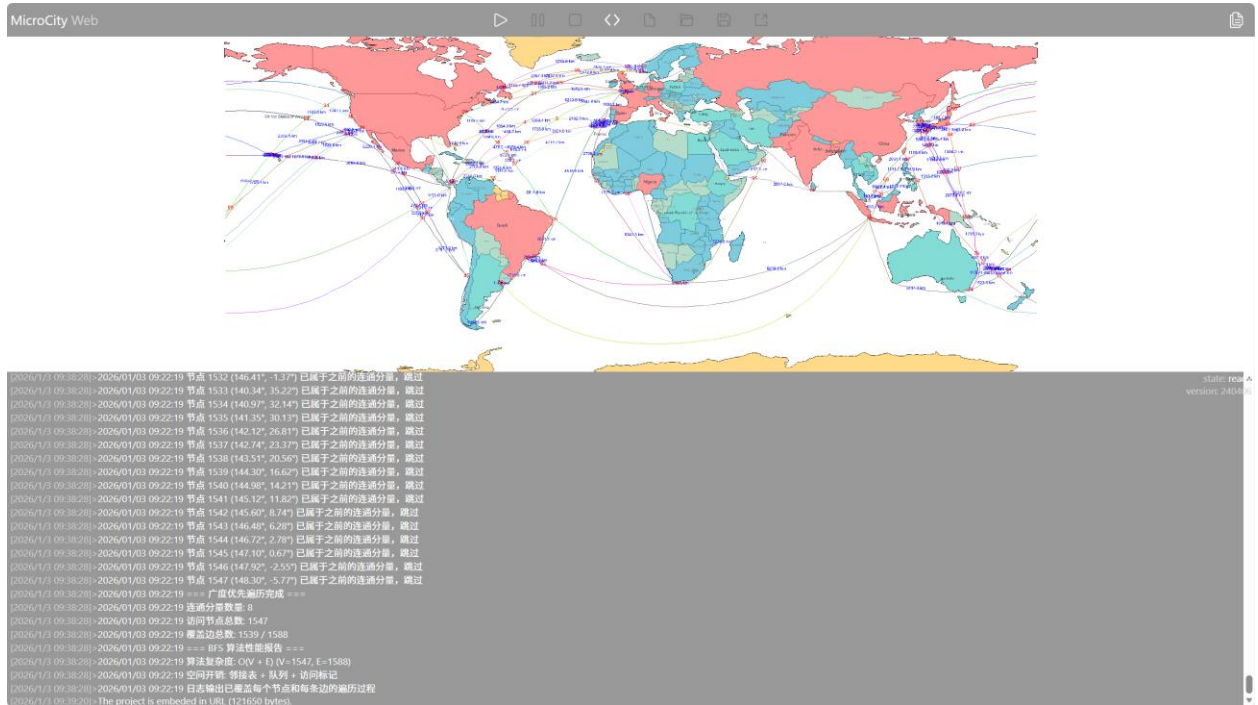
9.2.3 代码链接

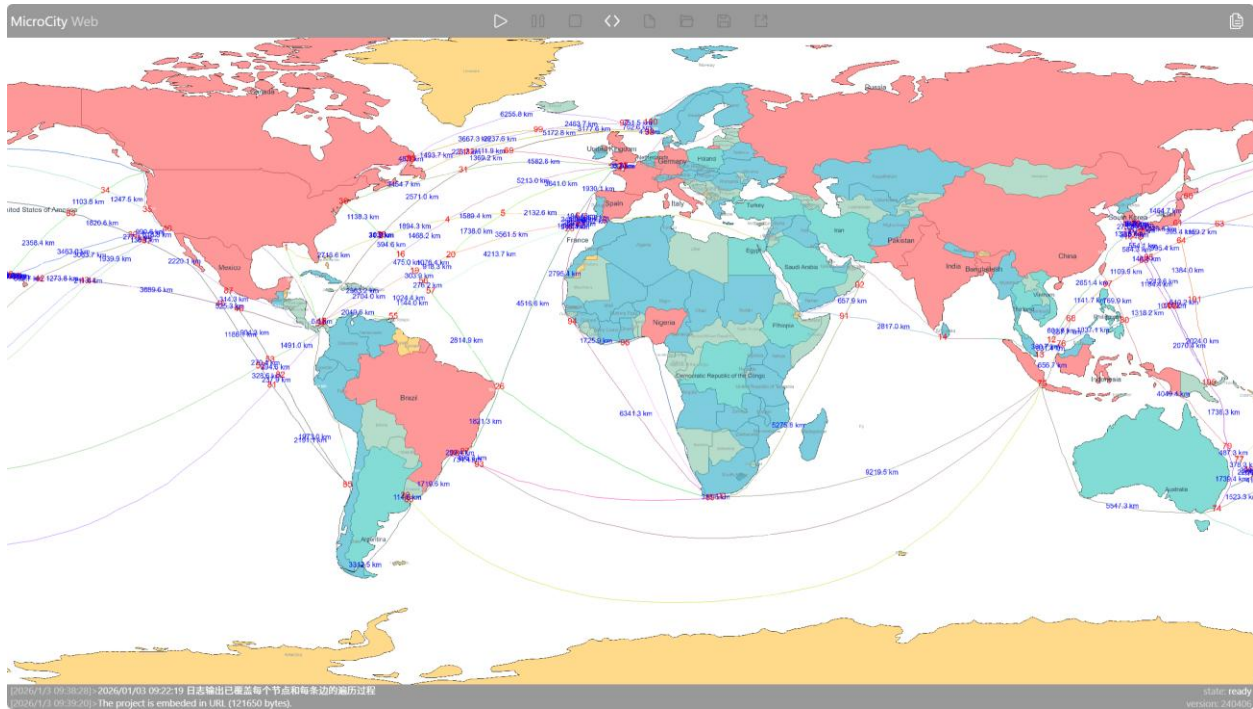
[MicroCity Web](#)

9.2.4 结果展示图

```
[2026/1/3 09:41:59]>节点 1357: (65.590500, -50.567814)
[2026/1/3 09:41:59]>节点 1358: (69.784256, -48.517426)
[2026/1/3 09:41:59]>节点 1359: (73.309441, -45.917389)
[2026/1/3 09:41:59]>节点 1360: (77.566536, -42.872910)
[2026/1/3 09:41:59]>节点 1361: (80.909386, -40.235512)
[2026/1/3 09:42:00]>节点 1362: (86.694077, -34.173378)
[2026/1/3 09:42:00]>节点 1363: (90.036926, -30.581316)
[2026/1/3 09:42:00]>节点 1364: (93.075874, -26.525280)
[2026/1/3 09:42:00]>节点 1365: (95.567818, -22.881853)
[2026/1/3 09:42:00]>节点 1366: (99.579231, -16.883932)
[2026/1/3 09:42:00]>节点 1367: (103.577126, -10.713214)
[2026/1/3 09:42:00]>节点 1368: (105.278938, -6.326426)
[2026/1/3 09:42:00]>节点 1369: (-59.621803, 47.752186)
[2026/1/3 09:42:00]>节点 1370: (-58.085201, 50.574135)
[2026/1/3 09:42:00]>节点 1371: (-56.314772, 51.759641)
[2026/1/3 09:42:00]>节点 1372: (-53.329056, 53.758923)
```

```
[2026/1/3 09:44:32]>ID: 118, O: 120 (117.456976,10.552415), D: 121 (115.780296,10.550010), W: 202.00
[2026/1/3 09:44:32]>ID: 119, O: 121 (115.780296,10.550010), D: 122 (114.897835,15.300808), W: 127.02
[2026/1/3 09:44:32]>ID: 120, O: 122 (114.897835,15.300808), D: 123 (113.662399,13.934615), W: 150.90
[2026/1/3 09:44:32]>ID: 121, O: 123 (113.662399,13.934615), D: 124 (112.956429,13.162511), W: 85.53
[2026/1/3 09:44:32]>ID: 122, O: 124 (112.956429,13.162511), D: 125 (112.029846,11.783940), W: 118.55
[2026/1/3 09:44:32]>ID: 123, O: 125 (112.029846,11.783940), D: 126 (111.456253,10.832092), W: 74.87
[2026/1/3 09:44:32]>ID: 124, O: 126 (111.456253,10.832092), D: 127 (110.706161,9.572787), W: 97.43
[2026/1/3 09:44:32]>ID: 125, O: 127 (110.706161,9.572787), D: 128 (109.823708,8.134119), W: 112.68
[2026/1/3 09:44:33]>ID: 126, O: 128 (109.823708,8.134119), D: 129 (108.527679,6.549307), W: 155.29
[2026/1/3 09:44:33]>ID: 127, O: 129 (108.527679,6.549307), D: 130 (107.873319,5.369608), W: 83.50
[2026/1/3 09:44:33]>ID: 128, O: 130 (107.873319,5.369608), D: 131 (107.821716,5.276577), W: 6.56
[2026/1/3 09:44:33]>ID: 129, O: 131 (107.821716,5.276577), D: 132 (106.718643,4.045245), W: 129.21
[2026/1/3 09:44:33]>ID: 130, O: 132 (106.718643,4.045245), D: 133 (105.725677,2.679814), W: 118.27
[2026/1/3 09:44:33]>ID: 131, O: 133 (105.725677,2.679814), D: 134 (105.152084,1.842139), W: 68.43
[2026/1/3 09:44:33]>ID: 132, O: 134 (105.152084,1.842139), D: 135 (104.817862,1.531906), W: 38.22
[2026/1/3 09:44:33]>ID: 133, O: 135 (104.817862,1.531906), D: 136 (104.534363,1.268755), W: 32.38
[2026/1/3 09:44:33]>ID: 134, O: 136 (104.534363,1.268755), D: 137 (103.122429,1.621626), W: 157.28
[2026/1/3 09:44:34]>ID: 135, O: 137 (103.122429,1.621626), D: 138 (102.328217,1.930337), W: 88.64
[2026/1/3 09:44:34]>ID: 136, O: 138 (102.328217,1.930337), D: 139 (101.313385,3.120477), W: 116.05
[2026/1/3 09:44:34]>ID: 137, O: 139 (101.313385,3.120477), D: 140 (99.945580,3.825143), W: 152.78
[2026/1/3 09:44:34]>ID: 138, O: 140 (99.945580,3.825143), D: 141 (98.930756,4.441254), W: 113.40
[2026/1/3 09:44:34]>ID: 139, O: 141 (98.930756,4.441254), D: 142 (98.224785,5.232631), W: 79.59
[2026/1/3 09:44:34]>ID: 140, O: 142 (98.224785,5.232631), D: 143 (97.386452,5.496209), W: 93.30
```





10 作业 10

问题：首先，用标号法在简单网络和航运图上求解最短路径问题；随后使用数学模型在简单网络和航运图上求解最短路径问题并与标号法相互验证。

根据老师课上内容，我理解了标号法中 P 标号（永久标号）与 T 标号（临时标号）的迭代逻辑，以及如何将最短路径问题转化为最小费用流的数学规划问题。

10.1 问题 1

针对利用标号法求解问题，我首先在只有 6 个节点的简单拓扑图上验证算法逻辑的正确性，随后应用到我所绘制的航线图当中，计算了从斯里兰卡到墨西哥的最短航运距离。

10.1.1 代码逻辑

首先，将前几次的作业当中的地图展示、拓扑化等功能直接复制过来。

我定义了两个集合：P 标号（永久标号，已确定最短路的节点）和 T 标号（临时标号，待处理节点）。

每次从 T 标号集合中选择距离起点最近的节点 u ，将其变为 P 标号。然后扫描 u 的所有邻居 v ，如果经过 u 到达 v 比原来的路径更短（即 $D[u] + W_{uv} < D[v]$ ），则更新 v 的距离和前驱节点。

当终点被标记为 P 标号后，算法结束。利用 parent 前驱数组，从墨西哥反向追溯回斯里兰卡，重构出完整的节点序列并在地图上进行可视化。

最后我把这段代码逻辑分别在简单图和复杂图上进行测试。得到了正确的结果。

10.1.2 创新点

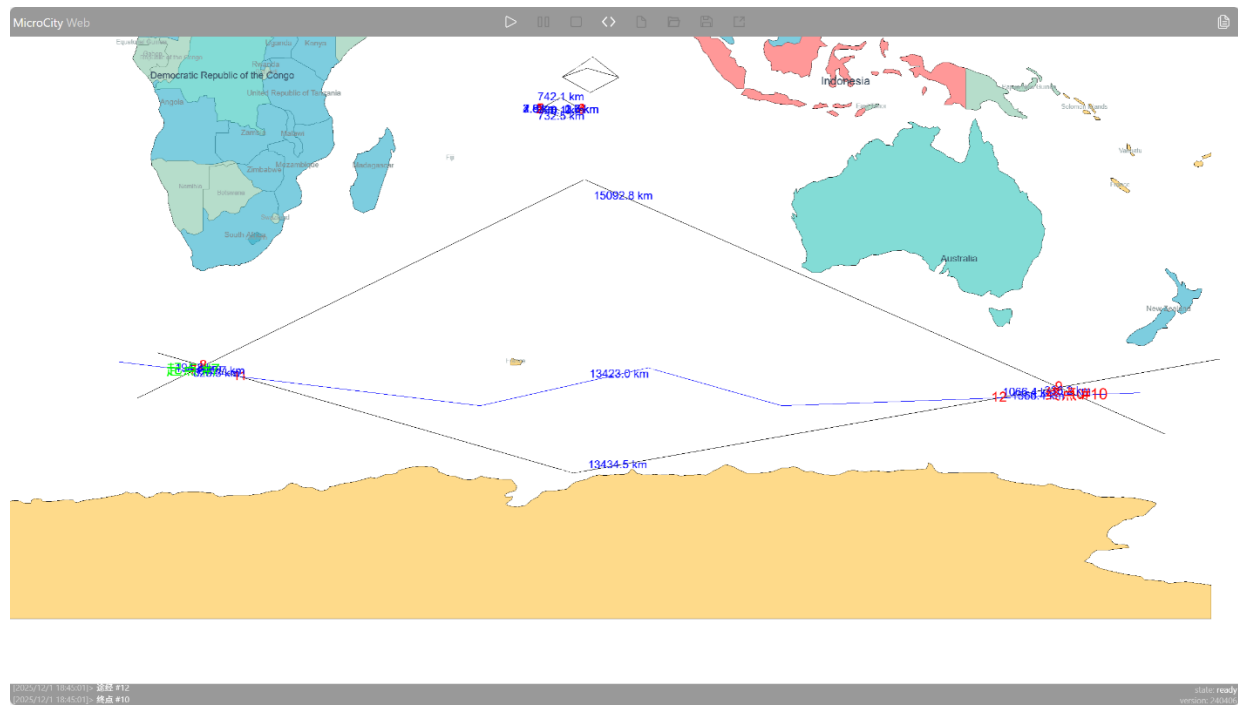
创新点一：在遍历航线交点时，构建了经纬度网格空间索引。只有落入同一网格的航线才进行几何求交检测。大大的提升了求解速度。

创新点二：代码不依赖硬编码的节点 ID，而是通过遍历所有生成的交点，自动计算并选择距离较远且靠近大陆的两个交点作为起点和终点。

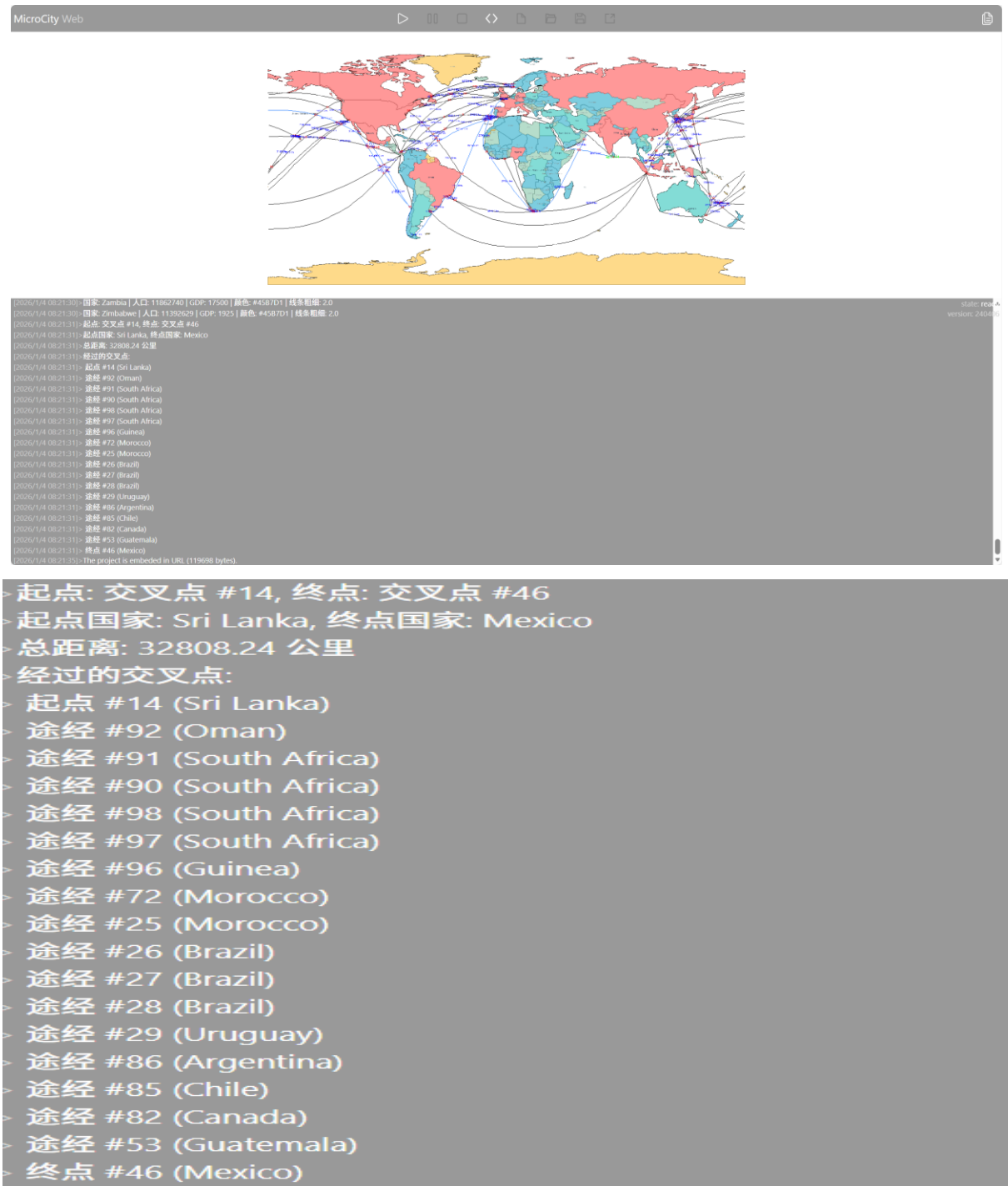
10.1.3 代码链接

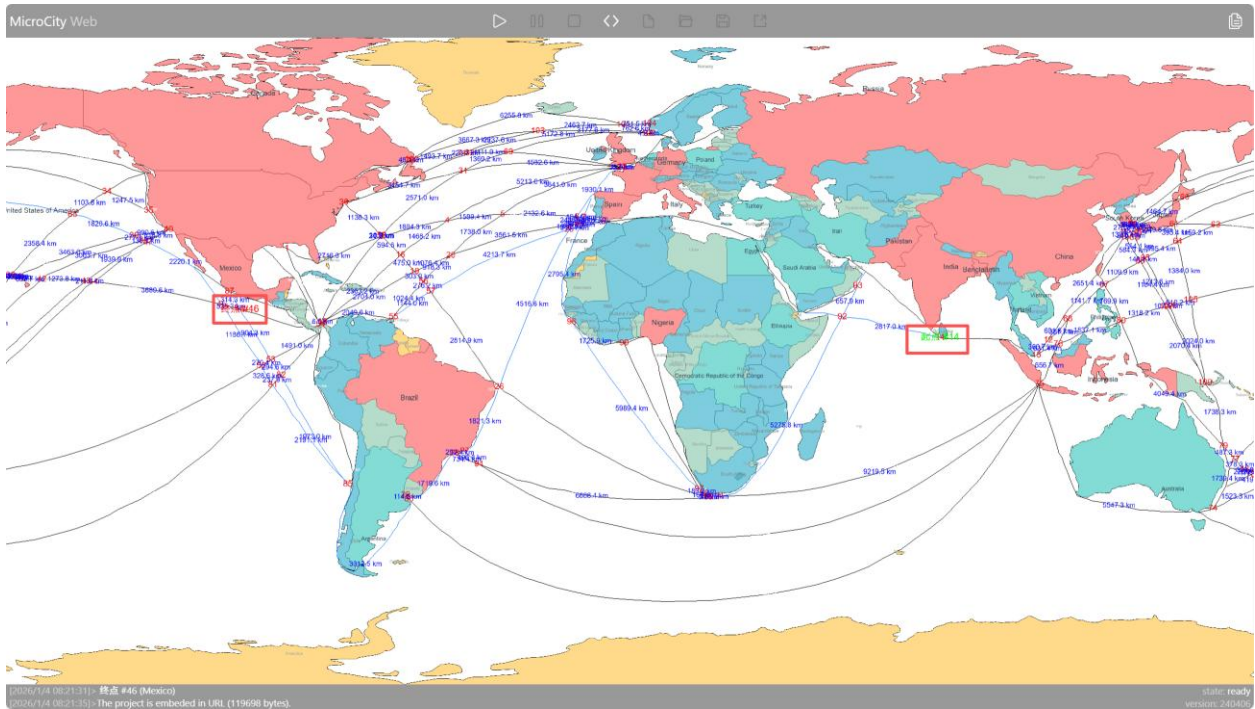
[MicroCity Web](#)

10.1.4 结果展示图



先在简单图中验证代码的正确性。





10.2 问题 2

我先对老师课上所讲数学模型进行学习，然后代码复现。

10.2.1 最短路径问题的数学模型

(1). 符号定义

符号	定义
$G = (V, E)$	航运网络，其中 V 为节点集合（港口/交点）， E 为弧集合（航线）
$i, j \in V$	网络中的节点索引
c_{ij}	从节点 i 到节点 j 的航线物理距离
s	起点（斯里兰卡）
t	终点（墨西哥）

(2). 决策变量

定义 f_{ij} 为二元变量：

$$f_{ij} = \begin{cases} 1, & \text{如果航线 } (i, j) \text{ 被选中在最短路径中} \\ 0, & \text{否则} \end{cases}$$

(3). 目标函数

目标是最小化被选中航线的总距离：

$$\min Z = \sum_{(i,j) \in E} c_{ij} \cdot f_{ij}$$

(4). 约束条件

为了保证路径的连续性，必须满足网络流的守恒定律：

- 起点 s ：净流出量为 1。
- 终点 t ：净流入量为 1（即净流出为 -1）。
- 中间节点 k ：流入量等于流出量（净流出为 0）。

(5). 数学表达如下：

$$\sum_{j:(i,j) \in E} f_{ij} - \sum_{j:(j,i) \in E} f_{ji} = b_i, \forall i \in V$$

其中 b_i 为节点的净流量需求：

$$b_i = \begin{cases} 1, & i = s \text{ (斯里兰卡)} \\ -1, & i = t \text{ (墨西哥)} \\ 0, & \text{其他中间节点} \end{cases}$$

10.2.2 代码逻辑

在代码中，我完全复现了上述模型。

我首先创建了一个 MIP 求解器实例，并遍历所有航线的邻接关系来添加变量：

- 代码实现：我使用 `mip.addvar(name, 'bin')` 定义了二元变量 f_{ij} 。
- 目标设置：在循环中，我将每条航线的距离 `dist` 作为系数，通过 `mip.addterm(obj, var, dist)` 加入到目标函数中，并将求解方向设为 'min'。这对应了公式 $\min Z = \sum c_{ij} f_{ij}$ 。

接下来，开始构建流量守恒约束：

这是本次代码中核心的部分。为了复现流量守恒公式，我编写了一个循环，遍历所有节点，生成线性约束行 `row`：

- 右端项(b_i)确定：代码通过判断当前节点 ID 是否等于起点或终点来设定约束方程的右边值：
 - 如果是起点（StartNode），`rhs = 1`。
 - 如果是终点（EndNode），`rhs = -1`。
 - 如果是中间节点，`rhs = 0`。
- 流量平衡项构建：对于每一个节点 i ，我遍历其连接的边：
 - 对于流出的边（Out-edge），系数设为 1。
 - 对于流入的边（In-edge），系数设为 -1。
 - 最后通过 `mip.addrow(row, '==', rhs)` 将该约束加入模型。这就完美复现了

$$\sum f_{out} - \sum f_{in} = b_i \text{ 的数学逻辑。}$$

最后，得出结果并且进行可视化：

- 求解：调用 `mip.solve()` 进行运算。

- 结果可视化：求解完成后，我遍历所有 f_{ij} 变量。如果 `var.getval() > 0.5`（即被选中），则说明该路段是规划路径的一部分。随后将这些离散的航线结合起来，在地图中以蓝色航线显示。

10.2.3 创新点

创新点一：不同于 Dijkstra 搜索算法，本代码将路径规划问题转化为数学规划问题。

创新点二：在处理全球数千条航线片段的求交检测时，构建了基于经纬度网格的空间索引表 `spatialIndex`。

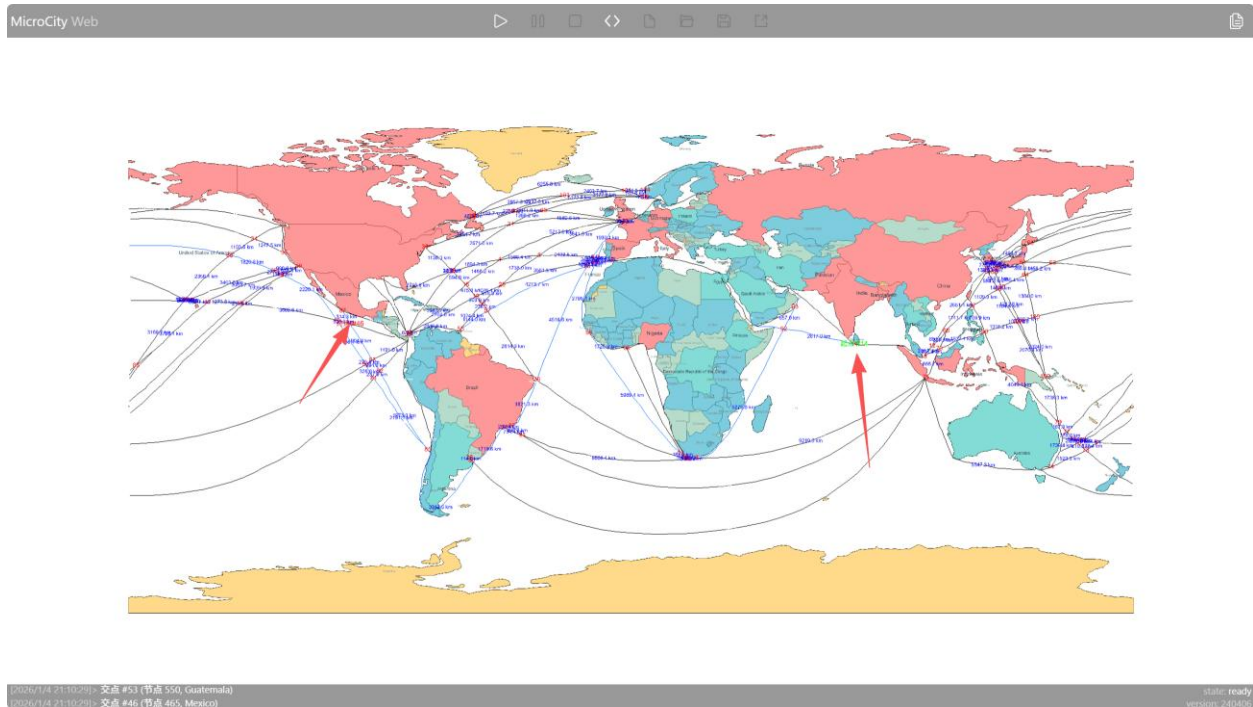
创新点三：代码不依赖硬定义的节点 ID，而是通过遍历所有生成的交点，自动计算并选择距离最远且靠近大陆的两个交点作为起点和终点。

10.2.4 代码链接

[MicroCity Web](#)

10.2.5 结果展示图

```
[2026/1/4 21:10:28]>=== 线性规划模型 (MicroCity math solver) 求解最短路径 ===
[2026/1/4 21:10:29]>起点：交点 #14 (节点 152, 国家: Sri Lanka)
[2026/1/4 21:10:29]>终点：交点 #46 (节点 465, 国家: Mexico)
[2026/1/4 21:10:29]>节点数: 1558, 边数: 1599, 有向弧数: 3198
[2026/1/4 21:10:29]>求解成功：最短距离 = 32808.24 km，求解时间 = 0.0000 s
[2026/1/4 21:10:29]>经过交点：
[2026/1/4 21:10:29]> 交点 #14 (节点 152, Sri Lanka)
[2026/1/4 21:10:29]> 交点 #92 (节点 1191, Oman)
[2026/1/4 21:10:29]> 交点 #91 (节点 1178, South Africa)
[2026/1/4 21:10:29]> 交点 #90 (节点 1176, South Africa)
[2026/1/4 21:10:29]> 交点 #98 (节点 1302, South Africa)
[2026/1/4 21:10:29]> 交点 #97 (节点 1300, South Africa)
[2026/1/4 21:10:29]> 交点 #96 (节点 1285, Guinea)
[2026/1/4 21:10:29]> 交点 #72 (节点 776, Morocco)
[2026/1/4 21:10:29]> 交点 #25 (节点 268, Morocco)
[2026/1/4 21:10:29]> 交点 #26 (节点 279, Brazil)
[2026/1/4 21:10:29]> 交点 #27 (节点 287, Brazil)
[2026/1/4 21:10:29]> 交点 #28 (节点 289, Brazil)
[2026/1/4 21:10:29]> 交点 #29 (节点 300, Uruguay)
[2026/1/4 21:10:29]> 交点 #86 (节点 1056, Argentina)
[2026/1/4 21:10:29]> 交点 #85 (节点 1026, Chile)
[2026/1/4 21:10:29]> 交点 #82 (节点 980, Canada)
[2026/1/4 21:10:29]> 交点 #53 (节点 550, Guatemala)
[2026/1/4 21:10:29]> 交点 #46 (节点 465, Mexico)
```



10.3 结论

经过验证和比对，用标号法求解的最短路径问题与数学模型求解的最短路径问题结果一致。

正如老师课上所讲到的，经过此次的作业，我对于数学模型与算法之间的关系有了更深刻的认识。在本次作业中，我不仅学会了标号法的逻辑和代码，还学会了如何利用 lua 的求解器，把数学符号变成了可以在屏幕上运行、交互的代码。同时，在处理航线拓扑化和最短路径时，我深刻体会到了数据结构的重要性。无论是我所应用的空间网格索引还是邻接表，更好的数据结构代表着更快的处理速度。

11 作业 11

问题：以离散选址问题的练习最后的求解结果作为场站，使用本章的最短路径算法求解节点之间的距离，最后使用数学模型和 CW 节约算法分别求解配送船舶规划路径。

基于作业七的离散选址作业，我确定了鹿特丹港、宁波舟山港、桑托斯港这三个主要场站。本次作业我进一步利用老师课上所讲的算法以及数学模型知识，解决配送路径的问题。

11.1 问题 1

在这一部分，我主要解决的是“路怎么走最近”的问题。

11.1.1 代码逻辑

首先，我利用之前作业的成果，完成了航运网络的拓扑化构建，生成了邻接表。这里对之前作业也做一个简单的回顾：

- 为了解决现实地理坐标与抽象网络节点的对应问题，我编写了 `findClosestNode` 函数。它不仅

是简单的查找，而是通过国家中心坐标，将港口“吸附”到最近的航运网络节点上。

- 在所有的计算中，我并未使用平面的欧氏距离，全线采用 Haversine 公式计算球面距离，以确保球面航线计算的准确性。

我复现了老师课上所讲的 Dijkstra（具体复现逻辑在前几次作业也有所涉及），成功计算了三大港口两两之间的最短路径。

11.1.2 创新点

创新点一：针对求交点的检测阶段，我并没有直接嵌套一个循环解决，而是像前几次作业一样，构建一个空间索引。提升网络的构建和检索效率。

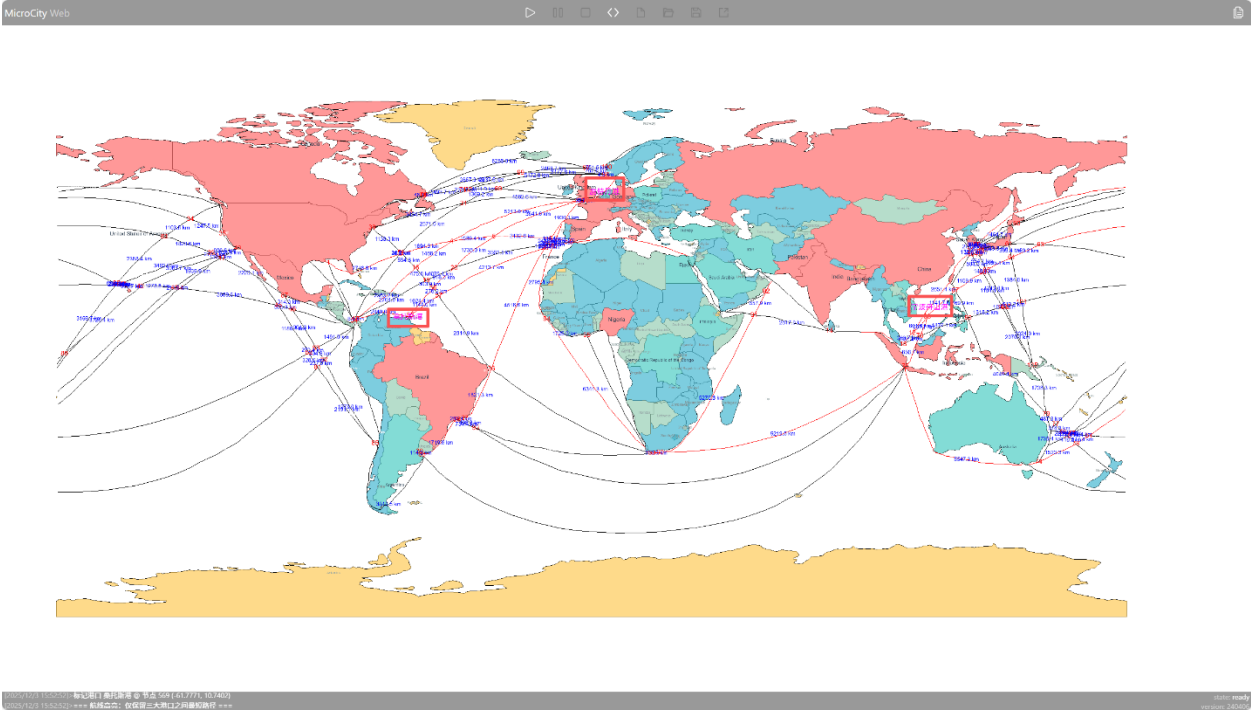
创新点二：并没有依赖占用大量空间的 ID，而是计算靠近大陆的交点为这个港口的七起始点。

11.1.3 代码链接

[MicroCity Web](#)

11.1.4 结果展示图

```
>标注交点 98 (节点 1390): (4.7336, 58.5046)
>标注交点 99 (节点 1397): (-23.7954, 59.1361)
>标注交点 100 (节点 1410): (5.0243, 61.0495)
>标注交点 101 (节点 1454): (144.5638, 15.6857)
>标注交点 102 (节点 1456): (139.2109, 14.1533)
>标注交点 103 (节点 1457): (138.2609, 13.9411)
>标注交点 104 (节点 1479): (128.1420, 31.7470)
>标注交点 105 (节点 1495): (148.2811, -5.5686)
>标注交点 106 (节点 1515): (130.1796, 32.6940)
>=== 三大港口最近节点匹配 ===
>鹿特丹港 -> 节点 218 (4.2985, 52.7402), 偏差距离: 158.37 公里
>宁波舟山港 -> 节点 123 (113.6624, 13.9346), 偏差距离: 581.29 公里
>桑托斯港 -> 节点 569 (-61.7771, 10.7402), 偏差距离: 1261.59 公里
>=== 三大港口最短路径结果 ===
>鹿特丹港 ↔ 宁波舟山港 最短路径总距离: 24258.74 公里
>路径交点序列 (对应图中红色编号) : 17 -> 24 -> 6 -> 7 -> 22 -> 72 -> 94 -> 89 -> 90 -> 75 -> 76 -> 68 -> 12
>鹿特丹港 ↔ 桑托斯港 最短路径总距离: 8229.64 公里
>路径交点序列 (对应图中红色编号) : 17 -> 24 -> 5 -> 20 -> 56 -> 55
>宁波舟山港 ↔ 桑托斯港 最短路径总距离: 26810.50 公里
>路径交点序列 (对应图中红色编号) : 12 -> 68 -> 76 -> 75 -> 90 -> 89 -> 94 -> 72 -> 25 -> 57 -> 55
>=== 三大港口可视化标记 ===
>标记港口 鹿特丹港 @ 节点 218 (4.2985, 52.7402)
>标记港口 桑托斯港 @ 节点 569 (-61.7771, 10.7402)
>标记港口 宁波舟山港 @ 节点 123 (113.6624, 13.9346)
```



11.2 问题 2

在解决了“路”的问题后，根据老师课上内容，我们应进一步引入供需关系，构建了一个基础的 CVRP 模型。

11.2.1 数学模型

1. 符号定义：

(1). 集合定义

符号	定义
$V = \{0, 1, \dots, n\}$	节点集合。其中 0 代表供应场站（港口）， $1 \dots n$ 代表客户需求点（国家）。
K	可用船舶（车辆）集合。

(2). 参数定义

符号	定义
c_{ij}	节点 i 到节点 j 的运输成本（本代码中体现为通过 Dijkstra 算出的海运距离）。
d_i	客户 i 的需求量。
q_k	船舶 k 的最大载重（即港口的供应能力限制）。

(3). 决策变量

符号	定义
x_{ij}^k	0-1 变量。若船舶 k 从节点 i 行驶到节点 j ，则为 1，否则为 0。

y_{ik} 0-1 变量。若客户 i 的需求由船舶 k 满足，则为 1。

2. 模型构建：

目标是最小化总运输成本（总航程）：

$$\text{Min } Z = \sum_{k \in K} \sum_{i \in V} \sum_{j \in V} c_{ij} x_{ij}^k$$

3. 约束：

(1). 访问约束：每个客户的需求必须被满足（在代码中通过供需扣减逻辑实现）。

$$\sum_{k \in K} y_{ik} = 1, \forall i \in \{1, \dots, n\}$$

(2). 流平衡约束：船舶进入一个节点必须离开该节点（路径连续性）。

$$\sum_{i \in V} x_{ij}^k - \sum_{i \in V} x_{ji}^k = 0, \forall j \in V, \forall k \in K$$

(3). 容量/资源约束：这是本次代码最核心的约束。任何一条路径上的总需求不能超过港口的供应上限。

$$\sum_{i=1}^n d_i y_{ik} \leq q_k, \forall k \in K$$

正如老师课上所提到的那样，这是一个经典的 NP-Hard 问题。即使数学模型给出了最优解的定义，但在实际计算中，求解非常耗时。因此，在代码实现层面，我根据老师课上所讲的启发式算法来近似求解该模型，在保证“供需平衡”这一硬约束的前提下，追求路径成本的局部最优。

11.2.2 代码逻辑

为了求解上述数学模型，我编写了名为 `solve_supply_demand` 的核心函数。相比于简单的连线，这段代码实现了严密的逻辑判断，具体流程如下：

1. 成本矩阵预计算（初始化 c_{ij} ）

模型中的 c_{ij} 是决策的基础。我没有使用直线距离，而是复用了问题 1 中的逻辑。

- 首先利用 `findClosestNode` 将所有港口（鹿特丹、宁波、桑托斯）和客户（中、美、巴、德、澳）映射到航运网络的最近的交点上。
- 利用 Dijkstra 算法，计算出每一个“港口-客户”对之间的最短海运距离。
- 构建了一个嵌套表 `costs[port_id][cust_id]`，存储了精确的 c_{ij} 值，作为后续贪婪选择的依据。

2. 设置资源与需求（初始化 d_i 和 q_k ）

我在 Lua 表中设置了每个港口的资源和每个客户的需求：

```

local ports = {}
print("=== 三大港口（场站）节点匹配 ===")
for _, def in ipairs(portDefinitions) do
    local plat, plon = getPortLatLon(def)
    if plat and plon then
        local nodeId, d = findClosestNode(plat, plon)
        if nodeId then
            ports[def.key] = {id = nodeId, def = def, matchDistance = d, supply = 0}
            local node = nodeSet[nodeId]
            print(string.format("%s -> 节点 %d (%.4f, %.4f)",
                                def.name, nodeId, node.lat, node.lon))
        else
            print(string.format("%s 未找到匹配节点", def.name))
        end
    else
        print(string.format("%s 未找到对应国家中心坐标", def.name))
    end
end
end

-- 设置港口供应量
ports["rotterdam"].supply = 90
ports["ningbo"].supply = 120
ports["santos"].supply = 55

-- 客户定义：国家名称、需求量
local customerDefinitions = {
    {name = "中国", name_en = "China", demand = 60},
    {name = "美国", name_en = "United States of America", demand = 45},
    {name = "巴西", name_en = "Brazil", demand = 20},
    {name = "德国", name_en = "Germany", demand = 30},
    {name = "澳大利亚", name_en = "Australia", demand = 15}
}

```

3. 构建配送和调度逻辑（求解 x_{ij} ）

这是代码最复杂的部分，我采用了双重循环结构来逼近最优解：

- (1). 外层循环（遍历客户）：依次处理每个客户的需求 d_i 。
- (2). 内层排序（决策选择）：
 - 对于当前客户，系统会将所有港口按照距离由近到远进行排序（试图最小化 c_{ij} ）。
- (3). 核心调度算法（处理约束）：
 - 判定库存：检查当前距离最近的港口 k 的剩余供应量 S_k 。
 - 情况 A：库存充足 ($S_k \geq d_i$)。
 - 直接分配：建立 $k \rightarrow i$ 的配送关系。
 - 更新状态： $S_k = S_k - d_i$ ，客户需求清零。
 - 情况 B：库存不足 ($S_k < d_i$) —— 创新点。
 - 多源拆单：这是处理复杂情况的关键。如果最近的港口（例如桑托斯）只有 100 吨货，但客户（中国）需要 500 吨。
 - 操作：先将桑托斯的 100 吨全部运走 ($S_{Santos} = 0$)。
 - 剩余需求：客户仍有 400 吨需求 ($d_i = d_i - 100$)。

- 循环：代码自动转向次近的港口（例如宁波）继续请求剩余的 400 吨，直到需求被完全满足。

4. 结果可视化

不同于以往的静态展示，这里有一个逆向过滤的逻辑：

- 代码维护了一个 `highlightedRoutes` 表，记录所有被决策变量 $x_{ij} = 1$ 选中的航线段。
- 在渲染循环中，强制将所有未被选中的航线透明度设为 0。
- 根据出发港口的 ID，动态赋予航线颜色（如宁波出发为红色，鹿特丹出发为绿色），从而直观地展示出模型的求解结果。

11.2.3 创新点

创新点一：利用启发式的逻辑去解决 NP-hard 问题，处理资源约束和多源配送。

创新点二：在可视化的部分，将没被选择的路线隐藏掉，只保留被选中的航线，同时利用不同颜色去表示。

创新点三：我在代码中实现了一个物流调度逻辑，让这段代码具备了“思考”能力

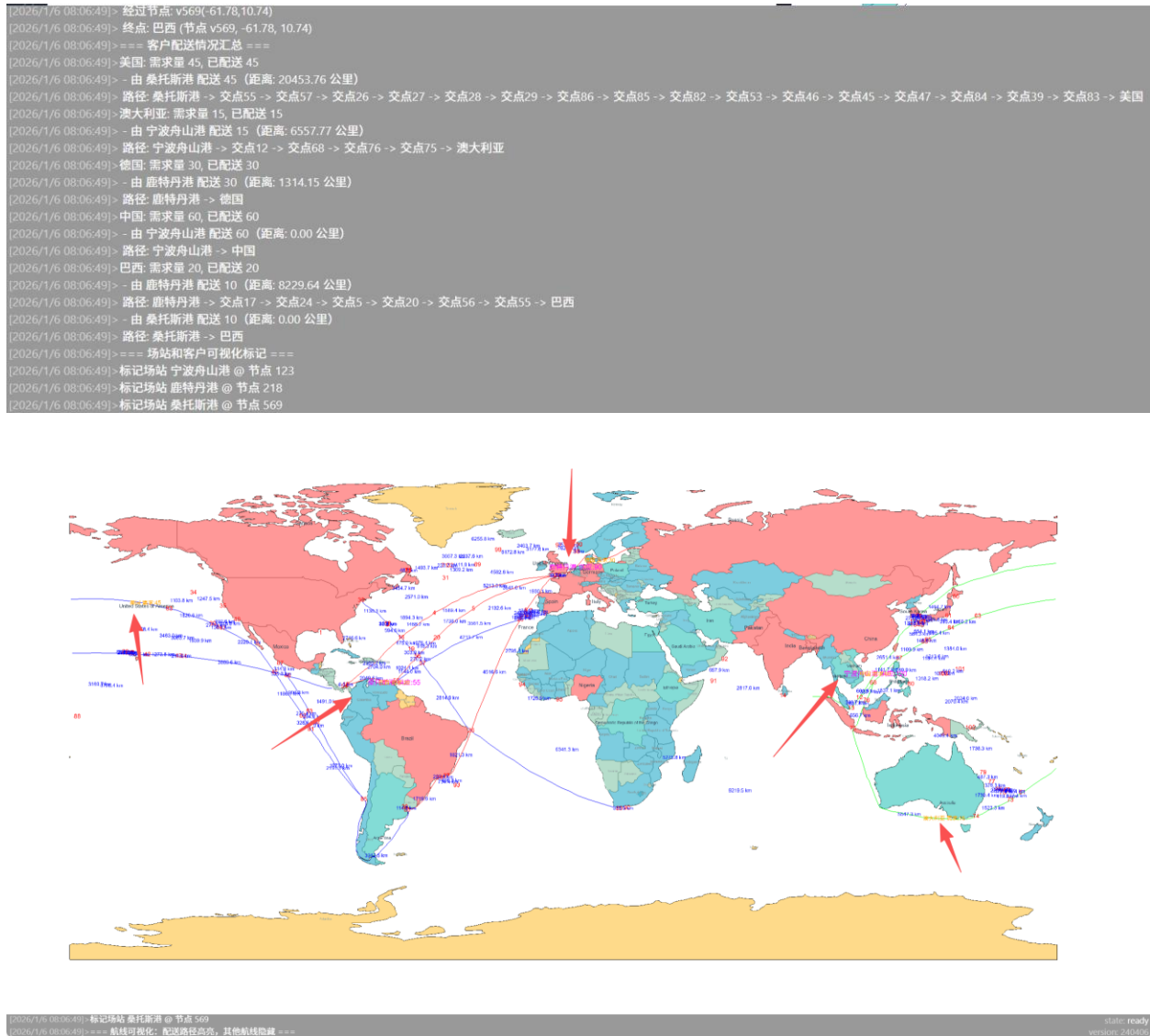
创新点四：考虑到，我绘制的航线，不一定精准的经过某个国家的某个港口，所以我通过“国家中心 -> 最近节点”的分配机制，代码可以适应任何形状的航线地图，无需人工指定节点 ID。

11.2.4 代码链接

[MicroCity Web](#)

11.2.5 结果展示图

```
[2026/1/6 08:06:48]> 桑托斯港 -> 节点 569 (-61.7771, 10.7402)
[2026/1/6 08:06:48]> === 客户节点匹配 ===
[2026/1/6 08:06:48]> 中国 -> 节点 123 (113.6624, 13.9346), 需求量: 60
[2026/1/6 08:06:48]> 美国 -> 节点 991 (-152.4666, 39.7975), 需求量: 45
[2026/1/6 08:06:48]> 巴西 -> 节点 569 (-61.7771, 10.7402), 需求量: 20
[2026/1/6 08:06:48]> 德国 -> 节点 227 (13.1779, 55.2956), 需求量: 30
[2026/1/6 08:06:48]> 澳大利亚 -> 节点 800 (138.7135, -38.6775), 需求量: 15
[2026/1/6 08:06:48]> === CVRP配送路径规划 ===
[2026/1/6 08:06:49]> === 配送方案详情 ===
[2026/1/6 08:06:49]> 宁波舟山港配送详情 (供应量: 120, 已用: 75, 总距离: 6557.77 公里) :
[2026/1/6 08:06:49]> 配送任务 1: 送往 澳大利亚, 配送量: 15, 距离: 6557.77 公里
[2026/1/6 08:06:49]> 起点: 宁波舟山港 (节点 v123, 113.66, 13.93)
[2026/1/6 08:06:49]> 段 1: 宁波舟山港 -> 交点12
[2026/1/6 08:06:49]> 经过节点: v123(113.66,13.93) -> v124(112.96,13.16) -> v125(112.03,11.78) -> v126(111.46,10.83) -> v127(110.71,9.57) -> v128(109.82,8.13) -> v129(108.53,6.55) -> v130(107.87,5.37)[交点12]
[2026/1/6 08:06:49]> 段 2: 交点12 -> 交点68
[2026/1/6 08:06:49]> 经过节点: v716(110.90,8.05) -> v715(112.91,10.79)[交点68]
[2026/1/6 08:06:49]> 段 3: 交点68 -> 交点76
[2026/1/6 08:06:49]> 经过节点: v831(112.36,8.52) -> v830(110.43,4.35)[交点76]
[2026/1/6 08:06:49]> 段 4: 交点76 -> 交点75
[2026/1/6 08:06:49]> 经过节点: v829(110.42,4.32) -> v828(108.77,0.80) -> v827(107.77,-2.21) -> v826(107.05,-4.36) -> v825(105.59,-5.88)[交点75]
[2026/1/6 08:06:49]> 段 5: 交点75 -> 澳大利亚
[2026/1/6 08:06:49]> 经过节点: v824(105.47,-6.00) -> v823(105.47,-7.07) -> v822(105.97,-9.06) -> v821(106.76,-11.67) -> v820(107.33,-13.28) -> v819(108.48,-16.54) -> v818(109.34,-19.20) -> v817(110.06,-21.75) -> v816(110.71,-24.28) -> v815(111.33,-26.82) -> v814(111.95,-28.34) -> v813(112.57,-29.86) -> v812(113.19,-31.38) -> v811(113.81,-32.90) -> v810(114.43,-34.42) -> v809(115.05,-35.94) -> v808(115.67,-37.46) -> v807(116.29,-38.98) -> v806(116.91,-40.50) -> v805(117.53,-42.02) -> v804(118.15,-43.54) -> v803(118.77,-45.06) -> v802(119.39,-46.58) -> v801(119.99,-48.10) -> v800(120.61,-49.62) -> v799(121.23,-50.14) -> v798(121.85,-51.66) -> v797(122.47,-53.18) -> v796(123.09,-54.70) -> v795(123.71,-56.22) -> v794(124.33,-57.74) -> v793(124.95,-59.26) -> v792(125.57,-60.78) -> v791(126.19,-62.30) -> v790(126.81,-63.82) -> v789(127.43,-65.34) -> v788(128.05,-66.86) -> v787(128.67,-68.38) -> v786(129.29,-69.90) -> v785(129.91,-71.42) -> v784(130.53,-72.94) -> v783(131.15,-74.46) -> v782(131.77,-75.98) -> v781(132.39,-77.50) -> v780(133.01,-79.02) -> v779(133.63,-80.54) -> v778(134.25,-82.06) -> v777(134.87,-83.58) -> v776(135.49,-85.10) -> v775(136.11,-86.62) -> v774(136.73,-88.14) -> v773(137.35,-89.66) -> v772(137.97,-91.18) -> v771(138.59,-92.70) -> v770(139.21,-94.22) -> v769(139.83,-95.74) -> v768(140.45,-97.26) -> v767(141.07,-98.78) -> v766(141.69,-100.30) -> v765(142.31,-101.82) -> v764(142.93,-103.34) -> v763(143.55,-104.86) -> v762(144.17,-106.38) -> v761(144.79,-107.90) -> v760(145.41,-109.42) -> v759(146.03,-110.94) -> v758(146.65,-112.46) -> v757(147.27,-113.98) -> v756(147.89,-115.50) -> v755(148.51,-117.02) -> v754(149.13,-118.54) -> v753(149.75,-120.06) -> v752(150.37,-121.58) -> v751(150.99,-123.10) -> v750(151.61,-124.62) -> v749(152.23,-126.14) -> v748(152.85,-127.66) -> v747(153.47,-129.18) -> v746(154.09,-130.70) -> v745(154.71,-132.22) -> v744(155.33,-133.74) -> v743(155.95,-135.26) -> v742(156.57,-136.78) -> v741(157.19,-138.30) -> v740(157.81,-139.82) -> v739(158.43,-141.34) -> v738(159.05,-142.86) -> v737(159.67,-144.38) -> v736(160.29,-145.90) -> v735(160.91,-147.42) -> v734(161.53,-148.94) -> v733(162.15,-150.46) -> v732(162.77,-151.98) -> v731(163.39,-153.50) -> v730(164.01,-155.02) -> v729(164.63,-156.54) -> v728(165.25,-158.06) -> v727(165.87,-159.58) -> v726(166.49,-161.10) -> v725(167.11,-162.62) -> v724(167.73,-164.14) -> v723(168.35,-165.66) -> v722(168.97,-167.18) -> v721(169.59,-168.70) -> v720(170.21,-170.22) -> v719(170.83,-171.74) -> v718(171.45,-173.26) -> v717(172.07,-174.78) -> v716(172.69,-176.30) -> v715(173.31,-177.82) -> v714(173.93,-179.34) -> v713(174.55,-180.86) -> v712(175.17,-182.38) -> v711(175.79,-183.90) -> v710(176.41,-185.42) -> v709(177.03,-186.94) -> v708(177.65,-188.46) -> v707(178.27,-189.98) -> v706(178.89,-191.50) -> v705(179.51,-193.02) -> v704(180.13,-194.54) -> v703(180.75,-196.06) -> v702(181.37,-197.58) -> v701(181.99,-199.10) -> v700(182.61,-200.62) -> v699(183.23,-202.14) -> v698(183.85,-203.66) -> v697(184.47,-205.18) -> v696(185.09,-206.70) -> v695(185.71,-208.22) -> v694(186.33,-209.74) -> v693(186.95,-211.26) -> v692(187.57,-212.78) -> v691(188.19,-214.30) -> v690(188.81,-215.82) -> v689(189.43,-217.34) -> v688(190.05,-218.86) -> v687(190.67,-220.38) -> v686(191.29,-221.90) -> v685(191.91,-223.42) -> v684(192.53,-224.94) -> v683(193.15,-226.46) -> v682(193.77,-227.98) -> v681(194.39,-229.50) -> v680(195.01,-231.02) -> v679(195.63,-232.54) -> v678(196.25,-234.06) -> v677(196.87,-235.58) -> v676(197.49,-237.10) -> v675(198.11,-238.62) -> v674(198.73,-240.14) -> v673(199.35,-241.66) -> v672(199.97,-243.18) -> v671(200.59,-244.70) -> v670(201.21,-246.22) -> v669(201.83,-247.74) -> v668(202.45,-249.26) -> v667(203.07,-250.78) -> v666(203.69,-252.30) -> v665(204.31,-253.82) -> v664(204.93,-255.34) -> v663(205.55,-256.86) -> v662(206.17,-258.38) -> v661(206.79,-259.90) -> v660(207.41,-261.42) -> v659(208.03,-262.94) -> v658(208.65,-264.46) -> v657(209.27,-265.98) -> v656(209.89,-267.50) -> v655(210.51,-269.02) -> v654(211.13,-270.54) -> v653(211.75,-272.06) -> v652(212.37,-273.58) -> v651(212.99,-275.10) -> v650(213.61,-276.62) -> v649(214.23,-278.14) -> v648(214.85,-279.66) -> v647(215.47,-281.18) -> v646(216.09,-282.70) -> v645(216.71,-284.22) -> v644(217.33,-285.74) -> v643(217.95,-287.26) -> v642(218.57,-288.78) -> v641(219.19,-290.30) -> v640(219.81,-291.82) -> v639(220.43,-293.34) -> v638(221.05,-294.86) -> v637(221.67,-296.38) -> v636(222.29,-297.90) -> v635(222.91,-299.42) -> v634(223.53,-300.94) -> v633(224.15,-302.46) -> v632(224.77,-303.98) -> v631(225.39,-305.50) -> v630(226.01,-307.02) -> v629(226.63,-308.54) -> v628(227.25,-310.06) -> v627(227.87,-311.58) -> v626(228.49,-313.10) -> v625(229.11,-314.62) -> v624(229.73,-316.14) -> v623(230.35,-317.66) -> v622(230.97,-319.18) -> v621(231.59,-320.70) -> v620(232.21,-322.22) -> v619(232.83,-323.74) -> v618(233.45,-325.26) -> v617(234.07,-326.78) -> v616(234.69,-328.30) -> v615(235.31,-329.82) -> v614(235.93,-331.34) -> v613(236.55,-332.86) -> v612(237.17,-334.38) -> v611(237.79,-335.90) -> v610(238.41,-337.42) -> v609(239.03,-338.94) -> v608(239.65,-340.46) -> v607(240.27,-341.98) -> v606(240.89,-343.50) -> v605(241.51,-345.02) -> v604(242.13,-346.54) -> v603(242.75,-348.06) -> v602(243.37,-349.58) -> v601(243.99,-351.10) -> v600(244.61,-352.62) -> v599(245.23,-354.14) -> v598(245.85,-355.66) -> v597(246.47,-357.18) -> v596(247.09,-358.70) -> v595(247.71,-360.22) -> v594(248.33,-361.74) -> v593(248.95,-363.26) -> v592(249.57,-364.78) -> v591(250.19,-366.30) -> v590(250.81,-367.82) -> v589(251.43,-369.34) -> v588(252.05,-370.86) -> v587(252.67,-372.38) -> v586(253.29,-373.90) -> v585(253.91,-375.42) -> v584(254.53,-376.94) -> v583(255.15,-378.46) -> v582(255.77,-380.98) -> v581(256.39,-382.50) -> v580(257.01,-384.02) -> v579(257.63,-385.54) -> v578(258.25,-387.06) -> v577(258.87,-388.58) -> v576(259.49,-390.10) -> v575(260.11,-391.62) -> v574(260.73,-393.14) -> v573(261.35,-394.66) -> v572(261.97,-396.18) -> v571(262.59,-397.70) -> v570(263.21,-399.22) -> v569(263.83,-400.74) -> v568(264.45,-402.26) -> v567(265.07,-403.78) -> v566(265.69,-405.30) -> v565(266.31,-406.82) -> v564(266.93,-408.34) -> v563(267.55,-409.86) -> v562(268.17,-411.38) -> v561(268.79,-412.90) -> v560(269.41,-414.42) -> v559(270.03,-415.94) -> v558(270.65,-417.46) -> v557(271.27,-418.98) -> v556(271.89,-420.50) -> v555(272.51,-422.02) -> v554(273.13,-423.54) -> v553(273.75,-425.06) -> v552(274.37,-426.58) -> v551(274.99,-428.10) -> v550(275.61,-429.62) -> v549(276.23,-431.14) -> v548(276.85,-432.66) -> v547(277.47,-434.18) -> v546(278.09,-435.70) -> v545(278.71,-437.22) -> v544(279.33,-438.74) -> v543(279.95,-440.26) -> v542(280.57,-441.78) -> v541(281.19,-443.30) -> v540(281.81,-444.82) -> v539(282.43,-446.34) -> v538(283.05,-447.86) -> v537(283.67,-449.38) -> v536(284.29,-450.90) -> v535(284.91,-452.42) -> v534(285.53,-453.94) -> v533(286.15,-455.46) -> v532(286.77,-456.98) -> v531(287.39,-458.50) -> v530(288.01,-459.98) -> v529(288.63,-461.46) -> v528(289.25,-462.94) -> v527(289.87,-464.42) -> v526(290.49,-465.90) -> v525(291.11,-467.38) -> v524(291.73,-468.86) -> v523(292.35,-470.34) -> v522(292.97,-471.82) -> v521(293.59,-473.30) -> v520(294.21,-474.78) -> v519(294.83,-476.26) -> v518(295.45,-477.74) -> v517(296.07,-479.22) -> v516(296.69,-480.70) -> v515(297.31,-482.18) -> v514(297.93,-483.66) -> v513(298.55,-485.14) -> v512(299.17,-486.62) -> v511(299.79,-488.10) -> v510(300.41,-489.58) -> v509(301.03,-491.06) -> v508(301.65,-492.54) -> v507(302.27,-494.02) -> v506(302.89,-495.50) -> v505(303.51,-496.98) -> v504(304.13,-498.46) -> v503(304.75,-500.94) -> v502(305.37,-502.42) -> v501(305.99,-503.90) -> v500(306.61,-505.38) -> v499(307.23,-506.86) -> v498(307.85,-508.34) -> v497(308.47,-509.82) -> v496(309.09,-511.30) -> v495(309.71,-512.78) -> v494(310.33,-514.26) -> v493(310.95,-515.74) -> v492(311.57,-517.20) -> v491(312.19,-518.68) -> v490(312.81,-520.14) -> v489(313.43,-521.60) -> v488(314.05,-523.06) -> v487(314.67,-524.52) -> v486(315.29,-525.98) -> v485(315.91,-527.44) -> v484(316.53,-528.90) -> v483(317.15,-530.36) -> v482(317.77,-531.82) -> v481(318.39,-533.28) -> v480(319.01,-534.74) -> v479(319.63,-536.20) -> v478(320.25,-537.66) -> v477(320.87,-539.12) -> v476(321.49,-540.58) -> v475(322.11,-542.04) -> v474(322.73,-543.50) -> v473(323.35,-544.96) -> v472(323.97,-546.42) -> v471(324.59,-547.88) -> v470(325.21,-549.34) -> v469(325.83,-550.80) -> v468(326.45,-552.26) -> v467(327.07,-553.72) -> v466(327.69,-555.18) -> v465(328.31,-556.64) -> v464(328.93,-558.10) -> v463(329.55,-559.56) -> v462(330.17,-561.02) -> v461(330.79,-562.48) -> v460(331.41,-563.94) -> v459(332.03,-565.40) -> v458(332.65,-566.86) -> v457(333.27,-568.32) -> v456(333.89,-569.78) -> v455(334.51,-571.24) -> v454(335.13,-572.70) -> v453(335.75,-574.16) -> v452(336.37,-575.62) -> v451(336.99,-577.08) -> v450(337.61,-578.54) -> v449(338.23,-580.00) -> v448(338.85,-581.46) -> v447(339.47,-582.92) -> v446(340.09,-584.38) -> v445(340.71,-585.84) -> v444(341.33,-587.30) -> v443(341.95,-588.76) -> v442(342.57,-590.22) -> v441(343.19,-591.68) -> v440(343.81,-593.14) -> v439(344.43,-594.60) -> v438(345.05,-596.06) -> v437(345.67,-597.52) -> v436(346.29,-598.98) -> v435(346.91,-600.44) -> v434(347.53,-601.90) -> v433(348.15,-603.36) -> v432(348.77,-604.82) -> v431(349.39,-606.28) -> v430(350.01,-607.74) -> v429(350.63,-609.20) -> v428(351.25,-610.66) -> v427(351.87,-612.12) -> v426(352.49,-613.58) -> v425(353.11,-615.04) -> v424(353.73,-616.50) -> v423(354.35,-617.96) -> v422(354.97,-619.42) -> v421(355.59,-620.88) -> v420(356.21,-622.34) -> v419(356.83,-623.80) -> v418(357.45,-625.26) -> v417(358.07,-626.72) -> v416(358.69,-628.18) -> v415(359.31,-629.64) -> v414(359.93,-631.10) -> v413(360.55,-632.56) -> v412(361.17,-634.02) -> v411(361.79,-635.48) -> v410(362.41,-636.94) -> v409(363.03,-638.40) -> v408(363.65,-639.86) -> v407(364.27,-641.32) -> v406(364.89,-642.78) -> v405(365.51,-644.24) -> v404(366.13,-645.70) -> v403(366.75,-647.16) -> v402(367.37,-648.62) -> v401(367.99,-650.08) -> v400(368.61,-651.54) -> v399(369.23,-653.00) -> v398(369.85,-654.46) -> v397(370.47,-655.92) -> v396(371.09,-657.38) -> v395(371.71,-658.84) -> v394(372.33,-660.30) -> v393(372.95,-661.76) -> v392(373.57,-663.22) -> v391(374.19,-664.68) -> v390(374.81,-666.14) -> v389(375.43,-667.60) -> v388(376.05,-669.06) -> v387(376.67,-670.52) -> v386(377.29,-671.98) -> v385(377.91,-673.44) -> v384(378.53,-674.90) -> v383(379.15,-676.36) -> v382(379.77,-677.82) -> v381(380.39,-679.28) -> v380(381.01,-680.74) -> v379(381.63,-682.20) -> v378(382.25,-683.66) -> v377(382.87
```



11.3 问题 3

在问题 3 中，我使用了老师课上所讲的 CW 节约算法，这不是贪婪策略，而是一个经典的启发式算法。

11.3.1 算法理解

在课上，老师曾提到过，VRP 问题的复杂度。对于只有几个客户的情况，或许可以穷举；但当节点增加时，路径组合呈指数级爆炸。因此，我们需要一种虽然求不出来精确解，但是求得很快的算法。

1. 算法核心原理：我认为 CW 算法的本质是“通过合并路径来减少总里程”。

- 初始状态：假设每个客户都有单独的船舶进行“港口 $\leftrightarrow$ 客户”的往返配送。
- 优化逻辑：如果有两个客户 i 和 j ，如果我们两条往返路线合并为一条“港口 $\rightarrow i \rightarrow j \rightarrow$ 港口”的路线，能否省油？

- 节约值公式：

$$S_{ij} = c_{i0} + c_{0j} - c_{ij}$$

- c_{i0}, c_{0j} : 客户到港口的直接距离。
- c_{ij} : 客户之间的距离。
- 物理含义: S_{ij} 越大, 说明把 i 和 j 连在一起运输, 相比单独运输, 省下的路程越多。根据三角不等式, 只要 i, j 不共线, S_{ij} 通常大于 0。

2. 适用性分析: CW 算法与本次作业高度匹配 (多点配送、有容量限制)。它是构造性算法, 从零开始构建路径, 计算速度快, 且结果通常优于最近邻法。

11.3.2 代码逻辑

我利用 Lua 代码实现了完整的 CW 迭代逻辑, 整个过程分为三个部分:

1. 规定数据与计算数据

在进行优化前, 必须先知道“所有点之间的距离”。

- 我复用了问题 1 的模块, 利用 Dijkstra 算法计算了一个完整的 OD 成本矩阵。
- 不仅计算了港口 $\rightarrow$ 客户的距离, 更关键的是计算了客户 $\leftrightarrow$ 客户之间的距离。这是计算 S_{ij} 的基础。

2. 节约值计算与排序

这是算法的“预处理”阶段。

- 遍历: 代码对归属于同一港口下的所有客户节点对 (i, j) 进行遍历。
- 计算: 套用公式 $savings = dist[port][i] + dist[port][j] - dist[i][j]$ 。
- 排序: 将所有计算出的 S_{ij} 放入一个列表, 并按从大到小排序。这意味着我们优先考虑“最顺路”的两个客户进行合并。

3. 路径合并循环 (核心)

这是一个十分复杂的模块, 需要处理复杂的拓扑约束。我使用了一个 While 循环来处理排序后的节约值列表:

(1). 取出最大节约项: 取出当前 S_{ij} 最大的客户对 (i, j) 。

(2). 校验合法性:

- 归属检查: i 和 j 是否已经被分配到了同一条路径中? (避免形成子回路)。
- 端点检查: 在当前的路径片段中, i 和 j 必须是路径的端点。因为只能在路径的两头延长, 不能从中间“插队”。
- 容量检查: 合并后的总需求量 $\sum d$ 是否超过了船舶的最大载重 Q ?

if (load_route_i + load_route_j <= max_capacity) then ...

(3). 执行合并:

- 如果上述条件都满足，修改链表指针，将 i 所在的路径和 j 所在的路径首尾相接。
- 更新该新路径的总载重。

(4). 重复：直到遍历完所有节约值或无法再合并为止。

11.3.3 创新点

创新点一：利用启发式的逻辑去解决 NP-hard 问题，处理资源约束和多源配送。

创新点二：在可视化的部分，将没被选择的路线隐藏掉，只保留被选中的航线，同时利用不同颜色去表示。

创新点三：引入了有限供应量。模拟了“就近港口缺货，必须远距离调货”。

创新点四：考虑到，我绘制的航线，不一定精准的经过某个国家的某个港口，所以我通过“国家中心 -> 最近节点”的分配机制，代码可以适应任何形状的航线地图，无需人工指定节点 ID。

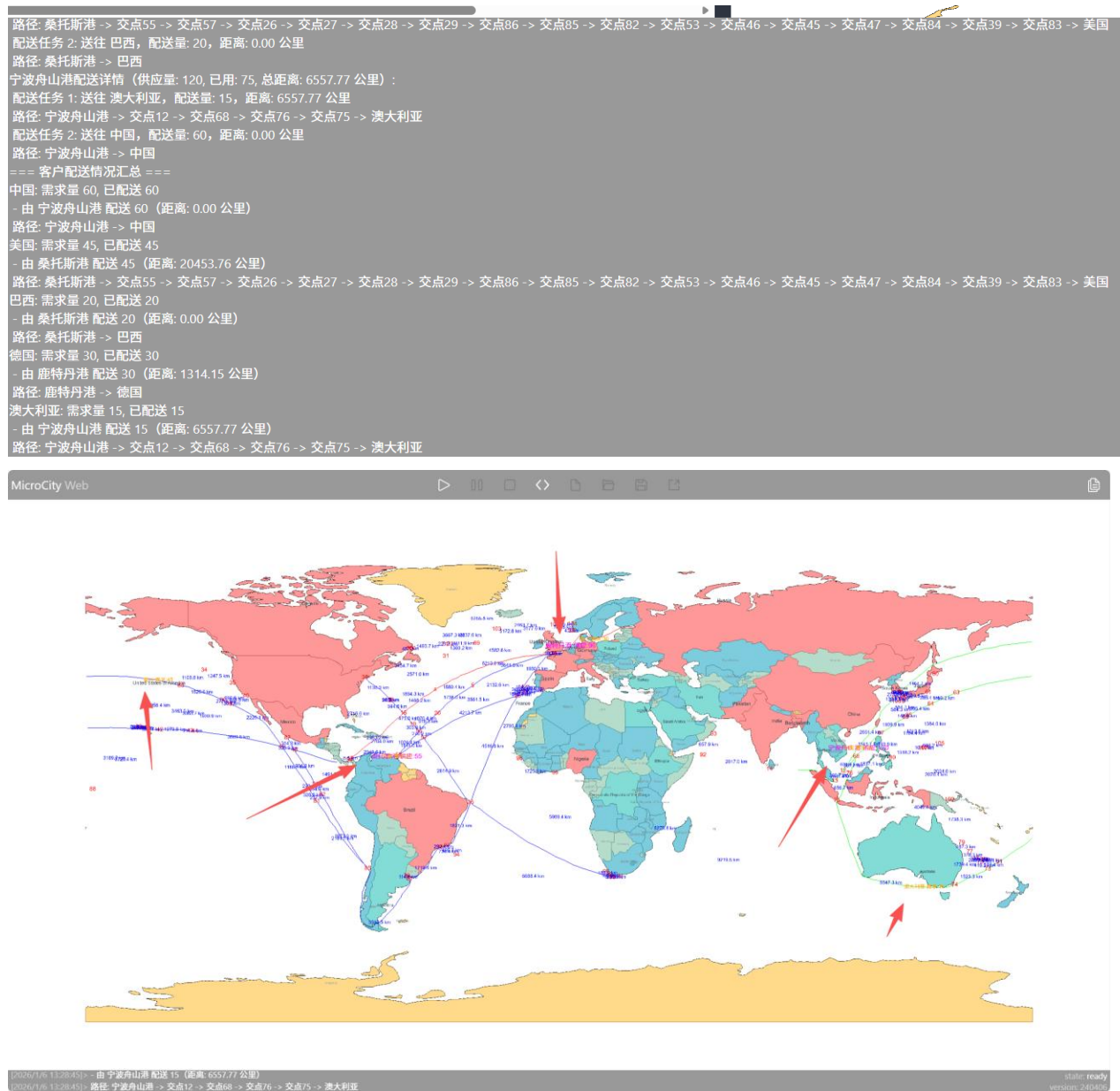
11.3.4 代码链接

[MicroCity Web](#)

11.3.5 结果展示图

```

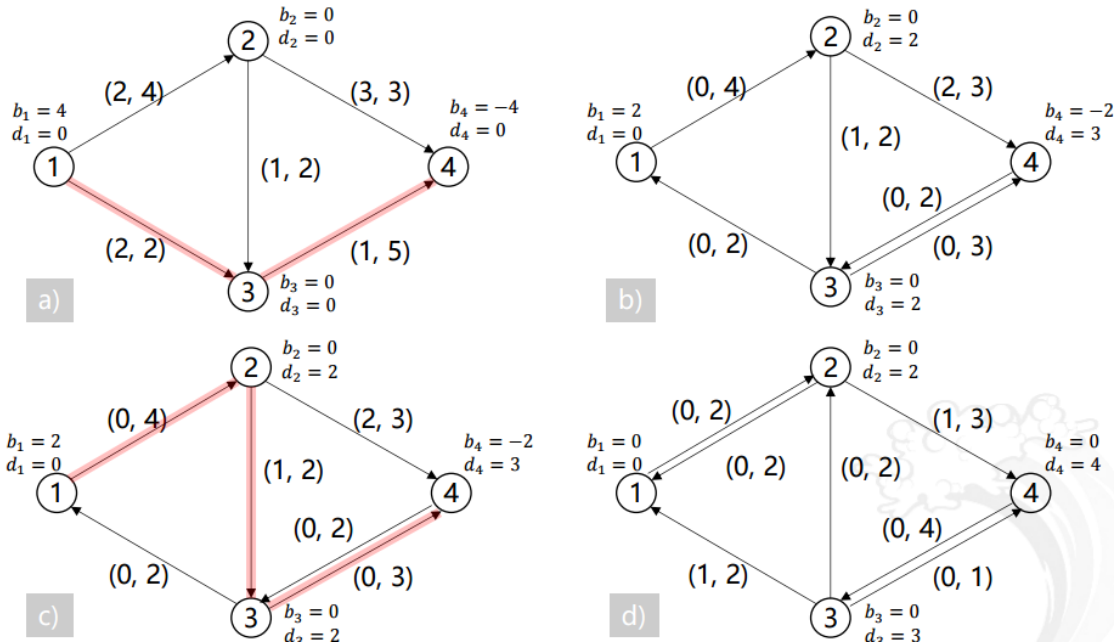
=== CVRP配送路径规划（CW节约算法） ===
=== 开始分配客户需求 ===
客户总数: 0
客户 1: 美国, 需求 45, 节点 991
客户 2: 德国, 需求 30, 节点 227
客户 3: 巴西, 需求 20, 节点 569
客户 4: 中国, 需求 60, 节点 123
客户 5: 澳大利亚, 需求 15, 节点 800
实际客户总数: 5
分配客户 美国 (需求: 45)...
找到 3 个可达场站
-> 桑托斯港 分配 45 (剩余需求: 0)
✓ 美国 需求全部分配完成
分配客户 德国 (需求: 30)...
找到 3 个可达场站
-> 鹿特丹港 分配 30 (剩余需求: 0)
✓ 德国 需求全部分配完成
分配客户 巴西 (需求: 20)...
找到 3 个可达场站
-> 桑托斯港 分配 10 (剩余需求: 10)
-> 鹿特丹港 分配 10 (剩余需求: 0)
✓ 巴西 需求全部分配完成
分配客户 中国 (需求: 60)...
找到 3 个可达场站
    
```



12 作业 12

问题：实现连续最短路径法，用程序实现下列问题。

网络流问题的算法：连续最短路径法



12.1 算法理解

通过老师课上的讲解，我理解了连续最短路径法的核心思想：它不仅仅是寻找路径，而是一个在残量网络上不断迭代、利用势来修正边权的过程。

针对课上的例题，我的算法执行流程如下：

12.1.1 初始化：

- 构建残量网络，初始流量均为 0，所有节点的势 $d_i = 0$ 。
- 确定供需关系：节点 1 供给 4，节点 4 需求 4。

12.1.2 迭代过程：

- 步骤一：计算约减费用。为了处理负权边（反向弧）并复现 Dijkstra 算法，我应用了课件公式 $c_{ij}^d = c_{ij} + d_i - d_j$ 。
- 步骤二：寻找最短路。在残量网络中，以 c_{ij}^d 为权重，寻找从源点（S=1）到汇点（T=4）的最短路径。
- 步骤三：势的更新。这是算法的关键，利用本轮计算的最短路距离更新节点的势： $d_i = d_i + dist[i]$ ，保证下一次迭代边权非负。
- 步骤四：流量增广。找出该路径上的瓶颈容量 δ ，将流量推送到路径上，同时更新正向弧剩余容量和反向弧容量。

12.1.3 终止条件:

当总流量达到需求量 (Demand=4) 或无法找到新的路径时, 算法结束。

12.2 代码逻辑

12.2.1 数据结构储存

我使用了邻接表来存储图结构。为了方便处理残量网络, 每条边都包含一个 rev 指针指向其反向边:

```
-- 定义前向边结构
local fwd = {to=v, cap=cap, init_cap=cap, cost=cost, flow=0, rev=(#g_v + 1)}
table.insert(g_u, fwd) -- 将前向边加入u的列表

-- 定义反向边结构 (容量为0, 费用取反)
local bwd = {to=u, cap=0, init_cap=0, cost=-cost, flow=0, rev=#g_u}
table.insert(g_v, bwd) -- 将反向边加入v的列表
```

这样在增广流量时, 我可以同时减少正向边的 cap 并增加反向边的 cap, 完美模拟残量网络的变化。

12.2.2 核心算法: min_cost_flow

在 solve 函数中, 我实现了算法的主循环:

- Dijkstra: 代码实现了标准的 Dijkstra 算法。特别注意的是: `local nd = dist[u] + e.cost + potential[u] - potential[e.to]`。这行代码完成了利用势修正边权核心理论。
- 路径回溯与增广: 通过 `parent_node` 数组回溯找到路径, 计算瓶颈流量 `delta`, 并更新图的残量。

12.2.3 结果验证

经过程序运行, 最终输出结果为:

```
状态 1 - 初始化残量网络
-----
节点(供需b_i,势d_i):
1(b=4,d=0) 2(b=0,d=0) 3(b=0,d=0) 4(b=-4,d=0)
边(流量/容量;费用c_ij), *为本轮最短路:
1->2(0/4;c=2) 2->4(0/3;c=3)
1->3(0/2;c=2) 3->4(0/5;c=1)
2->3(0/2;c=1)
```

状态 2 - 找到最短路 增广量=2

节点(供需 b_i ,势 d_i):

1($b=4, d=0$) 2($b=0, d=2$) 3($b=0, d=2$) 4($b=-4, d=3$)

边(流量/容量;费用 c_{ij}), *为本轮最短路:

1→2(0/4; $c=2$) 2→4(0/3; $c=3$)

*1→3(0/2; $c=2$) *3→4(0/5; $c=1$)

2→3(0/2; $c=1$)

状态 3 - 增广结束 累计流量=2 累计费用=6

节点(供需 b_i ,势 d_i):

1($b=4, d=0$) 2($b=0, d=2$) 3($b=0, d=2$) 4($b=-4, d=3$)

边(流量/容量;费用 c_{ij}), *为本轮最短路:

1→2(0/4; $c=2$) 2→4(0/3; $c=3$)

1→3(2/2; $c=2$) 3→4(2/5; $c=1$)

2→3(0/2; $c=1$)

状态 4 - 找到最短路 增广量=2

节点(供需 b_i ,势 d_i):

1($b=4, d=0$) 2($b=0, d=2$) 3($b=0, d=3$) 4($b=-4, d=4$)

边(流量/容量;费用 c_{ij}), *为本轮最短路:

*1→2(0/4; $c=2$) 2→4(0/3; $c=3$)

1→3(2/2; $c=2$) *3→4(2/5; $c=1$)

*2→3(0/2; $c=1$)

```

状态 5 - 增广结束 累计流量=4 累计费用=14
-----
节点(供需b_i,势d_i):
1(b=4,d=0) 2(b=0,d=2) 3(b=0,d=3) 4(b=-4,d=4)
边(流量/容量;费用c_ij), *为本轮最短路:
1->2(2/4;c=2) 2->4(0/3;c=3)

1->3(2/2;c=2) 3->4(4/5;c=1)

2->3(2/2;c=1)
-----

===== 最终结果 =====
需求总流量 4
实际发送流量 4
最小总费用 14
各弧最终流量 (i->j : current/capacity; cost):
1->2 : 2/4; cost=2
1->3 : 2/2; cost=2
2->3 : 2/2; cost=1
2->4 : 0/3; cost=3
3->4 : 4/5; cost=1
    
```

这与老师课上所讲结果完全一致，证明代码逻辑正确。

12.2.4 代码链接

[MicroCity Web](#)

12.3 创新版本

在完成基础算法后，我尝试进行创新扩展。

12.3.1 失败版本

一开始，我想把连续最短路径法应用到绘制的航线图里面，调用 `geojson`，后来我发现航线是没有权重的，然后经过询问老师，老师让我把一条航线的最大航运量当作权重，最后我还是没理解到位，没做出来。

12.3.2 最终创新

最后，我决定做一个可视化，把这个算法演绎出来：

- 技术实现：

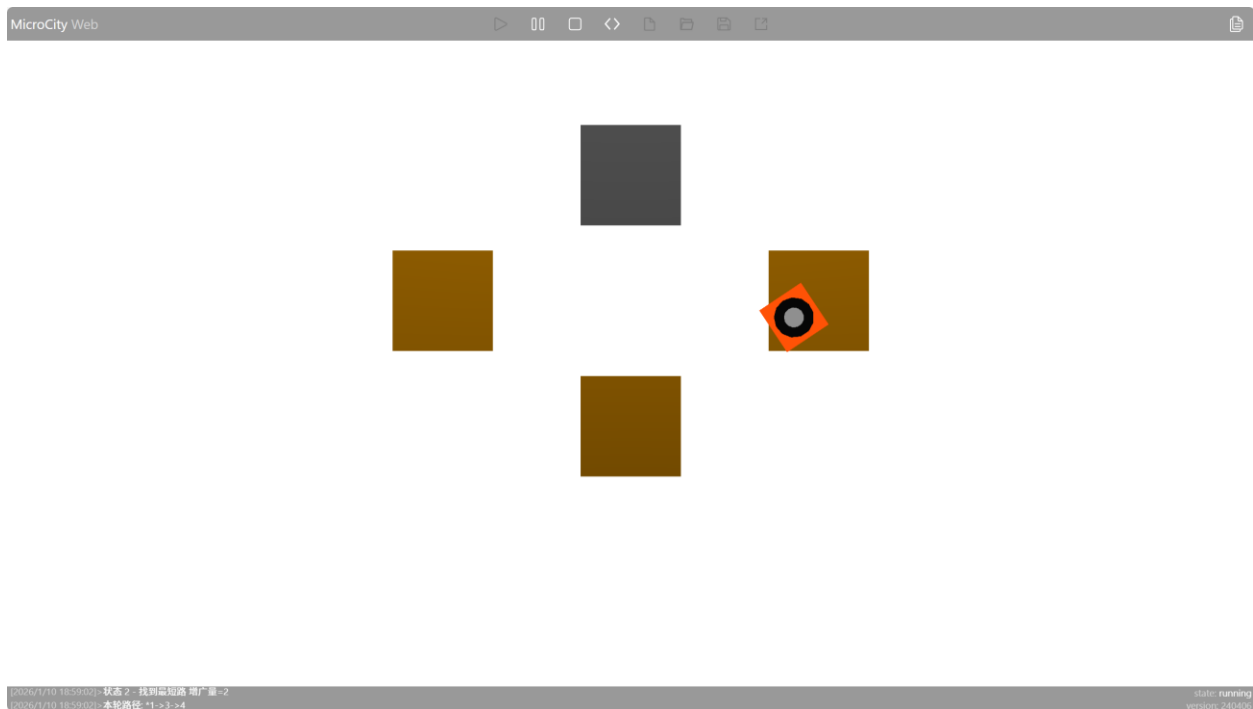
我没有让算法直接做出可视化，我觉得这样会卡顿，而是设计了一个“录像机”机制。在算法计算阶段，我使用 `record_path` 和 `record_print` 函数，将算法的每一步，比如“找到路径”、“计算增广量”等等，存入 `action_queue` 表中。

- 可视化效果：在 MicroCity 的渲染循环中，我读取这个队列并进行可视化呈现：
 - (1). 控制台同步：控制台会配合 AGV 的动作，实时打印当前是“状态 1”还是“状态 2”，完全还原了课上所讲解题步骤思路。
 - (2). AGV 实体演示：我从 MicroCity 中，复制了一个 AGV 的代码，然后加载到了这里面，当算法计算出一条最短路（例如 1->2->4）时，AGV 会真正在可视化场景中沿着这条路径行驶。
 - (3). 路径可视：为了更清晰的展示出 AGV 的移动轨迹，我让 AGV 移动到某个节点时，网格会变为橙色，直观展示流量的流向。

12.3.3 代码链接

[MicroCity Web](#)

12.3.4 结果展示图



MicroCity Web

[2026/1/10 18:59:24]>状态 4 - 找到最短路 增广量=2
[2026/1/10 18:59:24]>本轮路径: *1->2->3->4

state: running
version: 240405

```

>状态 4 - 找到最短路 增广量=2
>本轮路径: *1->2->3->4
>状态 5 - 增广结束 累计流量=4 累计费用=14
>===== 最终结果 =====
>需求总流量 4
>实际发送流量 4
>最小总费用 14
>演示结束
>>> 开始最小费用流可视化展示 <<<
>-----
>状态 1 - 初始化残量网络
>-----
>-----
>状态 2 - 找到最短路 增广量=2
>本轮路径: *1->3->4
>状态 3 - 增广结束 累计流量=2 累计费用=6
>-----
>状态 4 - 找到最短路 增广量=2
>本轮路径: *1->2->3->4
>状态 5 - 增广结束 累计流量=4 累计费用=14
>===== 最终结果 =====
>需求总流量 4
>实际发送流量 4
>最小总费用 14
>演示结束

```

13 作业 13

问题：分别使用 Frank-Wolfe 算法、增量分配法和连续平均法求实例，其道路阻抗为 BPR 函数。

本次作业中，我除了复现老师课上所讲的增量分配法、连续平均法和 Frank-Wolfe 算法外，我还结合了管理运筹的思想，引入了随机用户均衡模型作为创新点。

13.1 代码整体架构与设计思路

首先，我基于老师课上所讲内容，构建了一个 `links_config` 表。这个表存储了路段的拓扑结构，包括零流阻抗 (t_0)、通行能力(C)以及实时流量 (x_a)。这种结构使得代码具有很好的扩展性，就算网络再复杂，只需修改这个配置表就可以处理。

为了保证三个算法调用的统一性，我定义了两个辅助函数：

- (1). BPR 函数：按照老师课上所解读的公式 $t = t_0[1 + \alpha(x/C)^\beta]$ ，编写了 `calc_bpr` 函数。
- (2). 阻抗更新与最短路搜索：将“更新网络的阻抗”和“寻找最短路径”独立为函数。这确保了在 FW 算法和 MSA 算法中，整个代码使用的都是同一套底层逻辑，减少了逻辑错误的可能性。

13.2 三种基础算法的实现逻辑

针对要求使用的三个算法，我分别实现了不同的迭代逻辑：

13.2.1 增量分配法

将总需求 Q 进行了 N 等分（代码中设为 100 份）。

每一份流量都分配给当前阻抗最小的路径，分配后立即更新阻抗。这模拟了用户陆续到达、陆续择路的过程。

13.2.2 连续平均法

这是一种预设步长的迭代算法。

重点在于步长序列的设计。我采用了经典的 $1/n$ 序列。在第 n 次迭代中，新流量 x_{n+1} 等于旧流量 x_n 加上 $1/n$ 倍的搜索方向（即 $y_n - x_n$ ）。

13.2.3 Frank-Wolfe 算法：

根据老师课上所讲，FW 算法是最准确的算法，就是复杂一点。

FW 的核心在于如何寻找最佳迭代步长 λ 。课上老师提到这是一个一维搜索问题。

我在 Lua 中编写了一个二分法求解器。通过计算目标函数的导数 ($\sum (y_i - x_i) \cdot t(x)$)，寻找导数为 0 的点。这比单纯的固定步长更具数学严谨性，确保了算法能快速收敛到 Wardrop 均衡点。

13.3 算法创新点

在完成了基础作业后，我思考了一个问题：Wardrop 第一原理假设所有行走的都是“完全理性”的，且拥有“完美信息”，这在现实中并不成立。

现实中，司机可能对路况感知有误差，或者受到习惯影响。为了模拟这种有限理性，我增加了一个创新算法模块：Logit 模型下的随机用户均衡。

13.3.1 创新思路

新模型：引入 Logit 概率模型。路径被选择的概率 P_k 与其费用的指数成反比： $P_k \propto \exp(-\theta \cdot C_k)$ 。

13.3.2 代码实现

我保留了 MSA 的迭代框架，但替换了核心的流量加载机制。不再单纯寻找 min_cost，而是计算两条路径的选择概率 P_1 和 P_2 ，按概率分配辅助流。

定义了感知参数 $\theta(Theta)$ 。我进行了敏感性分析：当 θ 很大时，SUE 结果逼近 FW 算法（趋于理性）；当 θ 很小时，结果趋于平均分配（趋于随机）。

13.4 代码链接

[MicroCity Web](#)

13.5 结果展示图

```
[2025/12/9 19:55:46]>=== 交通流分配算法对比 (总需求 Q=100.0) ===
[2025/12/9 19:55:46]>>>> 开始计算：增量分配法 (分100次分配)
[2025/12/9 19:55:46]> 分配结果: 路段1流量=81.0000, 路段2流量=19.0000
[2025/12/9 19:55:46]> 最终阻抗: 路段1阻抗=203.3121, 路段2阻抗=200.0391
[2025/12/9 19:55:46]>>>> 开始计算：连续平均法 (MSA - UE, 迭代200次)
[2025/12/9 19:55:46]> 分配结果: 路段1流量=80.0000, 路段2流量=20.0000
[2025/12/9 19:55:46]> 最终阻抗: 路段1阻抗=198.3040, 路段2阻抗=200.0480
[2025/12/9 19:55:46]>>>> 开始计算：Frank-Wolfe 算法 (UE - 标准最精确)
[2025/12/9 19:55:46]> (在第 2 次迭代收敛)
[2025/12/9 19:55:46]> 分配结果: 路段1流量=80.3518, 路段2流量=19.6482
[2025/12/9 19:55:46]> 最终阻抗: 路段1阻抗=200.0447, 路段2阻抗=200.0447
[2025/12/9 19:55:46]>=== 以下为创新算法对比 ===
[2025/12/9 19:55:46]>>>> [创新点] 开始计算：随机用户均衡 (SUE-Logit, theta=0.1)
[2025/12/9 19:55:46]> 分配结果: 路段1流量=77.7224, 路段2流量=22.2776
[2025/12/9 19:55:46]> 最终阻抗: 路段1阻抗=187.5783, 路段2阻抗=200.0739
[2025/12/9 19:55:46]> (解释: SUE结果中阻抗不完全相等，因为包含了用户的'感知误差')
[2025/12/9 19:55:46]>=== 计算结束 ===
```

14 作业 14

问题：在一个有 3*2 储位空间的仓库中，一个 AGV 从 D1 出发上下右（禁左）随机走行，除了从 D2 出发的走行时间为 2，其他走行时间均为 1，到 U3 停止。可视化 AGV 随机走行动作。打印所有随机路径走行时间。找到最短路径。

本次作业，我首先完成老师课上所带领分析的题目，随后，将网格拓展，做了个小创新。

14.1 基础版：3x2 网络的随机游走与最短路

14.1.1 代码思路

在基础版中，我严格按照老师上课所讲还原了场景和逻辑。

首先，进行场景搭建：

构建了 3x2 的网格。采用邻接表的方式定义了 U1-U3 和 D1-D3 的连接关系。在邻接表中，根据老师题目要求，达到禁左的规则，我没有定义“向左”的连接（例如 D2 的邻居只有 U2 和 D3）。

接下来，编写了 get_travel_cost 函数，对起点为 D2 的移动赋予 2.0s 耗时，其余为 1.0s。

最短路计算：

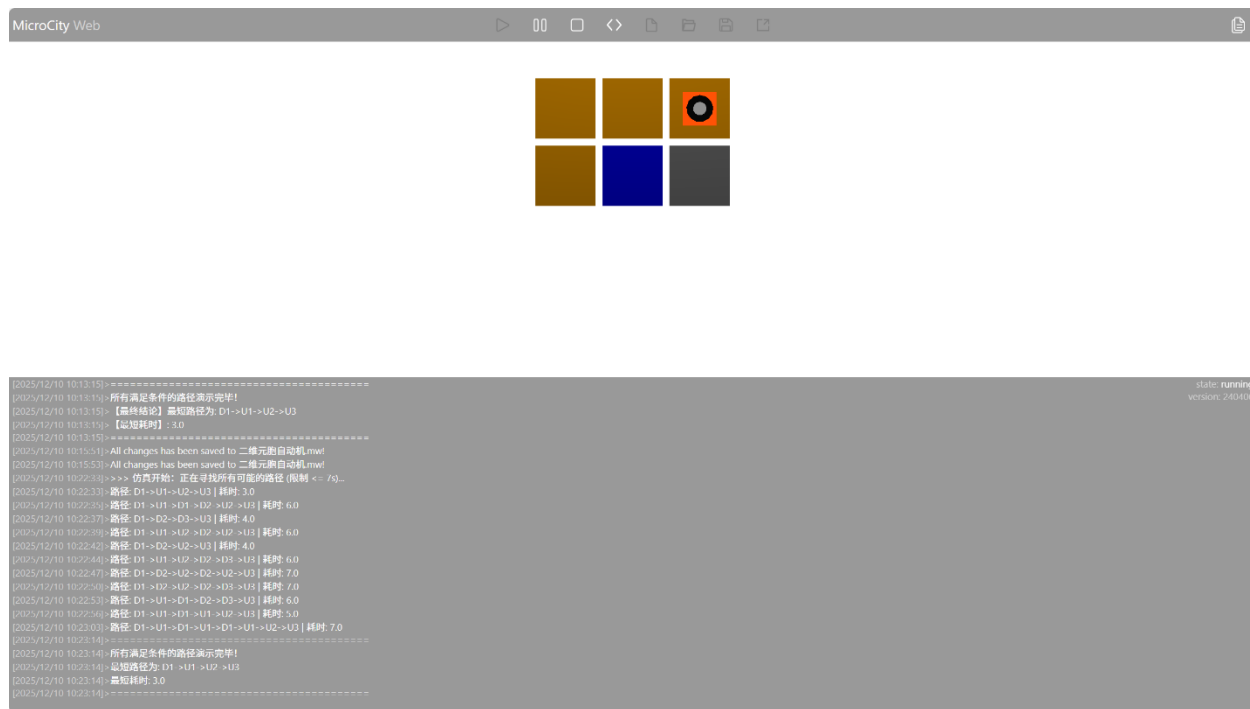
使用 Dijkstra 算法算出理论最短路径（D1->U1->U2->U3, 耗时 3.0s）作为参考标准。

仿真策略：采用随机游走策略。AGV 在每个节点随机选择下一个可行邻居，模拟 AGV 在无导航指令下的随机探索行为。

14.1.2 代码链接

[MicroCity Web](#)

14.1.3 结果展示图



14.2 创新版：4x4 网格的全路径遍历与可视化

考虑到 3x2 网格路径较少，不能体现出我对这段代码编写的用心程度，所以我在创新版中将系统复杂度进行了升级。

14.2.1 代码编写逻辑

不再像基础版当中那样，手动硬编码坐标（对每一个坐标挨个定义），而是编写了双层循环，自动生成 4x4（16 个节点）的网格。

编写了通用的邻接生成逻辑，自动判断边界，生成支持“上、下、右”移动的图结构。这使得代

码具有极强的通用性，未来扩展到 10x10 网格也无需修改核心逻辑。

在算法方面也做了升级，从随机游走到 DFS 全遍历：

- 因为 4×4 的网格，如果随机游走，对路径的遍历的重复度会很高，这样就大大降低了我们的效率，所以我就放弃随机游走。我想到了，前几次作业当中所完成的 DFS，也就是实用深度优先遍历。
- 利用递归与回溯，强制找出从起点（ID 13）到终点（ID 4）的所有不重复路径。

排序展示：将找到的所有路径按总耗时从高到低排序，演示时先播放慢的路径，最后展示最短路径。

状态机控制：

引入了 READY -> PLAYING -> NEXT -> FINISHED 的状态机模式，能够流畅地连续播放多条路径的演示动画，而不是跑完一次就停止。

14.2.2 核心创新点

创新点一：将网格从 32 拓展到 44，增加了代码复杂度，深刻去感悟元胞自动机的原理。

创新点二：因为特殊格子的走行时间是 2，所以我定义了特殊的物理减速区，也就是 ID 7 和 ID 10，可视化为蓝色方块。

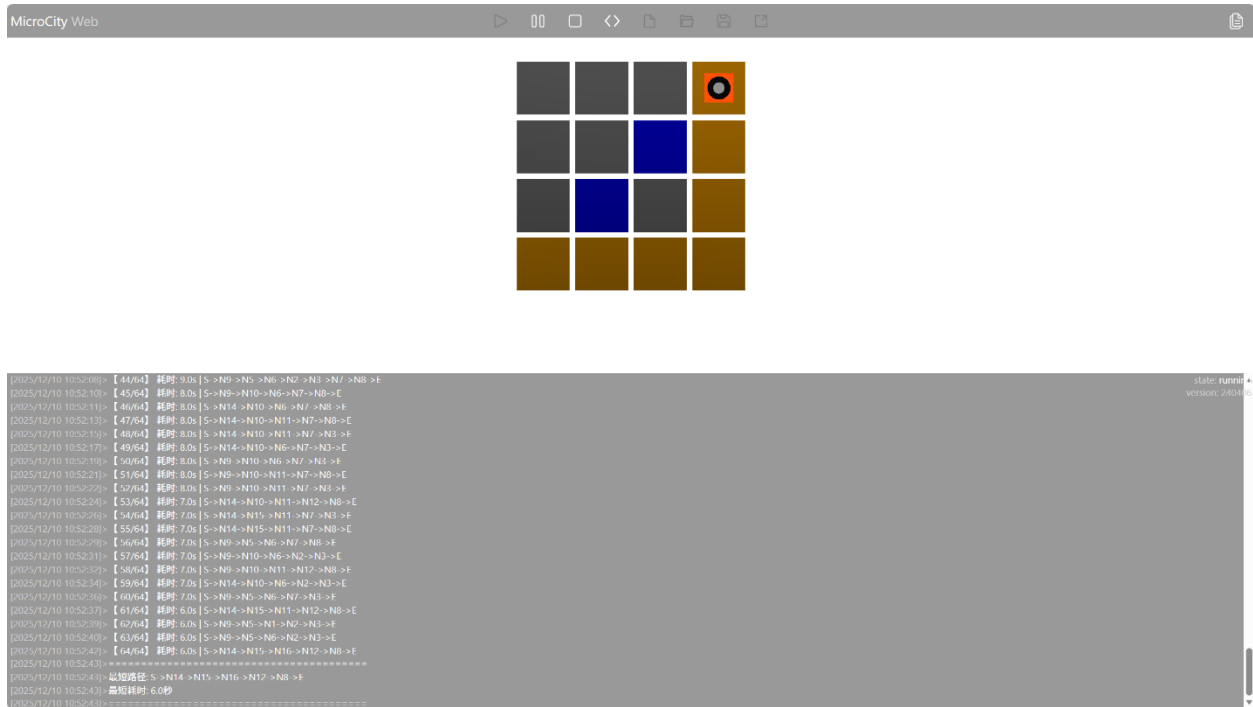
创新点三：在 32 中，我严格落实了老师的要求：让 AGV 随机行走，但是在 44 的网格中，若是随机行走就不一定可以遍历出所有的路径，如果想遍历出所有的路径，需要大量的模拟次数，所以我先使用了前几次作业所完成的深度优先遍历算法。

创新点四：可视化部分的小创新，实现了一个足迹追踪功能，AGV 每走过一个格子，格子就会变色，把路径展示出来，直接看出每条路径的行走差异。

14.2.3 代码链接

[MicroCity Web](#)

14.2.4 结果展示图



15 作业 15

问题：在一个有 3*2 储位空间的仓库中，一个 AGV 从 D1 出发上下右（禁左）随机走行，除了从 D2 出发的走行时间为 2，其他走行时间均为 1，D2 为湿滑地面，从 D1 到 D2 的 AGV 有 70% 的概率直接滑到 D3，用时 1.5。问：1. 编程实现贝尔曼最优方程。2. 可视化 AGV 最优动作策略。

本次作业我分为了基础复现和进阶创新两个部分来完成。

15.1 基础版

15.1.1 代码编写逻辑

首先进行场景搭建，这一部分的逻辑与作业 14 的思路一致，这里不做过多赘述。

我在代码中完美复现了老师课上所讲的贝尔曼算法。建立了 6 个节点的状态空间 S 。

我的代码逻辑完全对应课件中的公式：

$$V^*(s_t) = \max_{a_t} E[R_{t+1} + \gamma V^*(s_{t+1})]$$

具体实现分为三层逻辑：

- 第一层：状态价值的递归

在代码中，我定义了 V 表来存储状态价值。对于每一个非终点状态 s ，我遍历其所有动作 a ，计算 $R + \gamma V(s_{next})$ 。这对应了课件中提到的 $v(i) = \min\{c_{ij} + v(j)\}$ 的递归思想。

- 第二层：随机性与期望值计算

针对 D1 节点的特殊情况，题目要求 D1 到 D2 有 70% 的概率滑到 D3。在代码中，我没有简单相加，而是按照期望值公式进行编写：

```
-- 策略B: 向右走 (随机性)
-- 期望价值 = 30% * (R=-1 + γ*V(D2)) + 70% * (R=-1.5 + γ*V(D3))
local v_right = 0.3 * (-1.0 + GAMMA * V[5]) + 0.7 * (-1.5 + GAMMA * V[6])
if v_right > max_v then max_v = v_right end
end
```

这段代码复现了老师课上所讲的 $-4 \times 0.3 - 2.5 \times 0.7 = -2.95$ 这一关键计算步骤，确保了机器决策是基于长期期望的回报的最大化，而不是短期的贪婪选择。

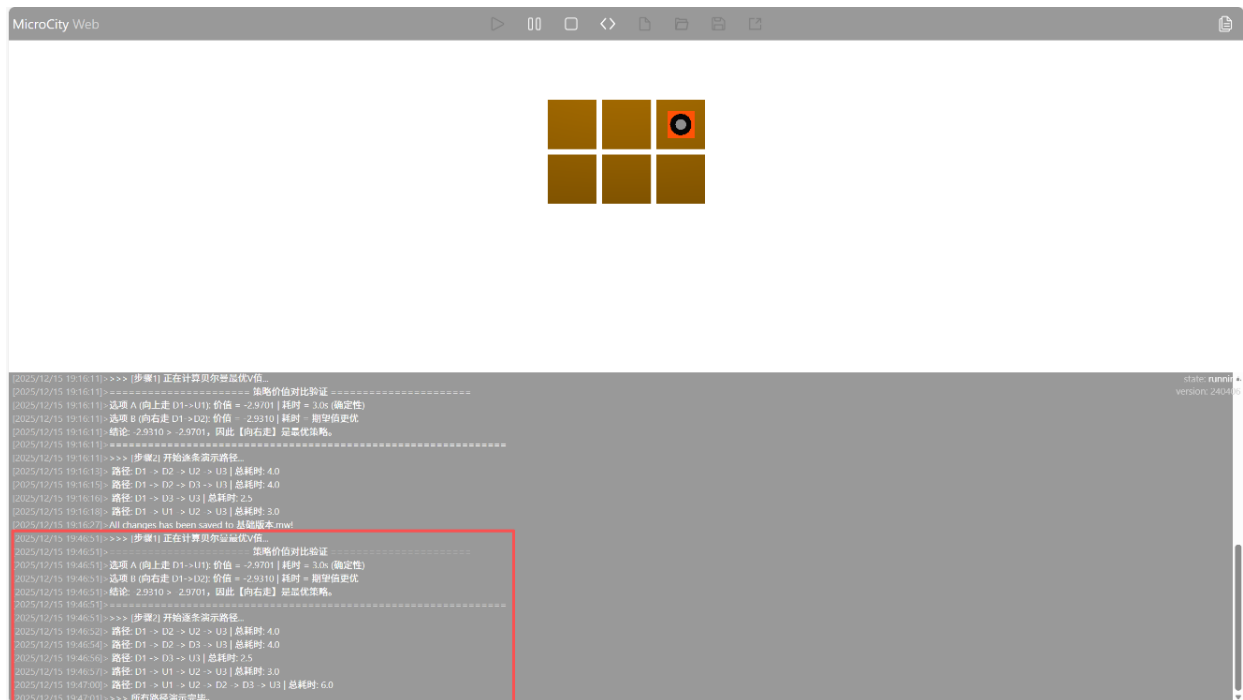
- 第三层：值迭代

为了求解这个递归方程，我采用了固定增加时间推进的仿真思想，在代码中构建了一个 while 循环，不断更新V表，直到 delta（新旧价值的差值）收敛于 0。这模拟了动态规划中信息从终点向起点反向传播的过程。

15.1.2 代码链接

[MicroCity Web](#)

15.1.3 结果展示图



代码计算得出，尽管向右走存在风险，但其期望价值依然优于向上绕行。代码最终演示的最优策略与课上结论完全一致，验证了理论公式的正确性。

因为远期利益折扣取值为 0.99，所以最后价值并不等于 2.95。

15.2 创新版

考虑到 3x2 网格路径较少，难以体现算法的优越性，我在创新版中将系统复杂度进行了升级。

15.2.1 代码逻辑

代码编写逻辑大致与基础版一致，先进行价值判断，加了 DFS 对所有路径遍历，我设置了，只有 AGV 在蓝色格子左侧的格子时，才会触发概率效果。

15.2.2 创新点

创新点一：为了直观展示强化学习如何做决策，我在仿真前增加了一个模块，显式打印出节点 6 和 9 的 $Q(s, a)$ 值。

我计算了 $Q(s, \text{Right})$ 的期望值与 $Q(s, \text{Up})$ 的确定性值。这验证了课件中提到的动作价值函数 $Q^*(s, a)$ 的概念 9，让黑盒的决策过程变得透明可见。

创新点二：随机行走针对 4×4 的节点并不好用，因为线路太多，如果想遍历所有线路就要进行大量的模拟仿真，无法体现概率的全貌。为了验证模型，我利用 DFS 算法实现了全概率空间遍历：

- 逻辑：每当 AGV 遇到概率节点，代码就会分裂成两个选择——一个演示“跳跃（70%）”，一个演示“打滑（30%）”。
- 结果：程序最终遍历并记录了 72 条不同的逻辑路径。这不仅验证了贝尔曼方程的最优解，也模拟系统在随机环境下的所有可能演化路径。

创新点三：可视化的交互反馈为了让概率结果“看得见”，我利用 Lua 脚本实现了动态动画：如果打滑，就不会留下路径颜色，如果被减速，就会减缓行走时间。为了增强仿真的可视化效果，我实现了足迹追踪功能。AGV 每走过一个格子，该格子会即时变为橙色。这不仅让路径走向一目了然，更方便观察者对比不同路径在空间分布上的差异（例如有的路径绕大圈，有的路径走对角线）。

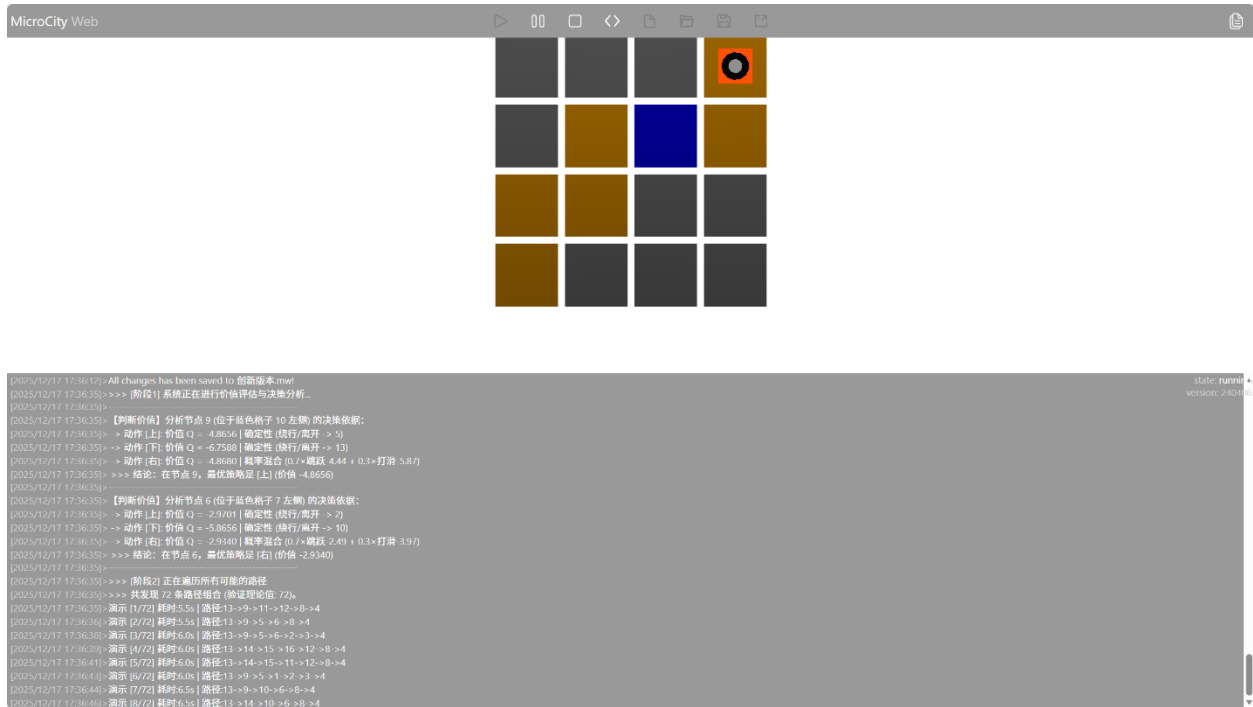
这对应了老师前几节课所提到的“时间推进法”在 MicroCity 中的具体实现，通过 `scene.render()` 每一帧精确控制对象的状态。

创新点四：拓展了节点为 4×4 ，加了一个概率方块，证实贝尔曼算法的先进。

15.2.3 代码链接

[MicroCity Web](#)

15.2.4 结果展示图



16 作业 16

问题：使用蒙特卡洛方法计算有湿滑地面的仓库 AGV 最优动作策略；使用 SARSA 算法计算有湿滑地面的仓库 AGV 最优动作策略。

针对蒙特卡洛方法和时序差分算法的练习，我先完成老师课上所讲的 3×2 网格求解，随后在此基础上，完成 4×4 复杂地图展示运行设计并将两种算法结合进行了对比。

16.1.1 代码逻辑

首先，按照前几次作业中同样的逻辑，把仓库网格复现，其中的方块展示形式以及 AGV 形式、运行形式都与之前作业保持一致。

(1). 蒙特卡洛方法实现：

- 根据老师上课所讲，我理解的蒙特卡洛方法为，通过大量的随机的模拟，记录每次完整的状态序列，利用 Gt 计算平均回报来更新状态价值 $V(s)$ 。
- 在这个过程当中，我使用了状态机（INIT, DECIDE, MOVE, CALC）来控制流程。在 DECIDE 阶段，AGV 随机选择路径，遇到 D2 节点时通过 $\text{math.random}() < 0.3$ 判断是否打滑。

(2). SARSA 算法实现：

我认为 SARSA 算法相比于蒙特卡洛算法优越性在于可以根据前面走过的情况去影响后面的走向。我利用老师上课所讲公式

$$\bar{Q}^k(s_t, a_t) = \bar{Q}^{k-1}(s_t, a_t) + \alpha \left[R_{t+1} + \gamma \bar{Q}^k(s_{t+1}, a_{t+1}) - \bar{Q}^{k-1}(s_t, a_t) \right]$$

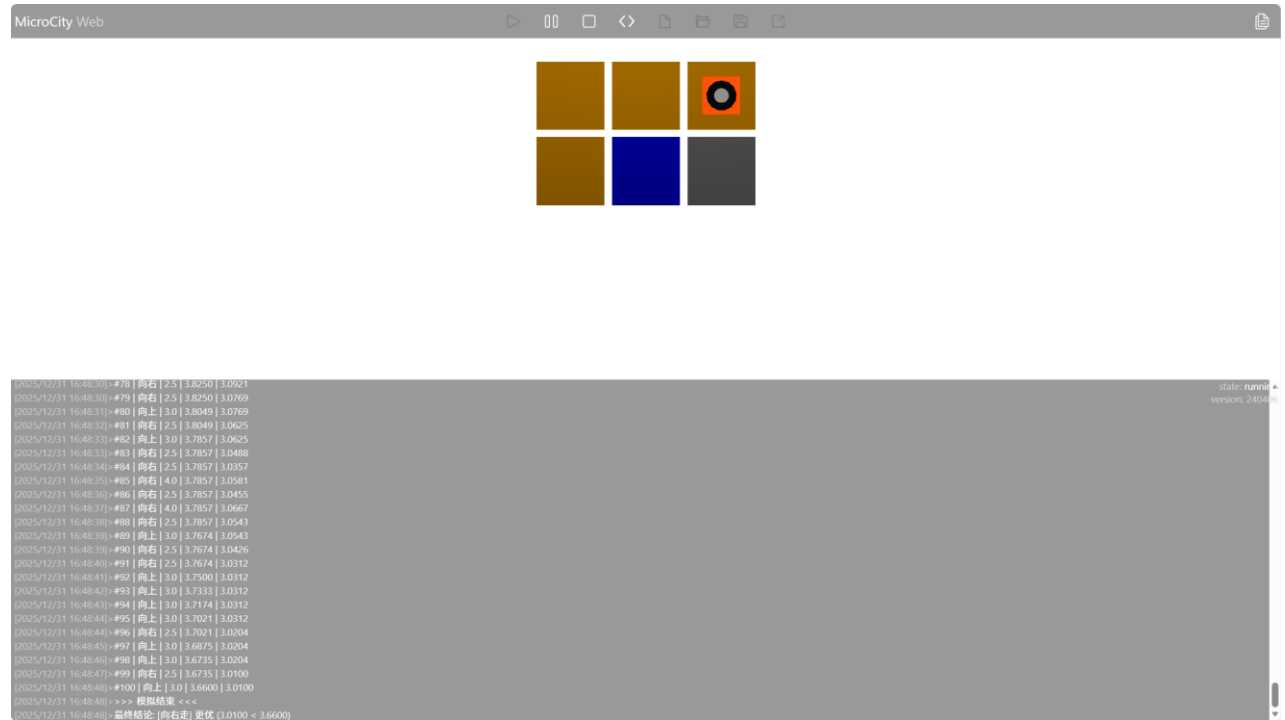
进行单步更新。我认为这个算法可以实现贪婪策略，以 $\text{EPSILON} = 0.1$ 的概率随机探索，其余时间选择 Q 值最小（耗时最少）的动作

16.1.2 代码链接

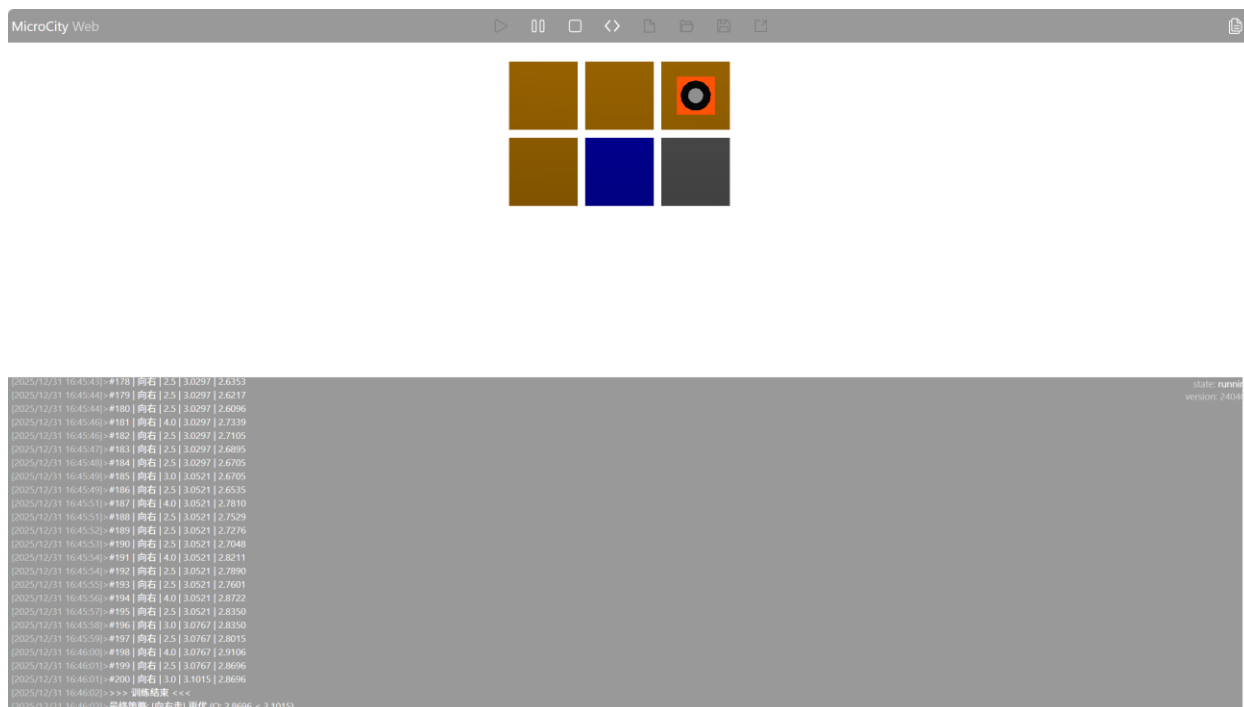
蒙特卡洛代码链接：[MicroCity Web](#)

SARSA 代码链接：[MicroCity Web](#)

16.1.3 蒙特卡洛结果展示图



16.1.4 SARSA 结果展示图



16.2 创新版一

我认为简单的 3×2 的网格并不能体现两种方法的优越性，也不能对两种方法达到一个比照的效果，因此我同前几次作业一样构建一个 4×4 的空间地图，并且再加一个概率方块（蓝色的方块）。

16.2.1 代码逻辑

对于场景的搭建，以及格子之间的关系，AGV 的运行逻辑等都与前几次作业保持一致，关键在于保证两种方法在更庞大的场景同样适用，验证其可推广性。

(1). 蒙特卡洛方法：

在蒙特卡洛方法的代码当中，记录了访问日志 `visit_log`，采用增量式更新公式计算每个关键节点的价值。

(2). SARSA 算法：

初始化更大的 Q 表，针对每个节点的方向维护独立的 Q 值和访问次数。

16.2.2 创新点

创新点一：构建 4×4 复杂场景，验证方法的可推广性。

为了验证代码不仅仅适用于简单的环境（ 3×2 网格），我重构了地图生成逻辑，将其扩展为 4×4 （16 节点）的庞大场景，并且加了一个蓝色概率方块。

结果证明了强化学习算法（无论是蒙特卡洛还是 SARSA）在状态空间指数级增加时，依然能够收敛并找到最优策略。

创新点二：先将所有路径遍历再探索，而非随机行走

若是完全随机行走，得出结果并不准确。所以我改变了基础作业中简单的 if-else 判断，结合我们运筹学的知识，引入了图论中的邻接表的思想。在模拟开始前，代码首先遍历所有 16 个节点，根据行列关系动态构建了邻接关系图

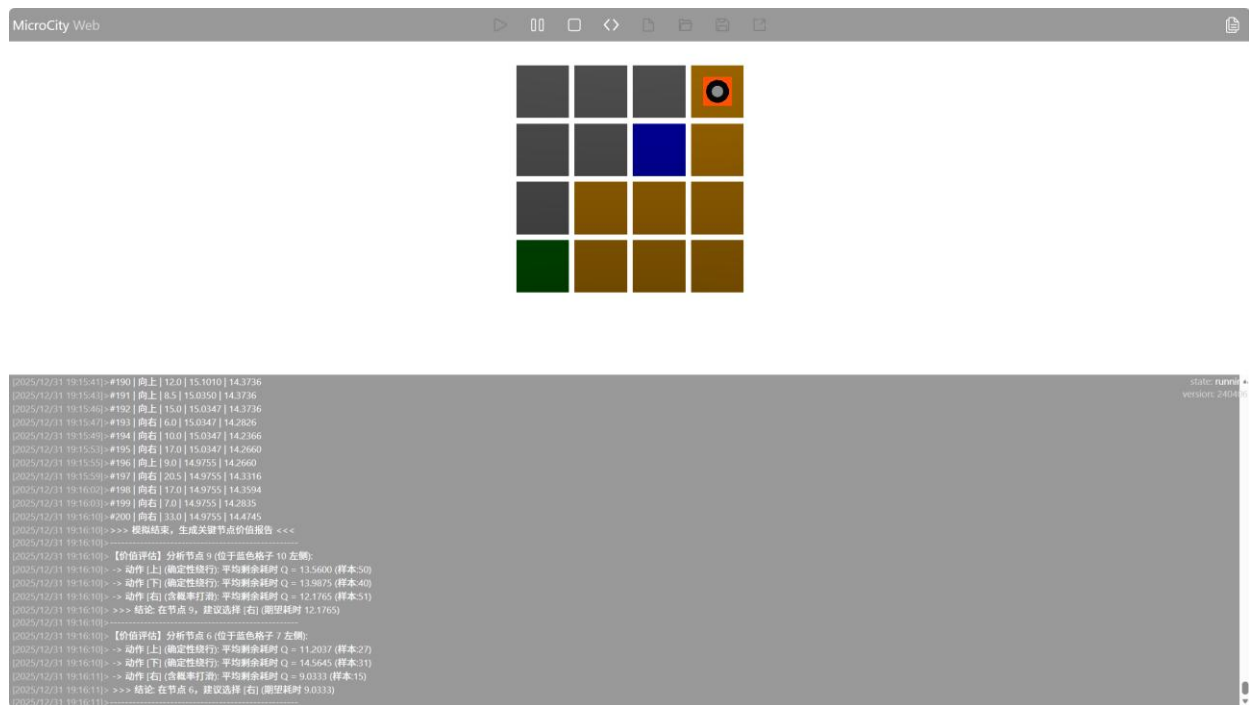
16.2.3 蒙特卡洛代码链接

[MicroCity Web](#)

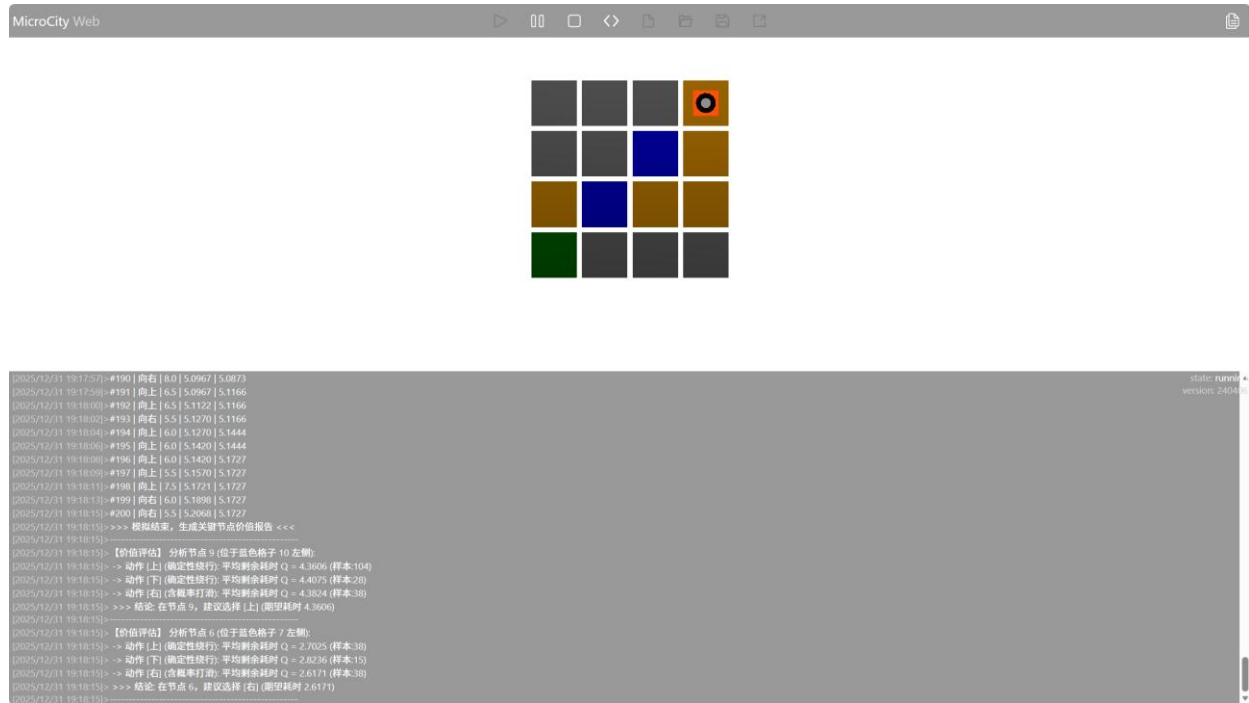
16.2.4 SARSA 代码链接

[MicroCity Web](#)

16.2.5 蒙特卡洛结果展示图



16.2.6 SARSA 结果展示图



16.3 创新版二

老师在上课时列出了一个 excel 表格，表格当中是各种方法迭代的对比，因此我把两种方法结合起来共同模拟仿真，以直观对比两种方法的收敛特性。

16.3.1 代码逻辑

首先，将每一部分阶段化，防止混乱。引入一个变量去控制仿真进程：阶段 1 为蒙特卡洛，阶段 2 为 SARSA，最后再进行总结报告。

先将蒙特卡洛运行 100 轮，其中蒙特卡洛的运算逻辑与代码与上述一致。再把 SARSA 运行 200 轮，利用公式实时更新 Q 表。

我发现，SARSA 能够相比于蒙特卡洛更早的利用“自举”特性调整策略。而且，再蒙特卡洛中，AGV 必须走完整个路径才能更新数据；而 SARSA，AGV 每走一步，Q 值就会发生变化。

16.3.2 创新点

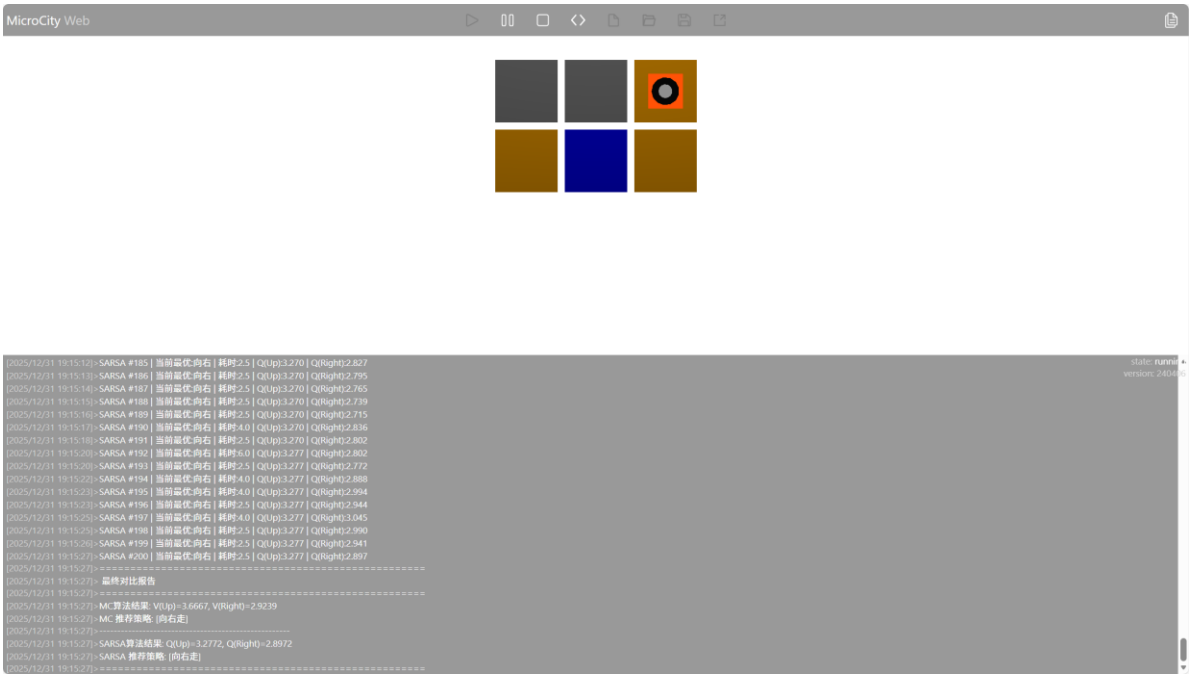
创新点三：构建“蒙特卡洛 + SARSA”融合对比框架，直观展示算法差异

这种创新可以将算法横向对比，突出 SARSA 的优越性，深刻理解两种无模型强化学习方法的区别。

16.3.3 代码链接

[MicroCity Web](#)

16.3.4 结果展示图



16.4 结论与思考

蒙特卡洛方法的方差比较大，在 4×4 地图中，如果随机性导致某次绕路极远，会显著拉高该路径的平均成本，收敛速度较慢，SARSA 利用了后继状态的估计值，方差较小。在我的仿真中，SARSA 能更快地锁定“向右走”优于“向上走”的策略，特别是在处理 D2 这种具有随机惩罚的节点时，表现更加稳定。

总体感想与思考

经过一学期的努力，看到文件夹中整整齐齐摆放 16 个文件，我最大的感想就是特别有成就。

我觉得，在孙老师的这门课上，我不止学会了怎么样在 MicroCity 中调用 lua 去完成各种任务，实现各种算法模型，更多是学会了对问题解决过程的逻辑。比如在解决 VRP 问题的部分，我除了学会了如何消除子回路的数学模型、CW 节约算法等等，我觉得我学会的、最重要的是，遇到一个 VRP 问题的解决逻辑，我记得老师上课时的循序渐进：如何推导出所有约束，如何写出伪代码，如何写出算法逻辑图等等。真正刻在我脑海中的反而不是那些固定的算法模型，反而是这种解决问题的逻辑。

一开始做创新，就是为了加分。到后面的作业，越做越爱做创新，这是一种探索的感觉，虽然我有很多做错，有很多不符合逻辑，但是老师仍然在积极的帮我解决我的问题。我记得在作业 12 当中，也就是实现连续最短路径法，我就不论如何都想不明白，线路的权重怎么给配置，随后经过老师在实验课的指点，我才把这一块难点突破。作业中有大部分的创新是依据老师的思想，拓展出来的，也有一部分是完全个人想象。我认为我做出的创新，可以让我更好的理解老师课上所讲的内容，比如：作业 8（Dijkstra 算法）和作业 16（强化学习），我做出的创新可以让我体会到，当实际情况变得复杂难解，算法的优越性就可以体现出来了，在这样的代码不断报错，不断修改的过程中，我才真正的弄明白算法的逻辑思维。

在之前的课程，也涉及过代码算法，最多的就是调用 python 的 Gurobi，也就是在 python 中调用一个库去解决问题。这样的过程我认为是简单的，写出模型公式就能实现了。但是在本课程中，我才能更深刻的体会算法的逻辑。从第 3 章算设施选址的迭代重心法，到第 5 章算网络流的 FW 算法，再到最后这个机器学习。我能体会到，把每行公式输入进去，出现什么效果，去理解什么叫迭代，什么叫收敛。

在平时我并没有感觉到，但是在写汇总作业的时候，我能够深刻的体会到，本门课程的逻辑闭环，学习的循序渐进。如果不完成前面的课程作业，没把前面的知识点吃透，是无法完成后面的作业的。比如，在中期学习的深度优先遍历，我在后面的作业 14，15，16 中的创新版就利用上了，当时在构思创新点时，我的思路就是，怎么样能体现算法的优越呢？那我就扩充地图，然后我就想，扩充地图之后的路径太多了，很难依靠随机遍历完成所有路径的遍历，我就想到了之前所学习过的深度优先遍历，先把所有路径遍历出来，然后做一个可视化去演示，然后展示出这个时间的长度的对比，再去算后续的价值，惩罚等等。所以说，在这门课程当中，老师的课程设计在不知不觉中，推动着我去完成课程作业，课程作业的创新。

最后我要重点去提到我们对于强化学习部分的学习，因为我经常都是听说强化学习，虽然自己尝试过去学习一部分，去看一些关于强化学习的书。但是真如老师课上所说，强化学习这个东西，你自学的话，也许看了很久，也不知道这个东西到底是什么意思，但是经过老师一个精妙的设计，去给我

讲解一下，就基本会给这个东西吃透。属实老师让我快速的掌握了蒙特卡洛、时序差分、SARSA。这样与时俱进的课程设计，属实可以让我学到更多的、当下比较热东西。这门课程可以说，既扎实了基础，又进行了拔高。

我要感谢老师这学期的教学付出。老师不断的、实时的更新课件，设计与与时俱进的内容，在实验课上对我们的问题的耐心解答，站在我们学生的角度去考虑，帮助我们去协调课程，激励同学们去学习。特别要感谢老师对我所有问题的耐心解答，上到算法流程，下到 MicroClity 的换行符问题，甚至老师在 MicroClity 中帮助有写到过的内容，依然会给我耐心讲解，告诉我具体在那个位置。最重要的是感谢老师对我逻辑思维、创新思维的培养与促进，比如最短路约束，我记忆特别清晰，在上周刚考完的管理运筹学，最后一道题，是一个建模题，我就是运用了我们 VRP 问题所讲的数学模型，回忆老师上课所讲的推导流程，怎么想的才能把这个约束全都想出来了，最后把这道题完美的解决。

最后，我想提到的是关于老师所讲的一些思考。老师在课程快结束时，曾提到过，老师准备了大量的知识，有很多很多的东西想交给我们，但是并没有看到我们的斗志。同时在课上也有很多同学在汇报时，根本说不出来自己的内容，完全可以看出就是 AI 制作内容。我也承认我也会利用 AI 去完成我的课程作业，因为高效，因为会辅助我检查出我的错误。我个人认为，AI 对于这一类算法代码的作业，是一把双刃剑，在于怎么使用。在我的学习过程当中，我有一套完成范式：先认真听老师课上所讲内容，然后我会不看课件 ppt，去跟 AI 说，今天学了什么内容，整个算法流程是怎么下来的，我对于整个问题解决过程的理解。我会问 AI，你觉得我对于老师的讲解有什么误解吗？还有什么补充吗，然后我会想伪代码流程，比如如何构建起整个算法，然后进行可视化，写的尽可能的详细。随后让 AI 完成代码的撰写，我会读一遍整个代码，看一下是否完美复现了我的逻辑思维，我先不管他对错，我先看一下是否复现了我的逻辑思维。然后如果与我的思维相符合，我就回去运行，再问 AI 运行结果与老师课上所讲内容是否相似，不对的话，我的逻辑思维哪错了，为什么这么想不对，最后不断的依靠这种“提问式”使用范式，既增加效率，也弄明白算法逻辑，更重要的是完成了代码的复现作业。然后对于创新的创作，我也会先问一下 AI，我这样做，不可行，有没有价值，然后整个流程同刚才一致。我觉得这样对于 AI 的使用，是利大于弊的，让我能够有更多的思考，更高的效率。所以说，我认为，AI 的好坏，在于使用者的使用模式。

最后，再次感谢老师对于学生的全方位培养付出，我会带着本课程所学习到的所有逻辑、知识，继续努力，在科研道路上深耕。